

SECTION 7

BUS OPERATION

This section provides a functional description of the bus, the signals that control it, and the bus cycles provided for data transfer operations. It also describes the error and halt conditions, bus arbitration, and the reset operation. Operation of the bus is the same whether the processor or an external device is the bus master; the names and descriptions of bus cycles are from the point of view of the bus master. For exact timing specifications, refer to **SECTION 13 ELECTRICAL CHARACTERISTICS**.

The MC68030 architecture supports byte, word, and long-word operands, allowing access to 8-, 16-, and 32-bit data ports through the use of asynchronous cycles controlled by the data transfer and size acknowledge inputs (DSACK0 and DSACK1).

Synchronous bus cycles controlled by the synchronous termination signal (STERM) can only be used to transfer data to and from 32-bit ports.

The MC68030 allows byte, word, and long-word operands to be located in memory on any byte boundary. For a misaligned transfer, more than one bus cycle may be required to complete the transfer, regardless of port size. For a port less than 32 bits wide, multiple bus cycles may be required for an operand transfer due to either misalignment or a port width smaller than the operand size. Instruction words and their associated extension words must be aligned on word boundaries. The user should be aware that misalignment of word or long-word operands can cause the MC68030 to perform multiple bus cycles for the operand transfer; therefore, processor performance is optimized if word and long-word memory operands are aligned on word or long-word boundaries, respectively.

7.1 BUS TRANSFER SIGNALS

The bus transfers information between the MC68030 and an external memory, coprocessor, or peripheral device. External devices can accept or provide 8 bits, 16 bits, or 32 bits in parallel and must follow the handshake protocol described in this section. The maximum number of bits accepted or provided during a bus transfer is defined as the port width. The MC68030 contains an

address bus that specifies the address for the transfer and a data bus that transfers the data. Control signals indicate the beginning of the cycle, the address space and the size of the transfer, and the type of cycle. The selected device then controls the length of the cycle with the signal(s) used to terminate the cycle. Strobe signals, one for the address bus and another for the data bus, indicate the validity of the address and provide timing information for the data.

The bus can operate in an asynchronous mode identical to the MC68020 bus for any port width. The bus and control input signals used for asynchronous operation are internally synchronized to the MC68030 clock, introducing a delay. This delay is the time period required for the MC68030 to sample an asynchronous input signal, synchronize the input to the internal clocks of the processor, and determine whether it is high or low. Figure 7-1 shows the relationship between the clock signal and the associated internal signal of a typical asynchronous input.

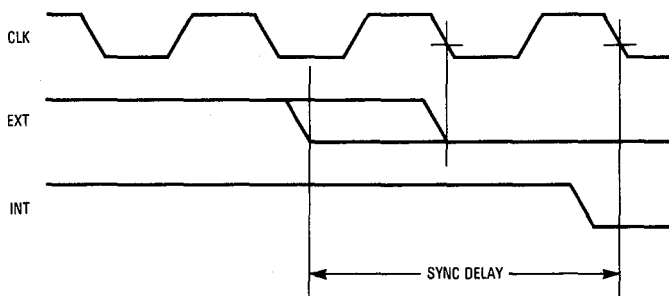


Figure 7-1. Relationship between External and Internal Signals

Furthermore, for all asynchronous inputs, the processor latches the level of the input during a sample window around the falling edge of the clock signal. This window is illustrated in Figure 7-2. To ensure that an input signal is recognized on a specific falling edge of the clock, that input must be stable during the sample window. If an input makes a transition during the window time period, the level recognized by the processor is not predictable; however, the processor always resolves the latched level to either a logic high or low before using it. In addition to meeting input setup and hold times for deterministic operation, all input signals must obey the protocols described in this section.

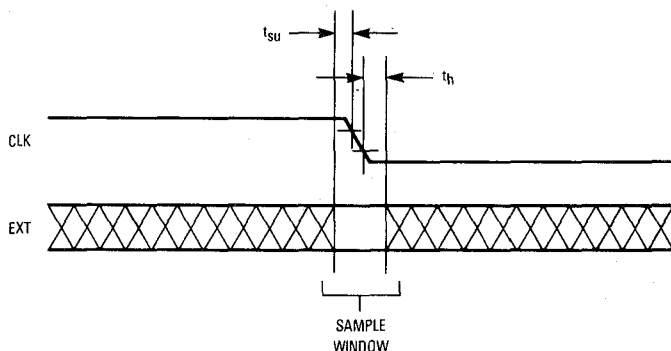


Figure 7-2. Asynchronous Input Sample Window

A device with a 32-bit port size can also provide a synchronous mode transfer. In synchronous operation, input signals are externally synchronized to the processor clock, and the synchronizing delay is not incurred.

Synchronous inputs ($\overline{\text{STERM}}$, $\overline{\text{CBACK}}$, and $\overline{\text{CIIN}}$) must remain stable during a sample window for all rising edges of the clock during a bus cycle (i.e., while address strobe ($\overline{\text{AS}}$) is asserted), regardless of when the signals are asserted or negated, to ensure proper operation. This sample window is defined by the synchronous input setup and hold times (see MC68030EC/D, *MC68030 Electrical Specifications*).

7.1.1 Bus Control Signals

The external cycle start ($\overline{\text{ECS}}$) signal is the earliest indication that the processor is initiating a bus cycle. The MC68030 initiates a bus cycle by driving the address, size, function code, read/write, and cache inhibit-out outputs and by asserting $\overline{\text{ECS}}$. However, if the processor finds the required program or data item in an on-chip cache, if a miss occurs in the address translation cache (ATC) of the memory management unit (MMU), or if the MMU finds a fault with the access, the processor aborts the cycle before asserting $\overline{\text{AS}}$. $\overline{\text{ECS}}$ can be used to initiate various timing sequences that are eventually qualified with $\overline{\text{AS}}$. Qualification with $\overline{\text{AS}}$ may be required since, in the case of an internal cache hit, an ATC miss, or an MMU fault, a bus cycle may be aborted after $\overline{\text{ECS}}$ has been asserted. The assertion of $\overline{\text{AS}}$ ensures that the cycle has not been aborted by these internal conditions.

During the first external bus cycle of an operand transfer, the operand cycle start ($\overline{\text{OCS}}$) signal is asserted with $\overline{\text{ECS}}$. When several bus cycles are required

to transfer the entire operand, $\overline{OCS}$ is asserted only at the beginning of the first external bus cycle. With respect to $\overline{OCS}$, an “operand” is any entity required by the execution unit, whether a program or data item.

The function code signals (FC0–FC2) are also driven at the beginning of a bus cycle. These three signals select one of eight address spaces (refer to Table 4-1) to which the address applies. Five address spaces are presently defined. Of the remaining three, one is reserved for user definition and two are reserved by Motorola for future use. The function code signals are valid while $\overline{AS}$ is asserted.

At the beginning of a bus cycle, the size signals (SIZ0 and SIZ1) are driven along with $\overline{ECS}$ and the FC0–FC2. SIZ0 and SIZ1 indicate the number of bytes *remaining to be transferred during an operand cycle (consisting of one or more bus cycles) or during a cache fill operation from a device with a port size that is less than 32 bits.* Table 7-2 shows the encoding of SIZ0 and SIZ1. These signals are valid while $\overline{AS}$ is asserted.

The read/write ($R/\overline{W}$) signal determines the direction of the transfer during a bus cycle. This signal changes state, when required, at the beginning of a bus cycle and is valid while $\overline{AS}$ is asserted. $R/\overline{W}$ only transitions when a write cycle is preceded by a read cycle or vice versa. The signal may remain low for two consecutive write cycles.

The read-modify-write cycle signal ($\overline{RMC}$) is asserted at the beginning of the first bus cycle of a read-modify-write operation and remains asserted until completion of the final bus cycle of the operation. The $\overline{RMC}$ signal is guaranteed to be negated before the end of state 0 for a bus cycle following a read-modify-write operation.

7.1.2 Address Bus

The address bus signals (A0–A31) define the address of the byte (or the most significant byte) to be transferred during a bus cycle. The processor places the address on the bus at the beginning of a bus cycle. The address is valid while $\overline{AS}$ is asserted.

7.1.3 Address Strobe

$\overline{AS}$ is a timing signal that indicates the validity of an address on the address bus and of many control signals. It is asserted one-half clock after the beginning of a bus cycle.

7.1.4 Data Bus

The data bus signals (D0–D31) comprise a bidirectional, nonmultiplexed parallel bus that contains the data being transferred to or from the processor. A read or write operation may transfer 8, 16, 24, or 32 bits of data (one, two, three, or four bytes) in one bus cycle. During a read cycle, the data is latched by the processor on the last falling edge of the clock for that bus cycle. For a write cycle, all 32 bits of the data bus are driven, regardless of the port width or operand size. The processor places the data on the data bus one-half clock cycle after $\overline{AS}$ is asserted in a write cycle.

7.1.5 Data Strobe

The data strobe ($\overline{DS}$) is a timing signal that applies to the data bus. For a read cycle, the processor asserts $\overline{DS}$ to signal the external device to place data on the bus. It is asserted at the same time as $\overline{AS}$ during a read cycle. For a write cycle, $\overline{DS}$ signals to the external device that the data to be written is valid on the bus. The processor asserts $\overline{DS}$ one full clock cycle after the assertion of $\overline{AS}$ during a write cycle.

7

7.1.6 Data Buffer Enable

The data buffer enable signal ($\overline{DBEN}$) can be used to enable external data buffers while data is present on the data bus. During a read operation, $\overline{DBEN}$ is asserted one clock cycle after the beginning of the bus cycle and is negated as $\overline{DS}$ is negated. In a write operation, $\overline{DBEN}$ is asserted at the time $\overline{AS}$ is asserted and is held active for the duration of the cycle. In a synchronous system supporting two-clock bus cycles, $\overline{DBEN}$ timing may prevent its use.

7.1.7 Bus Cycle Termination Signals

During asynchronous bus cycles, external devices assert the data transfer and size acknowledge signals ($\overline{DSACK0}$ and/or $\overline{DSACK1}$) as part of the bus protocol. During a read cycle, the assertion of $\overline{DSACKx}$ signals the processor to terminate the bus cycle and to latch the data. During a write cycle, the assertion of $\overline{DSACKx}$ indicates that the external device has successfully stored the data and that the cycle may terminate. These signals also indicate to the processor the size of the port for the bus cycle just completed, as shown in Table 7-1. Refer to **7.3.1 Asynchronous Read Cycle** for timing relationships of $\overline{DSACK0}$ and $\overline{DSACK1}$.

For synchronous bus cycles, external devices assert the synchronous termination signal ($\overline{\text{STERM}}$) as part of the bus protocol. During a read cycle, the assertion of $\overline{\text{STERM}}$ causes the processor to latch the data. During a write cycle, it indicates that the external device has successfully stored the data. In either case, it terminates the cycle and indicates that the transfer was made to a 32-bit port. Refer to **7.3.2 Asynchronous Write Cycle** for timing relationships of $\overline{\text{STERM}}$.

The bus error ($\overline{\text{BERR}}$) signal is also a bus cycle termination indicator and can be used in the absence of $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$ to indicate a bus error condition. It can also be asserted in conjunction with $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$ to indicate a bus error condition, provided it meets the appropriate timing described in this section and in MC68030EC/D, *MC68030 Electrical Specifications*. Additionally, the $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ signals can be asserted together to indicate a retry termination. Again, the $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ signals can be asserted simultaneously in lieu of or in conjunction with the $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$ signals.

Finally, the autovector ($\overline{\text{AVEC}}$) signal can be used to terminate interrupt acknowledge cycles, indicating that the MC68030 should internally generate a vector number to locate an interrupt handler routine. $\overline{\text{AVEC}}$ is ignored during all other bus cycles.

7.2 DATA TRANSFER MECHANISM

The MC68030 architecture supports byte, word, and long-word operands allowing access to 8-, 16-, and 32-bit data ports through the use of asynchronous cycles controlled by $\overline{\text{DSACK0}}$ and $\overline{\text{DSACK1}}$. It also supports synchronous bus cycles to and from 32-bit ports, terminated by $\overline{\text{STERM}}$. Byte, word, and long-word operands can be located on any byte boundary, but misaligned transfers may require additional bus cycles, regardless of port size.

When the processor requests a burst mode fill operation, it asserts the cache burst request ($\overline{\text{CBREQ}}$) signal to attempt to fill four entries within a line in one of the on-chip caches. This mode is compatible with nibble, static column, or page mode dynamic RAMs. The burst fill operation uses synchronous bus cycles, each terminated by $\overline{\text{STERM}}$, to fetch as many as four long words.

7.2.1 Dynamic Bus Sizing

The MC68030 dynamically interprets the port size of the addressed device during each bus cycle, allowing operand transfers to or from 8-, 16-, and 32-bit ports. During an asynchronous operand transfer cycle, the slave device

signals its port size (byte, word, or long word) and indicates completion of the bus cycle to the processor through the use of the $\overline{\text{DSACKx}}$ inputs. Refer to Table 7-1 for $\overline{\text{DSACKx}}$ encodings and assertion results.

Table 7-1. $\overline{\text{DSACK}}$ Codes and Results

$\overline{\text{DSACK1}}$	$\overline{\text{DSACK0}}$	Result
H	H	Insert Wait States in Current Bus Cycle
H	L	Complete Cycle — Data Bus Port Size is 8 Bits
L	H	Complete Cycle — Data Bus Port Size is 16 Bits
L	L	Complete Cycle — Data Bus Port Size is 32 Bits

For example, if the processor is executing an instruction that reads a long-word operand from a long-word aligned address, it attempts to read 32 bits during the first bus cycle. (Refer to **7.2.2 Misaligned Operands** for the case of a word or byte address.) If the port responds that it is 32 bits wide, the MC68030 latches all 32 bits of data and continues with the next operation. If the port responds that it is 16 bits wide, the MC68030 latches the 16 bits of valid data and runs another bus cycle to obtain the other 16 bits. The operation for an 8-bit port is similar, but requires four read cycles. The addressed device uses the $\overline{\text{DSACKx}}$ signals to indicate the port width. For instance, a 32-bit device *a/ways* returns $\overline{\text{DSACKx}}$ for a 32-bit port (regardless of whether the bus cycle is a byte, word, or long-word operation).

Dynamic bus sizing requires that the portion of the data bus used for a transfer to or from a particular port size be fixed. A 32-bit port must reside on data bus bits 0–31, a 16-bit port must reside on data bus bits 16–32, and an 8-bit port must reside on data bus bits 24–31. This requirement minimizes the number of bus cycles needed to transfer data to 8- and 16-bit ports and ensures that the MC68030 correctly transfers valid data. The MC68030 always attempts to transfer the maximum amount of data on all bus cycles; for a long-word operation, it always assumes that the port is 32 bit wide when beginning the bus cycle.

The bytes of operands are designated as shown in Figure 7-3. The most significant byte of a long-word operand is OP0, and OP3 is the least significant byte. The two bytes of a word-length operand are OP2 (most significant) and OP3. The single byte of a byte-length operand is OP3. These designations are used in the figures and descriptions that follow.

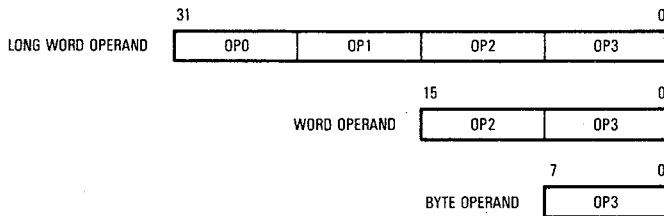


Figure 7-3. Internal Operand Representation

Figure 7-4 shows the required organization of data ports on the MC68030 bus for 8-, 16-, and 32-bit devices. The four bytes shown in Figure 7-4 are connected through the internal data bus and data multiplexer to the external data bus. This path is the means through which the MC68030 supports dynamic bus sizing and operand misalignment. Refer to **7.2.2 Misaligned Operands** for the definition of misaligned operand. The data multiplexer establishes the necessary connections for different combinations of address and data sizes.

The multiplexer takes the four bytes of the 32-bit bus and routes them to their required positions. For example, OP0 can be routed to D24–D31, as would be the normal case, or it can be routed to any other byte position to support a misaligned transfer. The same is true for any of the operand bytes. The positioning of bytes is determined by the size (SIZ0 and SIZ1) and address (A0 and A1) outputs.

The SIZ0 and SIZ1 outputs indicate the remaining number of bytes to be transferred during the current bus cycle, as shown in Table 7-2.

The number of bytes transferred during a write or noncachable read bus cycle is equal to or less than the size indicated by the SIZ0 and SIZ1 outputs, depending on port width and operand alignment. For example, during the first bus cycle of a long-word transfer to a word port, the size outputs indicate that four bytes are to be transferred, although only two bytes are moved on that bus cycle. Cachable read cycles must always transfer the number of bytes indicated by the port size.

A0 and A1 also affect operation of the data multiplexer. During an operand transfer, A2–A31 indicate the long-word base address of that portion of the operand to be accessed; A0 and A1 indicate the byte offset from the base. Table 7-3 shows the encodings of A0 and A1 and the corresponding byte offsets from the long-word base.

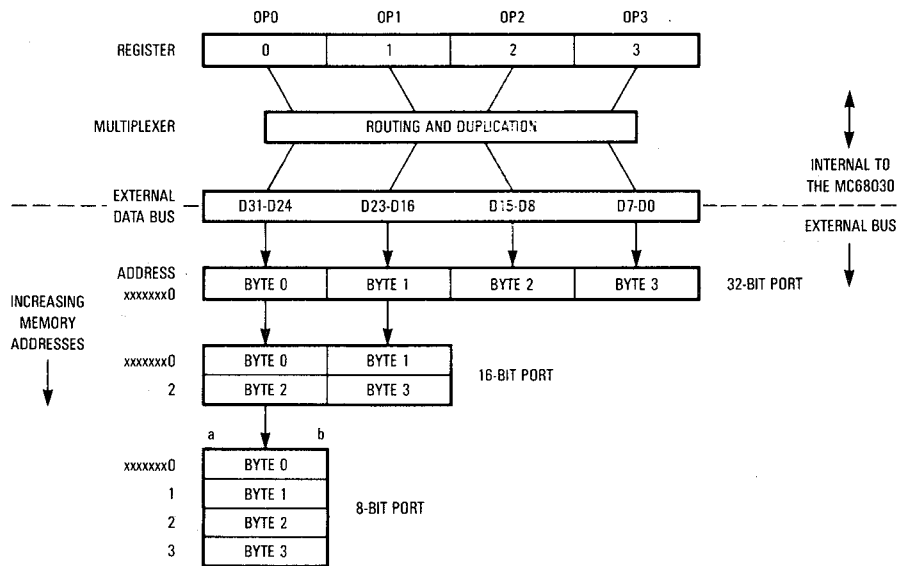


Figure 7-4. MC68030 Interface to Various Port Sizes

Table 7-4 lists the bytes required on the data bus for read cycles that are cachable. The entries shown as OPn are portions of the requested operand that are read or written during that bus cycle and are defined by SIZ0, SIZ1, A0, and A1 for the bus cycle. The PRn and the Nn bytes correspond to the previous and next bytes in memory, respectively, that must be valid on the data bus for the specified port size (long word or word) so that the internal caches operate correctly. (For cachable accesses, the MC68030 assumes that all portions of the data bus for a given port size are valid.) This same table applies to noncachable read cycles except that the bytes labeled PRn and Nn are not required and can be replaced by “don’t cares”.

Table 7-2. Size Signal Encoding

SIZ1	SIZ0	Size
0	1	Byte
1	0	Word
1	1	3 Bytes
0	0	Long Word

Table 7-3. Address Offset Encodings

A1	A0	Offset
0	0	+0 Bytes
0	1	+1 Byte
1	0	+2 Bytes
1	1	+3 Bytes

Table 7-4. Data Bus Requirements for Read Cycles

Transfer Size	Size		Address		Long-Word Port External Data Bytes Required				Word Port External Data Bytes Required		Byte Port External Data Bytes Required
	SIZ1	SIZ0	A1	A0	D31:D24	D23:D16	D15:D8	D7:D0	D31:D24	D23:D16	
Byte	0	1	0	0	OP3	N	N1	N2	OP3	N	OP3
	0	1	0	1	PR	OP3	N	N1	PR	OP3	OP3
	0	1	1	0	PR1	PR	OP3	N	OP3	N	OP3
	0	1	1	1	PR2	PR1	PR	OP3	PR	OP3	OP3
Word	1	0	0	0	OP2	OP3	N	N1	OP2	OP3	OP2
	1	0	0	1	PR	OP2	OP3	N	PR	OP2	OP2
	1	0	1	0	PR1	PR	OP2	OP3	OP2	OP3	OP2
	1	0	1	1	PR2	PR1	PR	OP2	PR	OP2	OP2
3 Byte	1	1	0	0	OP1	OP2	OP3	N	OP1	OP2	OP1
	1	1	0	1	PR	OP1	OP2	OP3	PR	OP1	OP1
	1	1	1	0	PR1	PR	OP1	OP2	OP1	OP2	OP1
	1	1	1	1	PR2	PR1	PR	OP1	PR	OP1	OP1
Long Word	0	0	0	0	OP0	OP1	OP2	OP3	OP0	OP1	OP0
	0	0	0	1	PR	OP0	OP1	OP2	PR	OP0	OP0
	0	0	1	0	PR1	PR	OP0	OP1	OP0	OP1	OP0
	0	0	1	1	PR2	PR1	PR	OP0	PR	OP0	OP0

NOTE: The bytes labeled as Nn (Next n) and PRn (Previous n) are only required to be valid for cachable read cycles. They can be interpreted as don't cares for noncachable read cycles.

Table 7-5 lists the combinations of SIZ0, SIZ1, A0, and A1 and the corresponding pattern of the data transfer for write cycles from the internal multiplexer of the MC68030 to the external data bus.

Figure 7-5 shows the transfer of a long-word operand to a word port. In the first bus cycle, the MC68030 places the four operand bytes on the external bus. Since the address is long-word aligned in this example, the multiplexer follows the pattern in the entry of Table 7-5 corresponding to SIZ0_SIZ1_A0_A1 = 0000. The port latches the data on bits D16–D31 of the data bus, asserts $\overline{\text{DSACK1}}$ (DSACK0 remains negated), and the processor

Table 7-5. MC68030 Internal to External Data Bus Multiplexer — Write Cycles

Transfer Size	Size		Address		External Data Bus Connection			
	SIZ1	SIZ0	A1	A0	D31:D24	D23:D16	D15:D8	D7:D0
Byte	0	1	x	x	OP3	OP3	OP3	OP3
Word	1	0	x	0	OP2	OP3	OP2	OP3
	1	0	x	1	OP2	OP2	OP3	OP2
3 Byte	1	1	0	0	OP1	OP2	OP3	OP0*
	1	1	0	1	OP1	OP1	OP2	OP3
	1	1	1	0	OP1	OP2	OP1	OP2
	1	1	1	1	OP1	OP1	OP2*	OP1
Long Word	0	0	0	0	OP0	OP1	OP2	OP3
	0	0	0	1	OP0	OP0	OP1	OP2
	0	0	1	0	OP0	OP1	OP0	OP1
	0	0	1	1	OP0	OP0	OP1*	OP0

*Due to the current implementation, this byte is output but never used.

x = don't care

NOTE: The OP tables on the external data bus refer to a particular byte of the operand that is written on that section of the data bus.

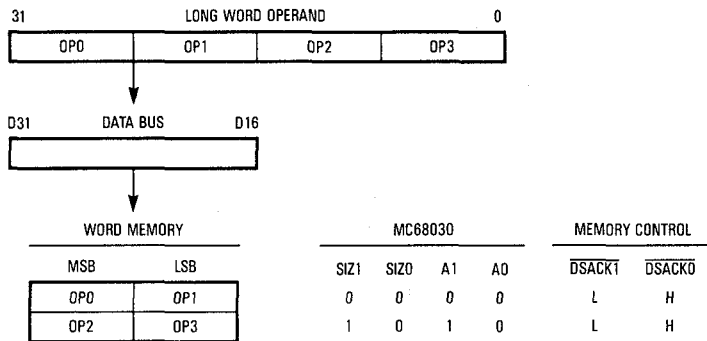


Figure 7-5. Example of Long-Word Transfer to Word Port

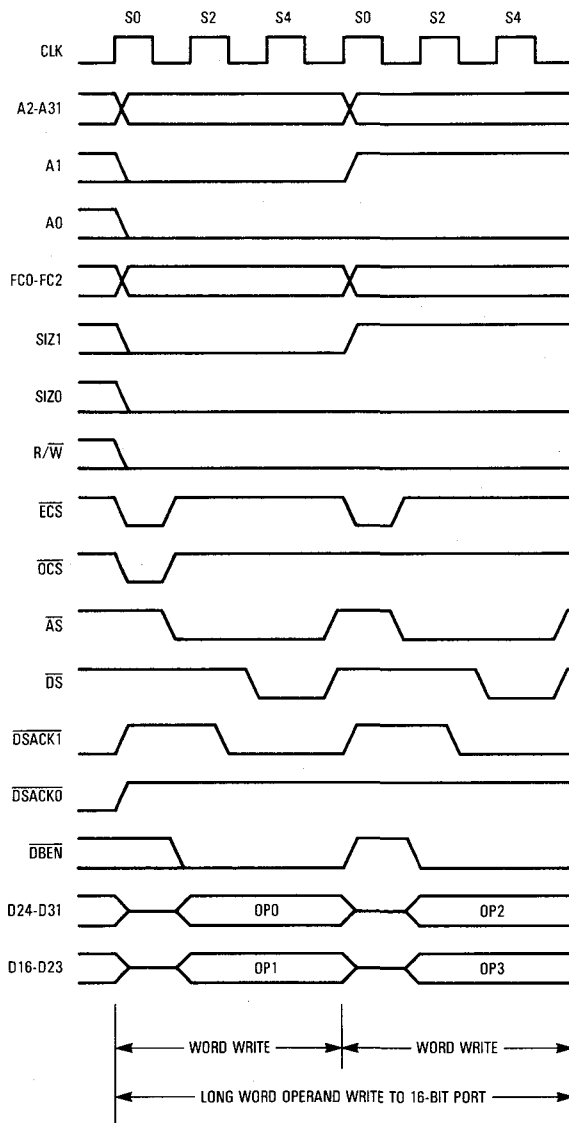


Figure 7-6. Long-Word Operand Write Timing (16-Bit Data Port)

terminates the bus cycle. It then starts a new bus cycle with $SIZ0_SIZ1_A0_A1 = 1010$ to transfer the remaining 16 bits. $SIZ0$ and $SIZ1$ indicate that a word remains to be transferred; $A0$ and $A1$ indicate that the word corresponds to an offset of two from the base address. The multiplexer follows the pattern corresponding to this configuration of the size and address signals and places the two least significant bytes of the long word on the word portion of the bus (D16–D31). The bus cycle transfers the remaining bytes to the word-size port. Figure 7-6 shows the timing of the bus transfer signals for this operation.

Figure 7-7 shows a word transfer to an 8-bit bus port. Like the preceding example, this example requires two bus cycles. Each bus cycle transfers a single byte. The size signals for the first cycle specify two bytes; for the second cycle, one byte. Figure 7-8 shows the associated bus transfer signal timing.

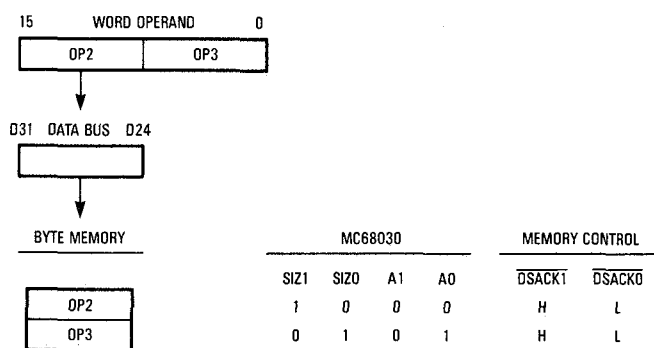


Figure 7-7. Example of Word Transfer to Byte Port

7.2.2 Misaligned Operands

Since operands may reside at any byte boundaries, they may be misaligned. A byte operand is properly aligned at any address; a word operand is misaligned at an odd address; a long word is misaligned at an address that is not evenly divisible by four. The MC68000, MC68008, and MC68010 implementations allow long-word transfers on odd-word boundaries but force exceptions if word or long-word operand transfers are attempted at odd-byte addresses. Although the MC68030 does not enforce any alignment restrictions for data operands (including PC relative data addresses), some performance degradation occurs when additional bus cycles are required for

Figure 7-8. Word Operand Write Timing (8-Bit Data Port)

long-word or word operands that are misaligned. For maximum performance, data items should be aligned on their natural boundaries. All instruction words and extension words must reside on word boundaries. Attempting to prefetch an instruction word at an odd address causes an address error exception.

Figure 7-9 shows the transfer of a long-word operand to an odd address in word-organized memory, which requires three bus cycles. For the first cycle, the size signals specify a long-word transfer, and the address offset (A2:A0) is 001. Since the port width is 16 bits, only the first byte of the long word is transferred. The slave device latches the byte and acknowledges the data transfer, indicating that the port is 16 bits wide. When the processor starts the second cycle, the size signals specify that three bytes remain to be transferred with an address offset (A2:A0) of 010. The next two bytes are transferred during this cycle. The processor then initiates the third cycle, with the size signals indicating one byte remaining to be transferred. The address offset (A2:A0) is now 100; the port latches the final byte; and the operation is complete. Figure 7-10 shows the associated bus transfer signal timing.

Figure 7-11 shows the equivalent operation for a cachable data read cycle.

Figures 7-12 and 7-13 show a word transfer to an odd address in word-organized memory. This example is similar to the one shown in Figures 7-9 and 7-10 except that the operand is word sized and the transfer requires only two bus cycles.

Figure 7-14 shows the equivalent operation for a cachable data read cycle.

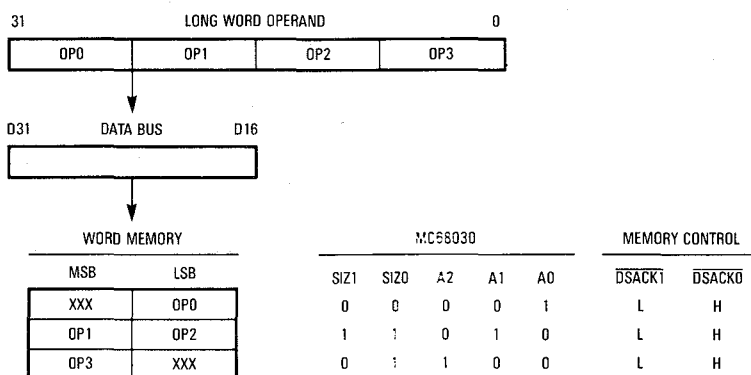


Figure 7-9. Misaligned Long-Word Transfer to Word Port Example

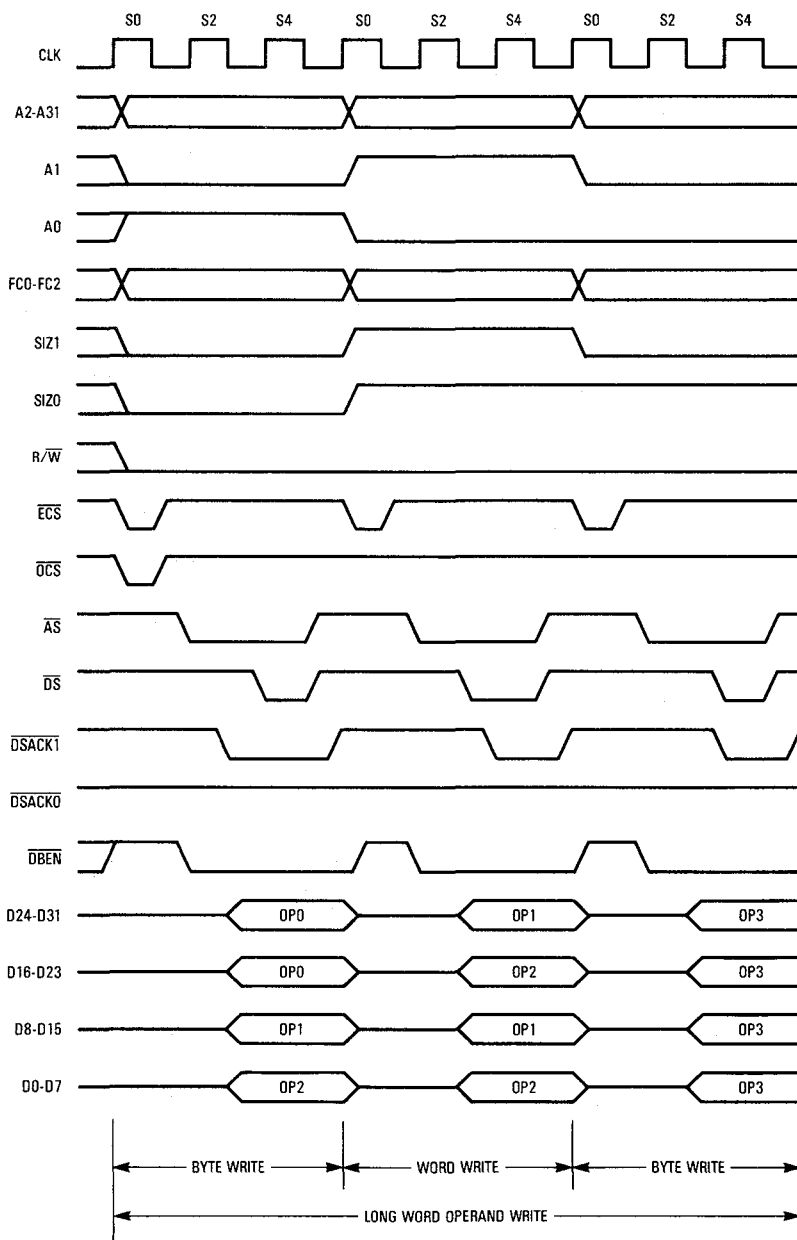


Figure 7-10. Misaligned Long-Word Transfer to Word Port

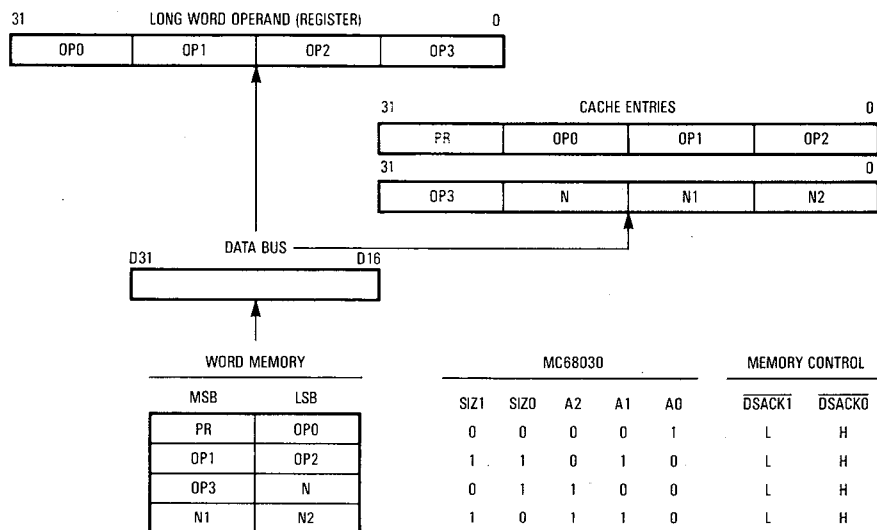


Figure 7-11. Misaligned Cachable Long-Word Transfer from Word Port Example

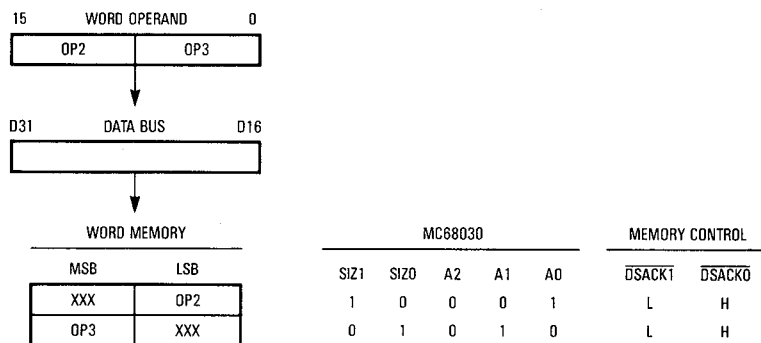


Figure 7-12. Misaligned Word Transfer to Word Port Example

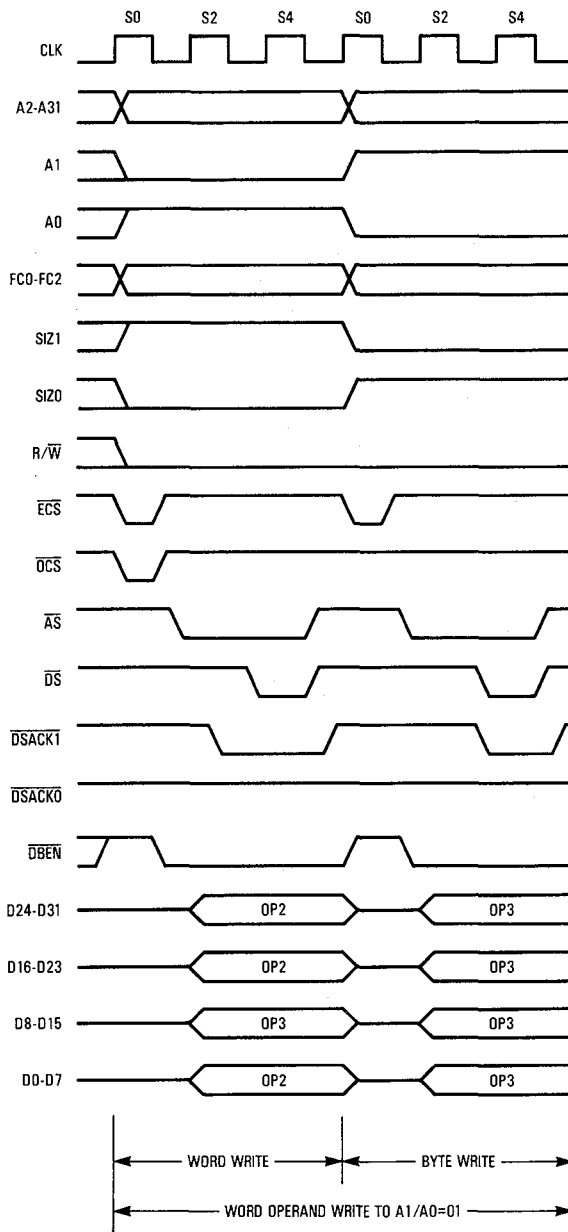


Figure 7-13. Misaligned Word Transfer to Word Port

Figures 7-15 and 7-16 show an example of a long-word transfer to an odd address in long-word-organized memory. In this example, a long-word access is attempted beginning at the least significant byte of a long-word-organized memory. Only one byte can be transferred in the first bus cycle. The second bus cycle then consists of a three-byte access to a long-word boundary. Since the memory is long-word organized, no further bus cycles are necessary.

Figure 7-17 shows the equivalent operation for a cachable data read cycle.

7.2.3 Effects of Dynamic Bus Sizing and Operand Misalignment

The combination of operand size, operand alignment, and port size determines the number of bus cycles required to perform a particular memory access. Table 7-6 shows the number of bus cycles required for different operand sizes to different port sizes with all possible alignment conditions for write cycles and noncachable read cycles.

Table 7-6. Memory Alignment and Port Size Influence on Write Bus Cycles

A1/A0	Number of Bus Cycles			
	00	01	10	11
Instruction*	1:2:4	N/A	N/A	N/A
Byte Operand	1:1:1	1:1:1	1:1:1	1:1:1
Word Operand	1:1:2	1:2:2	1:1:2	2:2:2
Long-Word Operand	1:2:4	2:3:4	2:2:4	2:3:4

Data Port Size — 32 Bits:16 Bits:8 Bits

*Instruction prefetches are always two words from a long-word boundary.

This table shows that bus cycle throughput is significantly affected by port size and alignment. The MC68030 system designer and programmer should be aware of and account for these effects, particularly in time-critical applications.

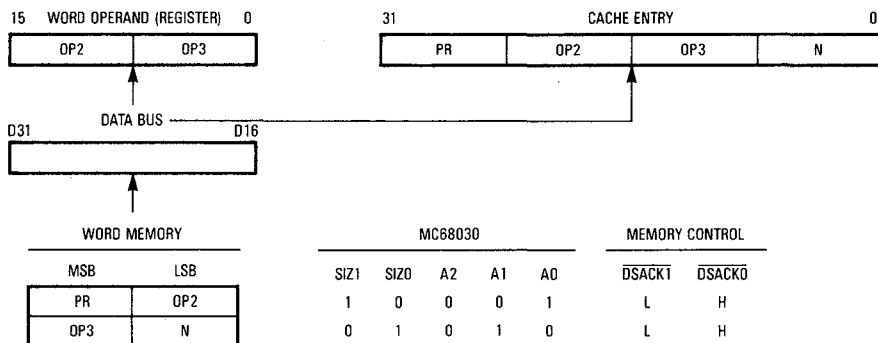


Figure 7-14. Example of Misaligned Cachable Word Transfer from Word Bus

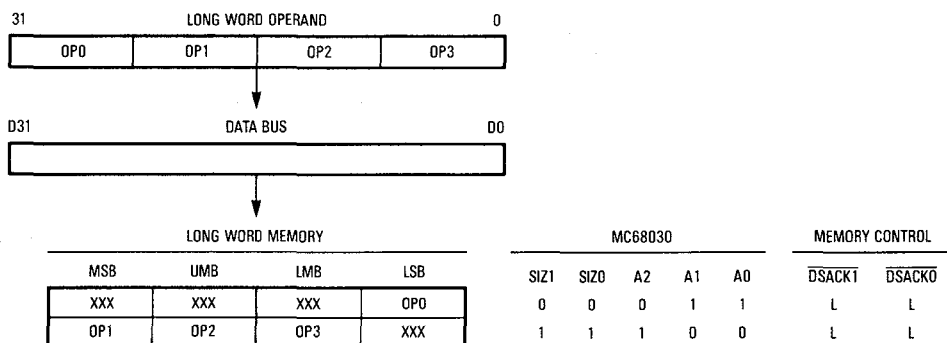


Figure 7-15. Misaligned Long-Word Transfer to Long-Word Port

Table 7-6 shows that the processor always prefetches instructions by reading a long word from a long-word address ($A1:A0=00$), regardless of port size or alignment. When the required instruction begins at an odd-word boundary, the processor attempts to fetch the entire 32 bits and loads both words into the instruction cache, if possible, although the second one is the required word. Even if the instruction access is not cached, the entire 32 bits are latched into an internal cache holding register from which the two instructions words can subsequently be referenced. Refer to **SECTION 11 INSTRUCTION EXECUTION TIMING** for a complete description of the cache holding register and pipeline operation.

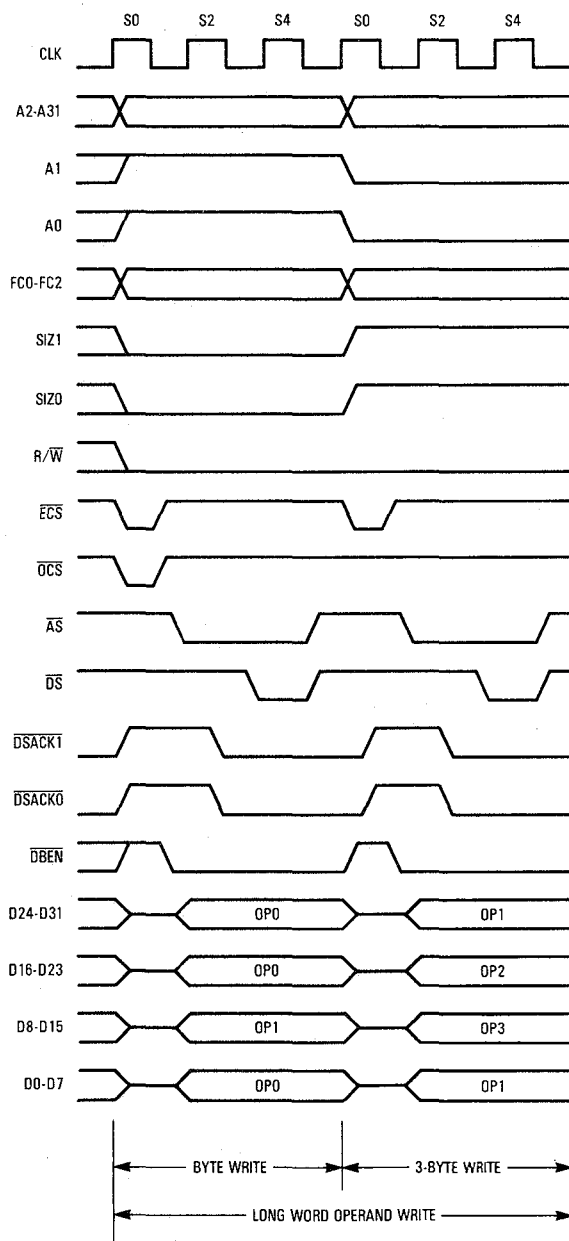


Figure 7-16. Misaligned Write Cycles to Long-Word Port

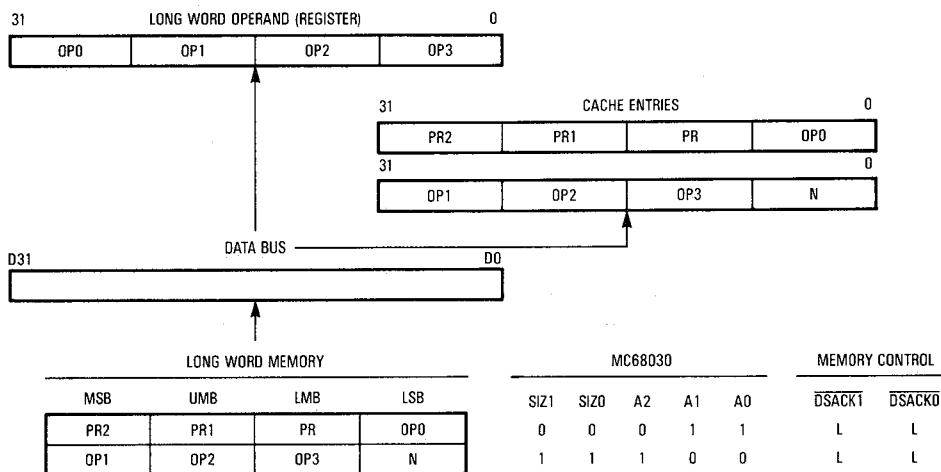


Figure 7-17. Misaligned Cachable Long-Word Transfer from Long-Word Bus

7

7.2.4 Address, Size, and Data Bus Relationships

The data transfer examples show how the MC68030 drives data onto or receives data from the correct byte sections of the data bus. Table 7-7 shows the combinations of the size signals and address signals that are used to generate byte enable signals for each of the four sections of the data bus for noncachable read cycles and all write cycles if the addressed device requires them. The port size also affects the generation of these enable signals as shown in the table. The four columns on the right correspond to the four byte enable signals. Letters B, W, and L refer to port sizes: B for 8-bit ports, W for 16-bit ports, and L for 32-bit ports. The letters B, W, and L imply that the byte enable signal should be true for that port size. A dash (—) implies that the byte enable signal does not apply.

The MC68030 always drives all sections of the data bus because, at the start of a write cycle, the bus controller does not know the port size. The byte enable signals in the table apply only to read operations that are not to be internally cached and to write operations. For cachable read cycles, during which the data is cached, the addressed port must drive all sections of the bus on which it resides.

Table 7-7. Data Bus Write Enable Signals for Byte, Word, and Long-Word Ports

Transfer Size	SIZ1	SIZ0	A1	A0	Data Bus Active Sections Byte (B) – Word (W) – Long-Word (L) Ports			
					D31:D24	D23:D16	D15:D8	D7:D0
Byte	0	1	0	0	B W L	—	—	—
	0	1	0	1	B	W L	—	—
	0	1	1	0	B W	—	L	—
	0	1	1	1	B	W	—	L
Word	1	0	0	0	B W L	W L	—	—
	1	0	0	1	B	W L	L	—
	1	0	1	0	B W	W	L	L
	1	0	1	1	B	W	—	L
3 Byte	1	1	0	0	B W L	W L	L	—
	1	1	0	1	B	W L	L	L
	1	1	1	0	B W	W	L	L
	1	1	1	1	B	W	—	L
Long Word	0	0	0	0	B W L	W L	L	L
	0	0	0	1	B	W L	L	L
	0	0	1	0	B W	W	L	L
	0	0	1	1	B	W	—	L

The table shows that the MC68030 transfers the number of bytes specified by the size signals to or from the specified address unless the operand is misaligned or the number of bytes is greater than the port width. In these cases, the device transfers the greatest number of bytes possible for the port. For example, if the size is four bytes and the address offset (A1:A0) is 01, a 32-bit slave can only receive three bytes in the current bus cycle. A 16- or 8-bit slave can only receive one byte. The table defines the byte enables for all port sizes. Byte data strobes can be obtained by combining the enable signals with the data strobe signal. Devices residing on 8-bit ports can use the data strobe by itself since there is only one valid byte for every transfer. These enable or strobe signals select only the bytes required for write cycles or for noncachable read cycles. The other bytes are not selected, which prevents incorrect accesses in sensitive areas such as I/O.

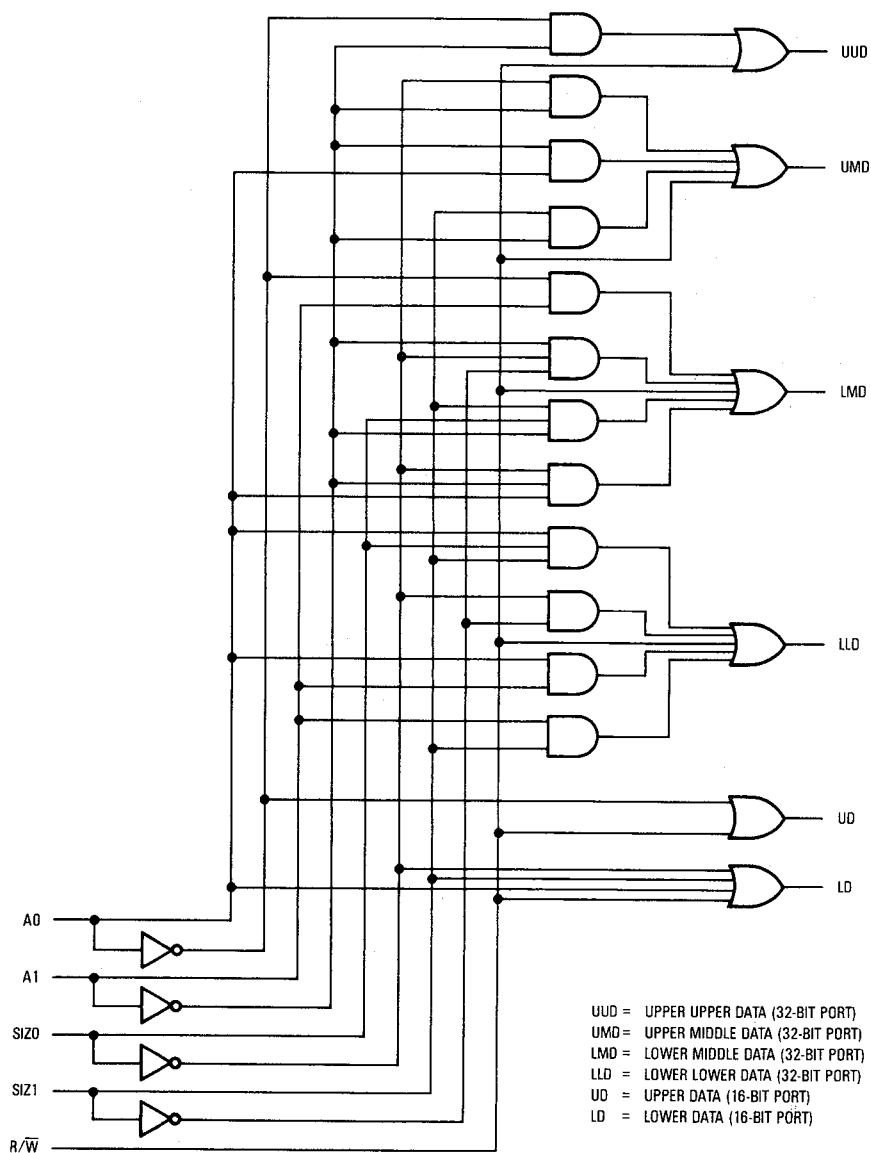
Figure 7-18 shows a logic diagram for one method for generating byte data enable signals for 16- and 32-bit ports from the size and address encodings and the read/write signal.

7.2.5 MC68030 versus MC68020 Dynamic Bus Sizing

The MC68030 supports the dynamic bus sizing mechanism of the MC68020 for asynchronous bus cycles (terminated with $\overline{DSACKx}$) with two restrictions. First, for a cachable access within the boundaries of an aligned long word, the port size must be consistent throughout the transfer of each long word. For example, when a byte port resides at address \$00, addresses \$01, \$02, and \$03 must also correspond to byte ports. Second, the port must supply as much data as it signals as port size, regardless of the transfer size indicated with the size signals and the address offset indicated by A0 and A1 for cachable accesses. Otherwise, dynamic bus sizing is identical in the two processors.

7.2.6 Cache Filling

The on-chip data and instruction caches, described in **SECTION 6 ON-CHIP CACHE MEMORIES**, are each organized as 16 lines of four long-word entries each. For each line, a tag contains the most significant bits of the logical address, FC2 (instruction cache) or FC0–FC2 (data cache), and a valid bit for each entry in the line. An entry fill operation loads an entire long word accessed from memory into a cache entry. This type of fill operation is performed when one entry of a line is not valid and an access is cachable. A burst fill operation is requested when a tag miss occurs for the current cycle or when all four entries in the cache line are invalid (provided the cache is enabled and burst filling for the cache is enabled). The burst fill operation attempts to fill all four entries in the line. To support burst filling, the slave device must have a 32-bit port and must have a burst mode capability; that is, it must acknowledge a burst request with the cache burst acknowledge ($\overline{CBACK}$) signal. It must also terminate the burst accesses with $\overline{STERM}$ and place a long word on the data bus for each transfer. The device may continue to supply successive long words, asserting $\overline{STERM}$ with each one, until the cache line is full. For further information about filling the cache, both entry fills and burst mode fills, refer to **6.1.3 Cache Filling**, **7.3.4 Synchronous Read Cycle**, **7.3.5 Synchronous Write Cycle**, and **7.3.7 Burst Operation Cycles**, which discuss in detail the required bus cycles.



NOTE: These select lines can be combined with the address decode circuitry, or all can be generated within the same programmed array logic unit.

Figure 7-18. Byte Data Select Generation for 16- and 32-Bit Ports

1.2.1 Cache Interactions

The organization and requirements of the on-chip instruction and data caches affect the interpretation of the $\overline{DSACKx}$ and $\overline{STERM}$ signals. Since the MC68030 attempts to load all data operands and instructions that are cachable into the on-chip caches, the bus may operate differently when caching is enabled. Specifically, on cachable read cycles that terminate normally, the low-order address signals (A0 and A1) and the size signals do not apply.

The slave device must supply as much aligned data on the data bus as its port size allows, regardless of the requested operand size. This means that an 8-bit port must supply a byte, a 16-bit port must supply a word, and a 32-bit port must supply an entire long word. This data is loaded into the cache. For a 32-bit port, the slave device ignores A0 and A1 and supplies the long word beginning at the long-word boundary on the data bus. For a 16-bit port, the device ignores A0 and supplies the entire word beginning at the lower word boundary on D16–D31 of the data bus. For a byte port, the device supplies the addressed byte on D24–D31.

7

If the addressed device cannot supply port-sized data or if the data should not be cached, the device must assert cache inhibit in $\overline{CIIN}$ as it terminates the read cycle. If the bus cycle terminates abnormally, the MC68030 does not cache the data. For details of interactions of port sizes, misalignments, and cache filling, refer to **6.1.3 Cache Filling**.

The caches can also affect the assertion of $\overline{AS}$ and the operation of a read cycle. The search of the appropriate cache by the processor begins when the microsequencer requires an instruction or a data item. At this time, the bus controller may also initiate an external bus cycle in case the requested item is not resident in the instruction or data cache. If the bus is not occupied with another read or write cycle, the bus controller asserts the $\overline{ECS}$ signal (and the $\overline{OCS}$ signal, if appropriate). If an internal cache hit occurs, the external cycle aborts, and $\overline{AS}$ is not asserted. This makes it possible to have $\overline{ECS}$ asserted on multiple consecutive clock cycles. Notice that there is a minimum time specified from the negation of $\overline{ECS}$ to the next assertion of $\overline{ECS}$ (refer to MC68030EC/D, *MC68030 Electrical Specifications*).

Instruction prefetches can occur every other clock so that if, after an aborted cycle due to an instruction cache hit, the bus controller asserts $\overline{ECS}$ on the next clock, this second cycle is for a data fetch. However, data accesses that hit in the data cache can also cause the assertion of $\overline{ECS}$ and an aborted cycle. Therefore, since instruction and data accesses are mixed, it is possible to see multiple successive $\overline{ECS}$ assertions on the external bus if the processor

is hitting in both caches and if the bus controller is free. Note that, if the bus controller is executing other cycles, these aborted cycles due to cache hits may not be seen externally. Also, $\overline{OCS}$ is asserted for the first *external* cycle of an operand transfer. Therefore, in the case of a misaligned data transfer where the first portion of the operand results in a cache hit (but the bus controller did not begin an external cycle and then abort it) and the second portion in a cache miss, $\overline{OCS}$ is asserted for the second portion of the operand.

7.2.8 Asynchronous Operation

The MC68030 bus may be used in an asynchronous manner. In that case, the external devices connected to the bus can operate at clock frequencies different from the clock for the MC68030. Asynchronous operation requires using only the handshake line ($\overline{AS}$, $\overline{DS}$, $\overline{DSACK1}$, $\overline{DSACK0}$, $\overline{BERR}$, and $\overline{HALT}$) to control data transfers. Using this method, $\overline{AS}$ signals the start of a bus cycle, and $\overline{DS}$ is used as a condition for valid data on a write cycle. Decoding the size outputs and lower address lines (A0 and A1) provides strobes that select the active portion of the data bus. The slave device (memory or peripheral) then responds by placing the requested data on the correct portion of the data bus for a read cycle or latching the data on a write cycle, and asserting the $\overline{DSACK1}/\overline{DSACK0}$ combination that corresponds to the port size to terminate the cycle. If no slave responds or the access is invalid, external control logic asserts the $\overline{BERR}$ or $\overline{BERR}$ and $\overline{HALT}$ signal(s) to abort or retry the bus cycle, respectively.

The $\overline{DSACKx}$ signals can be asserted before the data from a slave device is valid on a read cycle. The length of time that $\overline{DSACKx}$ may precede data is given by parameter #31, and it must be met in any asynchronous system to insure that valid data is latched into the processor. (Refer to MC68030EC/D, *MC68030 Electrical Specifications* for timing parameters.) Notice that no maximum time is specified from the assertion of $\overline{AS}$ to the assertion of $\overline{DSACKx}$. Although the processor can transfer data in a minimum of three clock cycles when the cycle is terminated with $\overline{DSACKx}$, the processor inserts wait cycles in clock period increments until $\overline{DSACKx}$ is recognized.

The $\overline{BERR}$ and/or $\overline{HALT}$ signals can be asserted after the $\overline{DSACKx}$ signal(s) is asserted. $\overline{BERR}$ and/or $\overline{HALT}$ must be asserted within the time given as parameter #48, after $\overline{DSACKx}$ is asserted in any asynchronous system. If this maximum delay time is violated, the processor may exhibit erratic behavior.

For asynchronous read cycles, the value of $\overline{CIIN}$ is internally latched on the rising edge of bus cycle state 4. Refer to **7.3.1 Asynchronous Read Cycle** for more details on the states for asynchronous read cycles.

During any bus cycle terminated by $\overline{DSACKx}$ or $\overline{BERR}$, the assertion of $\overline{CBACK}$ is completely ignored.

7.2.9 Synchronous Operation with $\overline{DSACKx}$

Although cycles terminated with the $\overline{DSACKx}$ signals are classified as asynchronous and cycles terminated with $\overline{STERM}$ are classified as synchronous, cycles terminated with $\overline{DSACKx}$ can also operate synchronously in that signals are interpreted relative to clock edges.

The devices that use these cycles must synchronize the responses to the MC68030 clock to be synchronous. Since they terminate bus cycles with the $\overline{DSACKx}$ signals, the dynamic bus sizing capabilities of the MC68030 are available. In addition, the minimum cycle time for these cycles is also three clocks.

To support those systems that use the system clock to generate $\overline{DSACKx}$ and other asynchronous inputs, the asynchronous input setup time (parameter #47A) and the asynchronous input hold time (parameter #47B) are given. If the setup and hold times are met for the assertion or negation of a signal, such as $\overline{DSACKx}$, the processor can be guaranteed to recognize that signal level on that specific falling edge of the system clock. If the assertion of $\overline{DSACKx}$ is recognized on a particular falling edge of the clock, valid data is latched into the processor (for a read cycle) on the next falling clock edge provided the data meets the data setup time (parameter #27). In this case, parameter #31 for asynchronous operation can be ignored. The timing parameters referred to are described in MC68030EC/D, *MC68030 Electrical Specifications*. If a system asserts $\overline{DSACKx}$ for the required window around the falling edge of S2 and obeys the proper bus protocol by maintaining $\overline{DSACKx}$ (and/or $\overline{BERR/HALT}$) until and throughout the clock edge that negates AS (with the appropriate asynchronous input hold time specified by parameter #47B), no wait states are inserted. The bus cycle runs at its maximum speed (three clocks per cycle) for bus cycles terminated with $\overline{DSACKx}$.

To assure proper operation in a synchronous system when $\overline{\text{BERR}}$ or $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ is asserted after $\overline{\text{DSACKx}}$, $\overline{\text{BERR}}$ (and $\overline{\text{HALT}}$) must meet the appropriate setup time (parameter #27A) prior to the falling clock edge one clock cycle after $\overline{\text{DSACKx}}$ is recognized. This setup time is critical, and the MC68030 may exhibit erratic behavior if it is violated.

When operating synchronously, the data-in setup and hold times for synchronous cycles may be used instead of the timing requirements for data relative to the $\overline{\text{DS}}$ signal.

The value of $\overline{\text{CIIN}}$ is latched on the rising edge of bus cycle state 4 for all cycles terminated with $\overline{\text{DSACKx}}$.

7.2.10 Synchronous Operation with $\overline{\text{STERM}}$

The MC68030 supports synchronous bus cycles terminated with $\overline{\text{STERM}}$. These cycles, for 32-bit ports only, are similar to cycles terminated with $\overline{\text{DSACKx}}$. The main difference is that $\overline{\text{STERM}}$ can be asserted (and data can be transferred) earlier than for a cycle terminated with $\overline{\text{DSACKx}}$, causing the processor to perform a minimum access time transfer in two clock periods. However, wait cycles can be inserted by delaying the assertion of $\overline{\text{STERM}}$ appropriately.

Using $\overline{\text{STERM}}$ instead of $\overline{\text{DSACKx}}$ in any bus cycle makes the cycle synchronous. Any bus cycle is synchronous if:

1. Neither $\overline{\text{DSACKx}}$ nor $\overline{\text{AVEC}}$ is recognized during the cycle.
2. The port size is 32 bits.
3. Synchronous input setup and hold time requirements (specifications #60 and #61) for $\overline{\text{STERM}}$ are met.

Burst mode operation requires the use of $\overline{\text{STERM}}$ to terminate each of its cycles. The first cycle of any burst transfer must be a synchronous cycle as described in the preceding paragraph. The exact timing of this cycle is controlled by the assertion of $\overline{\text{STERM}}$, and wait cycles can be inserted as necessary. However, the minimum cycle time is two clocks. If a burst operation is initiated and allowed to terminate normally, the second, third, and fourth cycles latch data on successive falling edges of the clock at a minimum. Again, the exact timing for these subsequent cycles is controlled by the timing of $\overline{\text{STERM}}$ for each of these cycles, and wait cycles can be inserted as necessary.

Although the synchronous input signals ($\overline{\text{STERM}}$, $\overline{\text{CIIN}}$, and $\overline{\text{CBACK}}$) must be stable for the appropriate setup and hold times relative to every rising edge of the clock during which $\overline{\text{AS}}$ is asserted, the assertion or negation of $\overline{\text{CBACK}}$ and $\overline{\text{CIIN}}$ is internally latched on the rising edge of the clock for which $\overline{\text{STERM}}$ is asserted in a synchronous cycle.

The $\overline{\text{STERM}}$ signal can be generated from the address bus and function code value and does not need to be qualified with the $\overline{\text{AS}}$ signal. If $\overline{\text{STERM}}$ is asserted and no cycle is in progress (even if the cycle has begun, $\overline{\text{ECS}}$ is asserted and then the cycle is aborted), $\overline{\text{STERM}}$ is ignored by the MC68030.

Similarly, $\overline{\text{CBACK}}$ can be asserted independently of the assertion of $\overline{\text{CBREQ}}$. If a cache burst is not requested, the assertion of $\overline{\text{CBACK}}$ is ignored.

The assertion of $\overline{\text{CIIN}}$ is ignored when the appropriate cache is not enabled or when cache inhibit out ($\overline{\text{CIOUT}}$) is asserted. It is also ignored during write cycles or translation table searches.

NOTE

$\overline{\text{STERM}}$ and $\overline{\text{DSACKx}}$ should *never* be asserted during the same bus cycle.

7.3 DATA TRANSFER CYCLES

The transfer of data between the processor and other devices involves the following signals:

- Address Bus A0–A31
- Data Bus D0–D31
- Control Signals

The address and data buses are both parallel nonmultiplexed buses. The bus master moves data on the bus by issuing control signals, and the asynchronous/synchronous bus uses a handshake protocol to insure correct movement of the data. In all bus cycles, the bus master is responsible for de-skewing all signals it issues at both the start and the end of the cycle. In addition, the bus master is responsible for de-skewing the acknowledge and data signals from the slave devices. The following paragraphs define read, write, and read-modify-write cycle operations. An additional paragraph describes burst mode transfers.

Each of the bus cycles is defined as a succession of states. These states apply to the bus operation and are different from the processor states described in **SECTION 4 PROCESSING STATES**. The clock cycles used in the descriptions and timing diagrams of data transfer cycles are independent of the clock frequency. Bus operations are described in terms of external bus states.

7.3.1 Asynchronous Read Cycle

During a read cycle, the processor receives data from a memory, coprocessor, or peripheral device. If the instruction specifies a long-word operation, the MC68030 attempts to read four bytes at once. For a word operation, it attempts to read two bytes at once, and for a byte operation, one byte. For some operations, the processor requests a three-byte transfer. The processor properly positions each byte internally. The section of the data bus from which each byte is read depends on the operand size, address signals (A0–A1), $\overline{\text{CIIN}}$ and $\overline{\text{CROUT}}$, whether the internal caches are enabled, and the port size. Refer to **7.2.1 Dynamic Bus Sizing**, **7.2.2 Misaligned Operands**, and **7.2.6 Cache Filling** for more information on dynamic bus sizing, misaligned operands, and cache interactions.

7

Figure 7-19 is a flowchart of an asynchronous long-word read cycle. Figure 7-20 is a flowchart of a byte read cycle. The following figures show functional read cycle timing diagrams specified in terms of clock periods. Figure 7-21 corresponds to byte and word read cycles from a 32-bit port. Figure 7-22 corresponds to a long-word read cycle from an 8-bit port. Figure 7-23 also applies to a long-word read cycle, but from a 16-bit port.

State 0

The read cycle starts in state 0 (S0). The processor drives $\overline{\text{ECS}}$ low, indicating the beginning of an external cycle. When the cycle is the first external cycle of a read operand operation, operand cycle start ($\overline{\text{OCS}}$) is driven low at the same time. During S0, the processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the cycle. The processor drives $\text{R}/\overline{\text{W}}$ high for a read cycle and drives $\overline{\text{DBEN}}$ inactive to disable the data buffers. SIZ0 – SIZ1 become valid, indicating the number of bytes requested to be transferred. $\overline{\text{CROUT}}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TTx register.

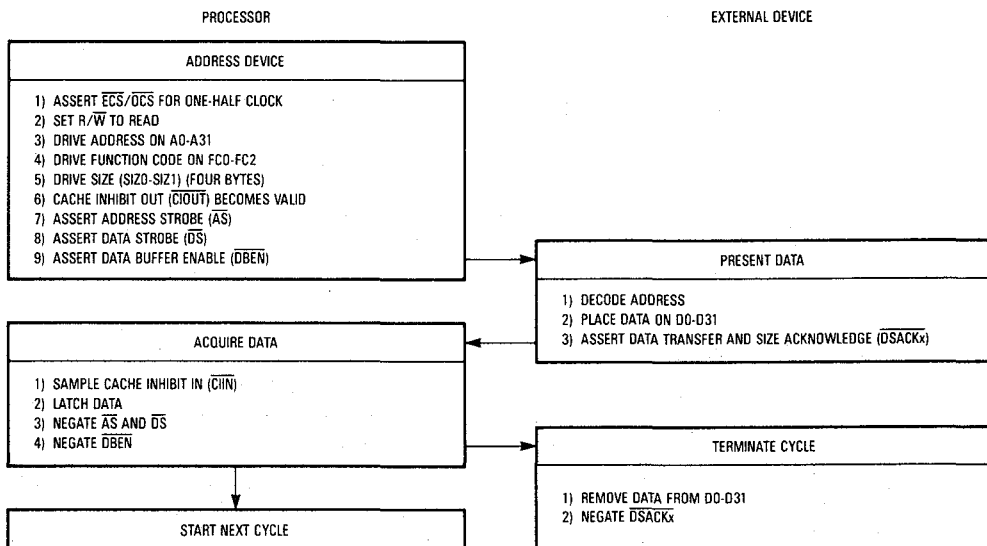


Figure 7-19. Asynchronous Long-Word Read Cycle Flowchart

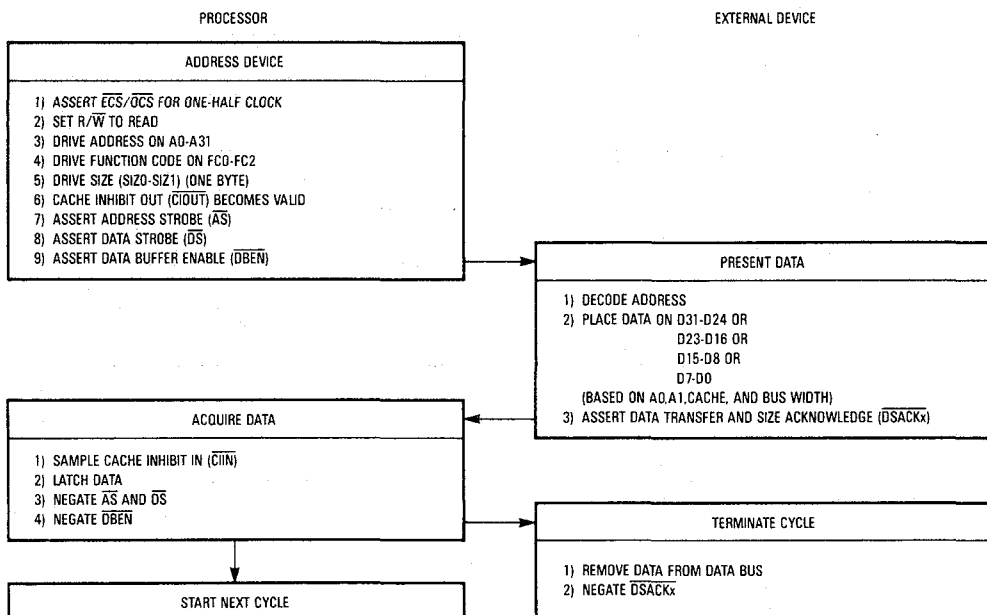


Figure 7-20. Asynchronous Byte Read Cycle Flowchart

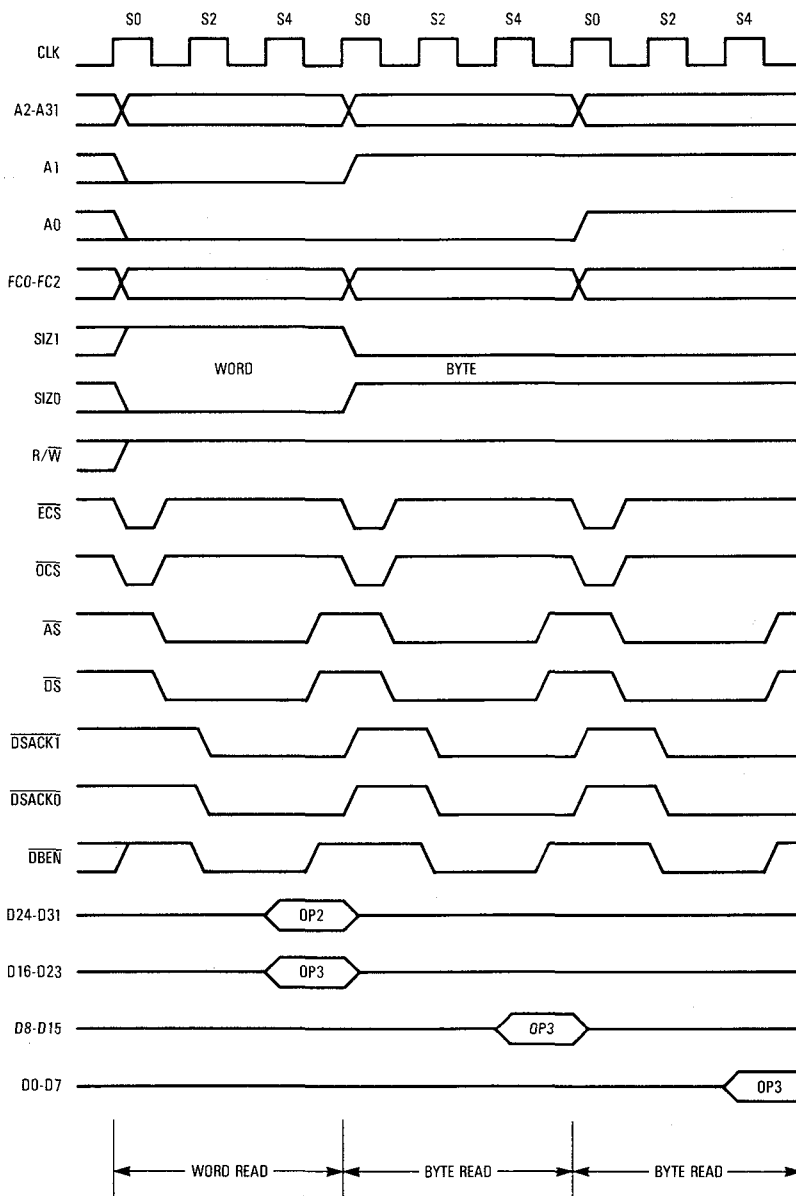


Figure 7-21. Asynchronous Byte and Word Read Cycles — 32-Bit Port

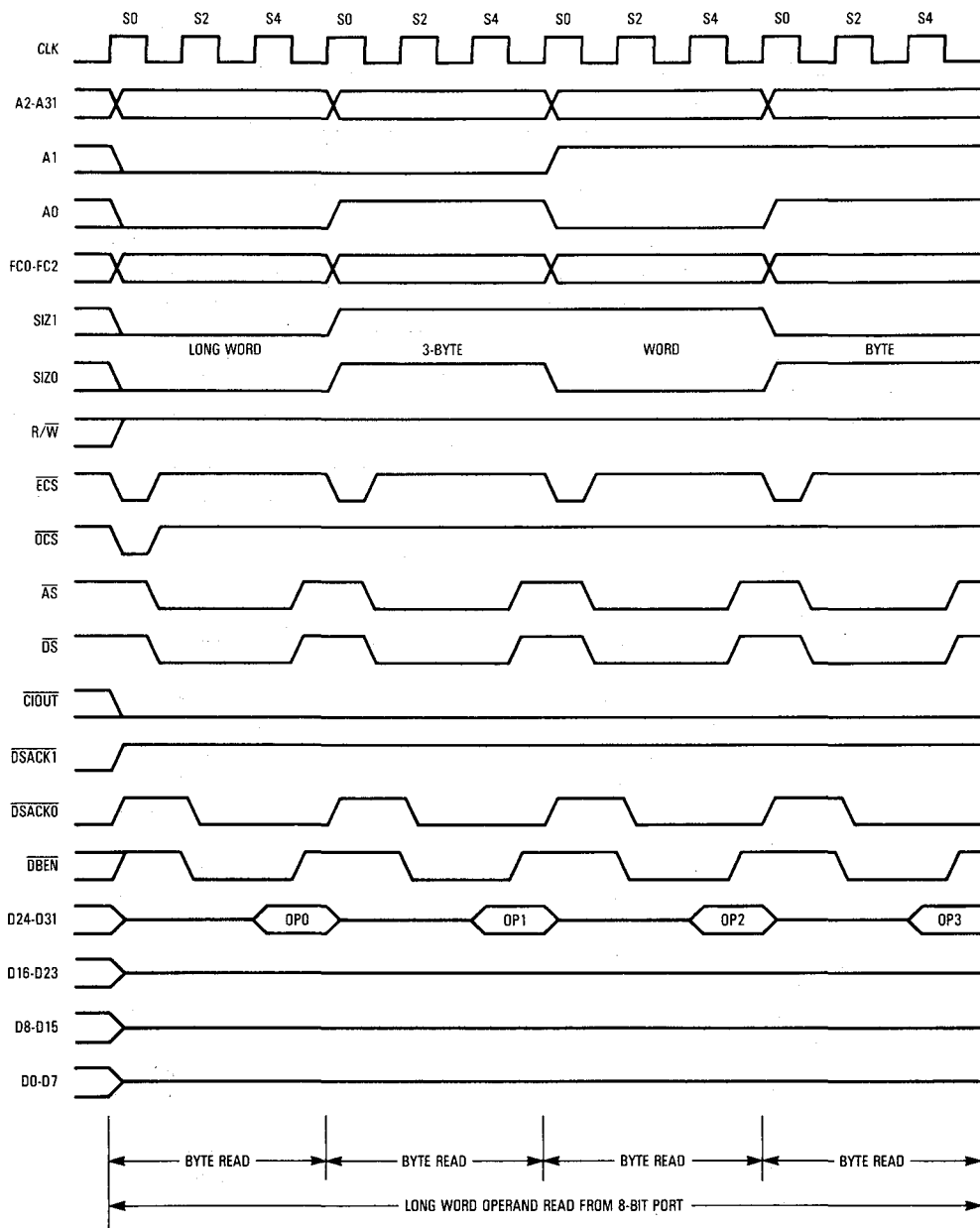


Figure 7-22. Long-Word Read — 8-Bit Port with $\overline{\text{CIOOUT}}$ Asserted

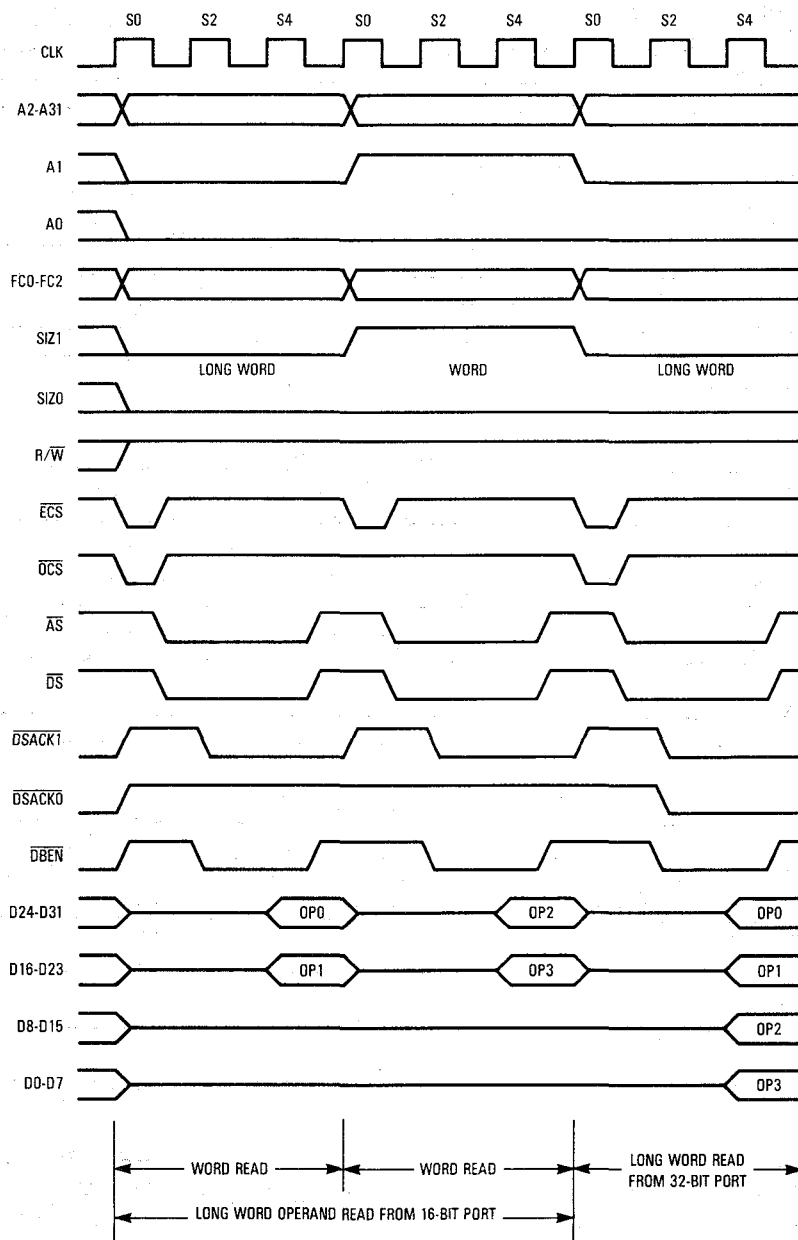


Figure 7-23. Long-Word Read — 16-Bit and 32-Bit Port

State 1

One-half clock later in state 1 (S1), the processor asserts $\overline{AS}$ indicating that the address on the address bus is valid. The processor also asserts $\overline{DS}$ also during S1. In addition, the $\overline{ECS}$ (and $\overline{OCS}$, if asserted) signal is negated during S1.

State 2

During state 2 (S2), the processor asserts $\overline{DBEN}$ to enable external data buffers. The selected device uses $R/\overline{W}$, $SIZ0-SIZ1$, $A0-A1$, $\overline{CIOUT}$, and $\overline{DS}$ to place its information on the data bus, and drives $\overline{CIIN}$ if appropriate. Any or all of the bytes (D24–D31, D16–D23, D8–D15, and D0–D7) are selected by $SIZ0-SIZ1$ and $A0-A1$. Concurrently, the selected device asserts $\overline{DSACKx}$.

State 3

As long as at least one of the $\overline{DSACKx}$ signals is recognized by the end of S2 (meeting the asynchronous input setup time requirement), data is latched on the next falling edge of the clock, and the cycle terminates. If $\overline{DSACKx}$ is not recognized by the start of state 3 (S3), the processor inserts wait states instead of proceeding to states 4 and 5. To ensure that wait states are inserted, both $\overline{DSACK0}$ and $\overline{DSACK1}$ must remain negated throughout the asynchronous input setup and hold times around the end of S2. If wait states are added, the processor continues to sample the $\overline{DSACKx}$ signals on the falling edges of the clock until one is recognized.

State 4

The processor samples $\overline{CIIN}$ at the beginning of state 4 (S4). Since $\overline{CIIN}$ is defined as a synchronous input, whether asserted or negated, it must meet the appropriate synchronous input setup and hold times on every rising edge of the clock while $\overline{AS}$ is asserted. At the end of S4, the processor latches the incoming data.

State 5

The processor negates $\overline{AS}$, $\overline{DS}$, and $\overline{DBEN}$ during state 5 (S5). It holds the address valid during S5 to provide address hold time for memory systems. $R/\overline{W}$, $SIZ0-SIZ1$, and $FC0-FC2$ also remain valid throughout S5.

The external device keeps its data and $\overline{DSACKx}$ signals asserted until it detects the negation of $\overline{AS}$ or $\overline{DS}$ (whichever it detects first). The device must remove its data and negate $\overline{DSACKx}$ within approximately one clock period after sensing the negation of $\overline{AS}$ or $\overline{DS}$. $\overline{DSACKx}$ signals that remain asserted beyond this limit may be prematurely detected for the next bus cycle.

7.3.2 Asynchronous Write Cycle

During a write cycle, the processor transfers data to memory or a peripheral device.

Figure 7-24 is a flowchart of a write cycle operation for a long-word transfer. The following figures show the functional write cycle timing diagrams specified in terms of clock periods. Figure 7-25 shows two write cycles (between two read cycles with no idle time) for a 32-bit port. Figure 7-26 shows byte and word write cycles to a 32-bit port. Figure 7-27 shows a long-word write cycle to an 8-bit port. Figure 7-28 shows a long-word write cycle to a 16-bit port.

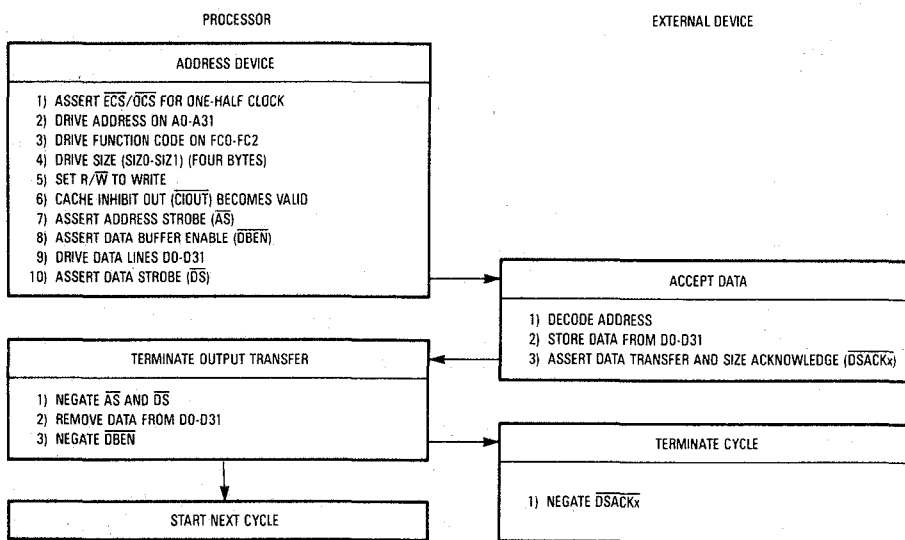


Figure 7-24. Asynchronous Write Cycle Flowchart

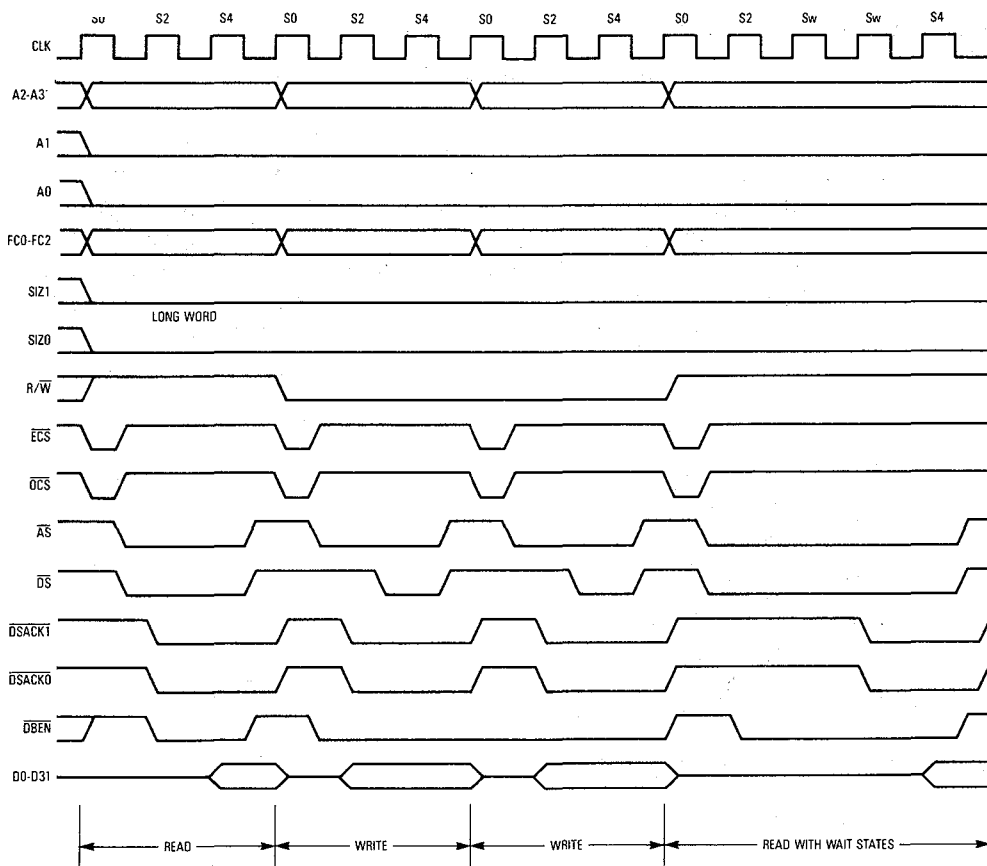


Figure 7-25. Asynchronous Read-Write-Read Cycles — 32-Bit Port

State 0

The write cycle starts in S0. The processor drives $\overline{ECS}$ low, indicating the beginning of an external cycle. When the cycle is the first external cycle of a write operation, $\overline{OCS}$ is driven low at the same time. During S0, the processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the cycle. The processor drives $\overline{R/W}$ low for a write cycle. SIZ0–SIZ1 become valid, indicating the number of bytes to be transferred. $\overline{CIOUT}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TTx register.

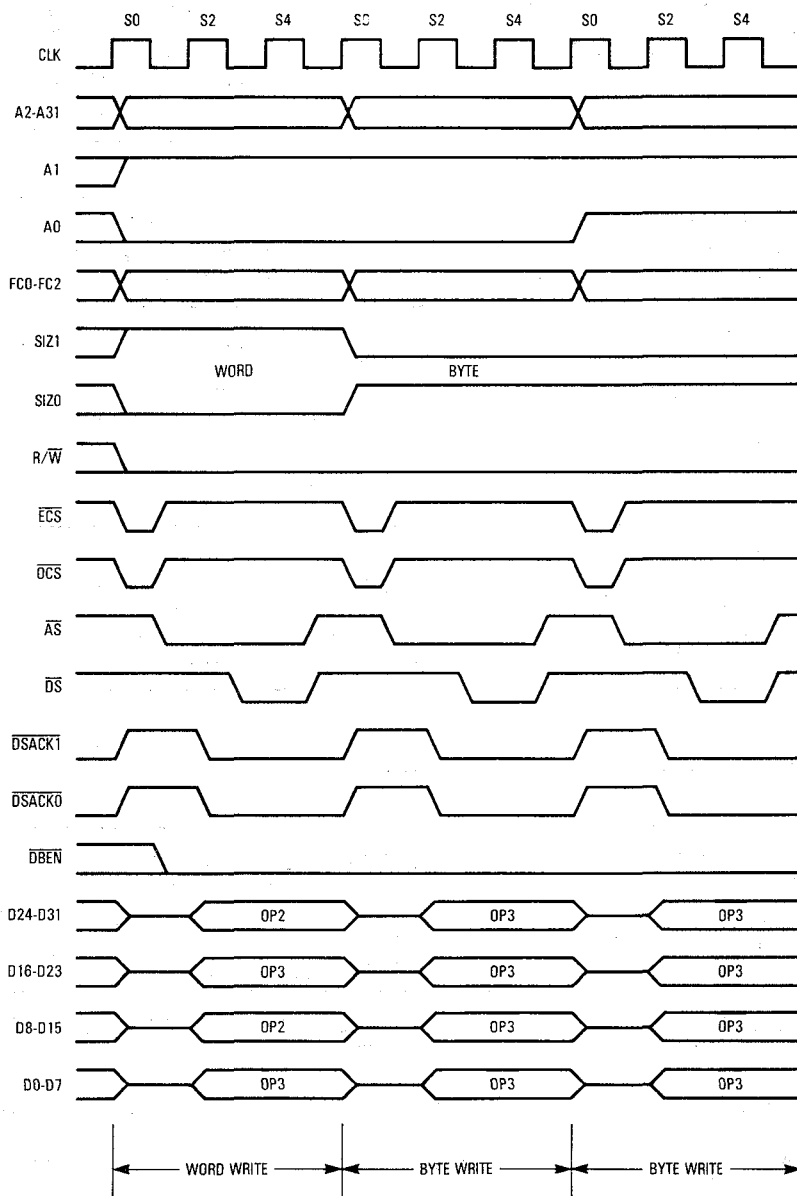


Figure 7-26. Asynchronous Byte and Word Write Cycles — 32-Bit Port

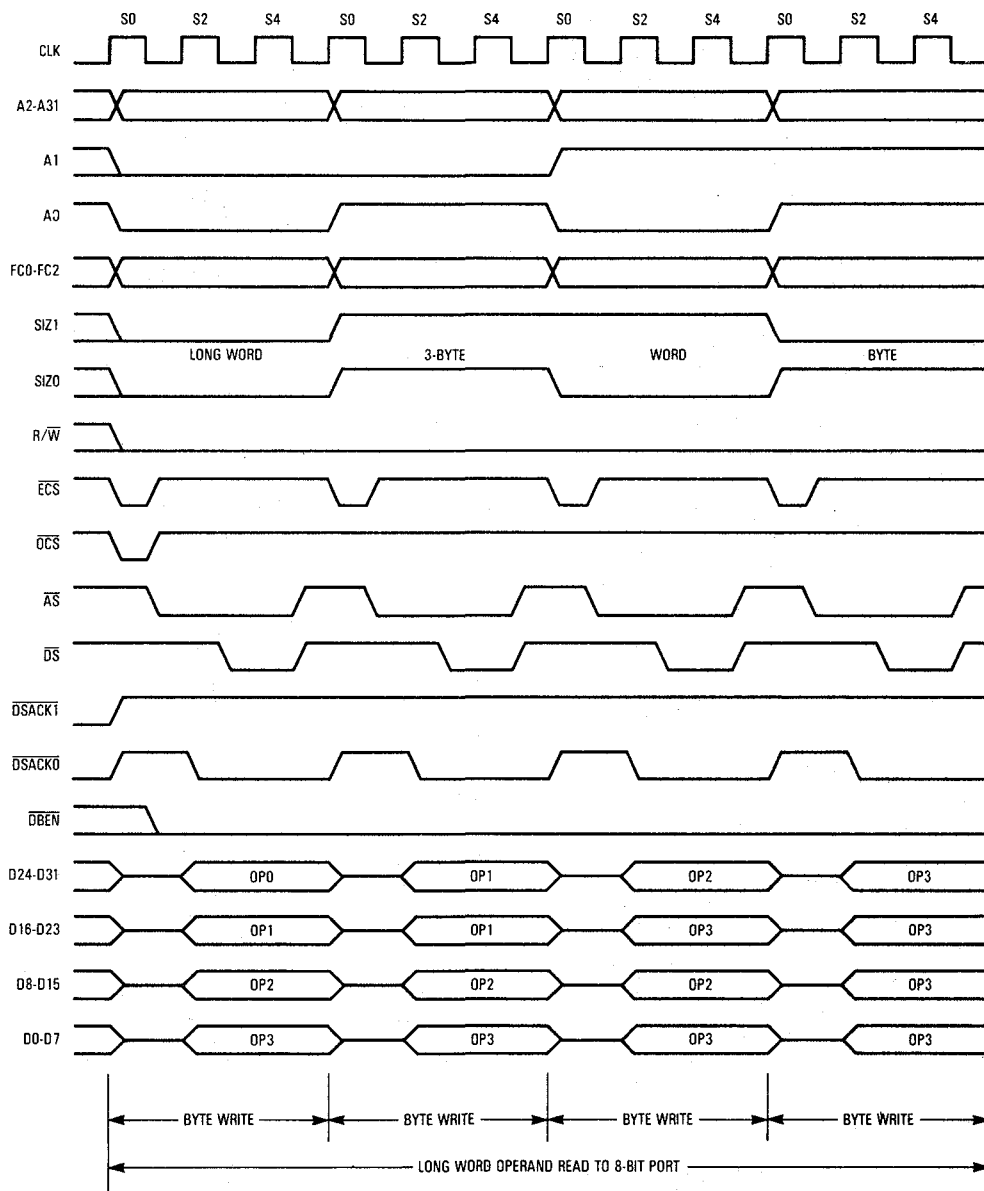


Figure 7-27. Long-Word Operand Write — 8-Bit Port

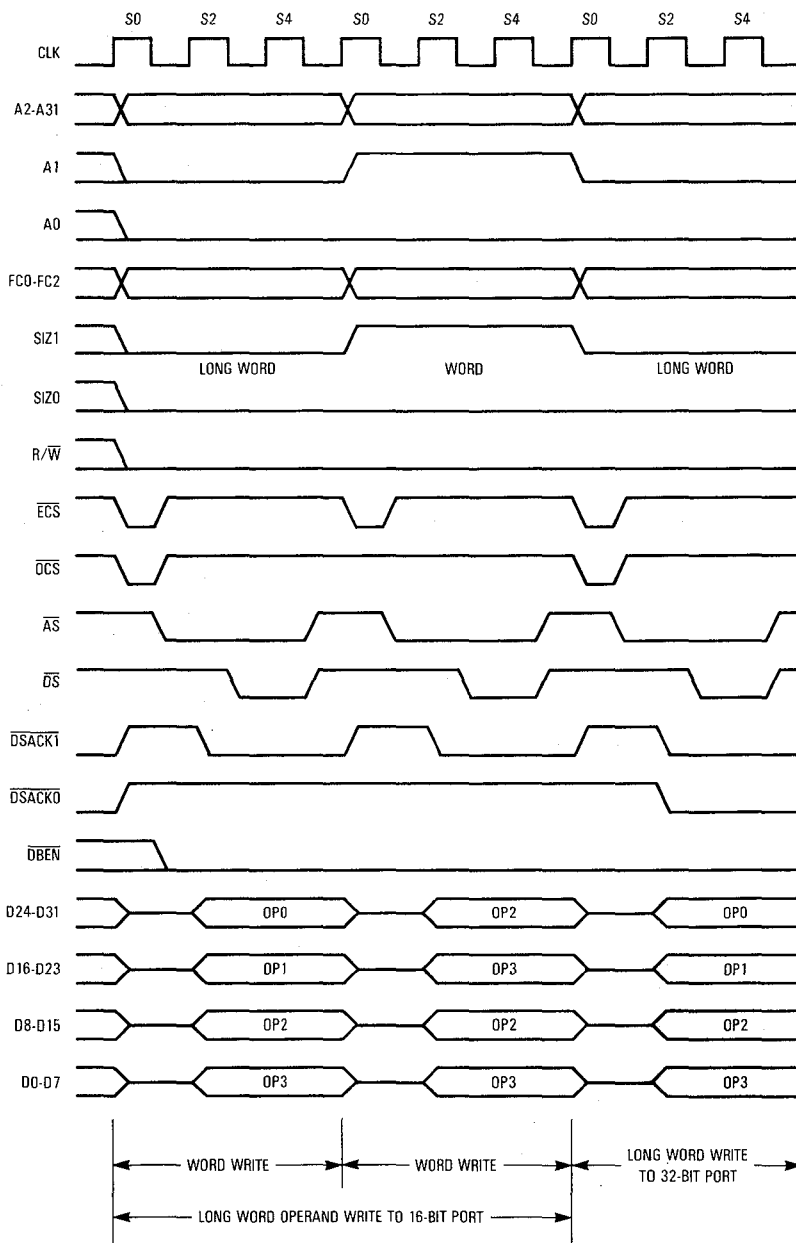


Figure 7-28. Long-Word Operand Write — 16-Bit Port

State 1

One-half clock later in S1, the processor asserts $\overline{AS}$, indicating that the address on the address bus is valid. The processor also asserts $\overline{DBEN}$ during S1, which can enable external data buffers. In addition, the $\overline{ECS}$ (and $\overline{OCS}$, if asserted) signal is negated during S1.

State 2

During S2, the processor places the data to be written onto the D0–D31, and samples $\overline{DSACKx}$ at the end of S2.

State 3

The processor asserts $\overline{DS}$ during S3, indicating that the data is stable on the data bus. As long as at least one of the $\overline{DSACKx}$ signals is recognized by the end of S2 (meeting the asynchronous input setup time requirement), the cycle terminates one clock later. If $\overline{DSACKx}$ is not recognized by the start of S3, the processor inserts wait states instead of proceeding to S4 and S5. To ensure that wait states are inserted, both $\overline{DSACK0}$ and $\overline{DSACK1}$ must remain negated throughout the asynchronous input setup and hold times around the end of S2. If wait states are added, the processor continues to sample the $\overline{DSACKx}$ signals on the falling edges of the clock until one is recognized. The selected device uses $R/\overline{W}$, $\overline{DS}$, $SIZ0$ – $SIZ1$, and $A0$ – $A1$ to latch data from the appropriate byte(s) of the data bus (D24–D31, D16–D23, D8–D15, and D0–D7). $SIZ0$ – $SIZ1$ and $A0$ – $A1$ select the bytes of the data bus. If it has not already done so, the device asserts $\overline{DSACKx}$ to signal that it has successfully stored the data.

State 4

The processor issues no new control signals during S4.

State 5

The processor negates $\overline{AS}$ and $\overline{DS}$ during S5. It holds the address and data valid during S5 to provide address hold time for memory systems. $R/\overline{W}$, $SIZ0$ – $SIZ1$, $FC0$ – $FC2$, and $\overline{DBEN}$ also remain valid throughout S5.

The external device must keep $\overline{DSACKx}$ asserted until it detects the negation of $\overline{AS}$ or $\overline{DS}$ (whichever it detects first). The device must negate $\overline{DSACKx}$ within approximately one clock period after sensing the negation of $\overline{AS}$ or $\overline{DS}$. $\overline{DSACKx}$ signals that remain asserted beyond this limit may be prematurely detected for the next bus cycle.

7.3.3 Asynchronous Read-Modify-Write Cycle

The read-modify-write cycle performs a read, conditionally modifies the data in the arithmetic logic unit, and may write the data out to memory. In the MC68030 processor, this operation is indivisible, providing semaphore capabilities for multiprocessor systems. During the entire read-modify-write sequence, the MC68030 asserts the $\overline{RMC}$ signal to indicate that an indivisible operation is occurring. The MC68030 does not issue a bus grant ($\overline{BG}$) signal in response to a bus request ($\overline{BR}$) signal during this operation. The read portion of a read-modify-write operation is forced to miss in the data cache because the data in the cache would not be valid if another processor had altered the value being read. However, read-modify-write cycles may alter the contents of the data cache as described in **6.1.2. Data Cache**.

No burst filling of the data cache occurs during a read-modify-write operation.

The test and set (TAS) and compare and swap (CAS and CAS2) instructions are the only MC68030 instructions that utilize read-modify-write operations. Depending on the compare results of the CAS and CAS2 instructions, the write cycle(s) may not occur. Table search accesses required for the MMU are always read-modify-write cycles to the supervisor data space. During these cycles, a write does not occur unless a descriptor is updated. No data is internally cached for table search accesses since the MMU uses physical addresses to access the tables. Refer to **SECTION 9 MEMORY MANAGEMENT UNIT** for information about the MMU.

Figure 7-29 is a flowchart of the asynchronous read-modify-write cycle operation. Figure 7-30 is an example of a functional timing diagram of a TAS instruction specified in terms of clock periods.

State 0

The processor asserts $\overline{ECS}$ and $\overline{OCS}$ in S0 to indicate the beginning of an external operand cycle. The processor also asserts $\overline{RMC}$ in S0 to identify a read-modify-write cycle. The processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the operation. SIZ0–SIZ1 become valid in S0 to indicate the operand size. The processor drives $R/\overline{W}$ high for the read cycle and sets $\overline{CIOUT}$ according to the value of the MMU CI bit in the address translation descriptor or in the appropriate TT_x register.

State 1

One-half clock later in S1, the processor asserts $\overline{AS}$, indicating that the address on the address bus is valid. The processor asserts $\overline{DS}$ during S1. In addition, the $\overline{ECS}$ (and $\overline{OCS}$, if asserted) signal is negated during S1.

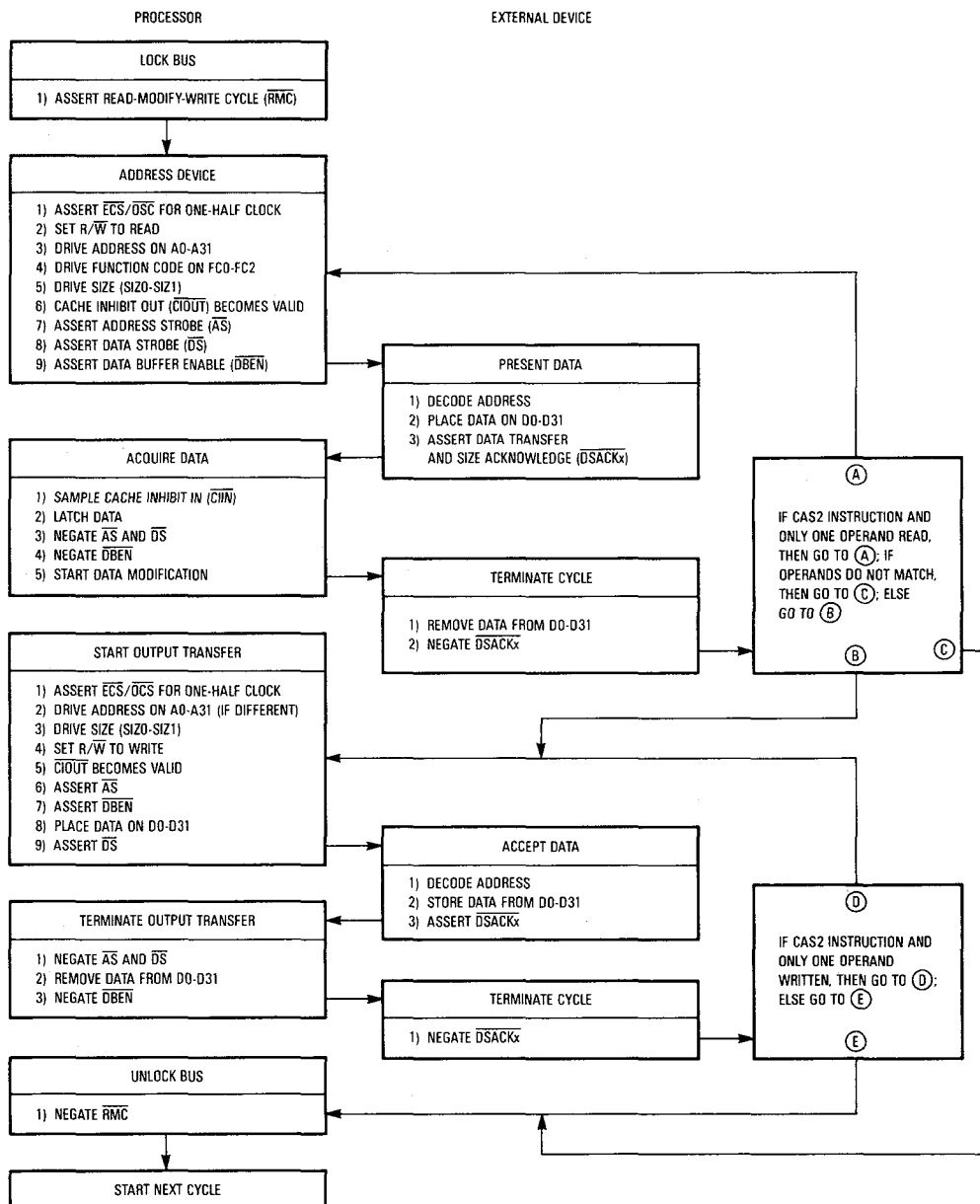
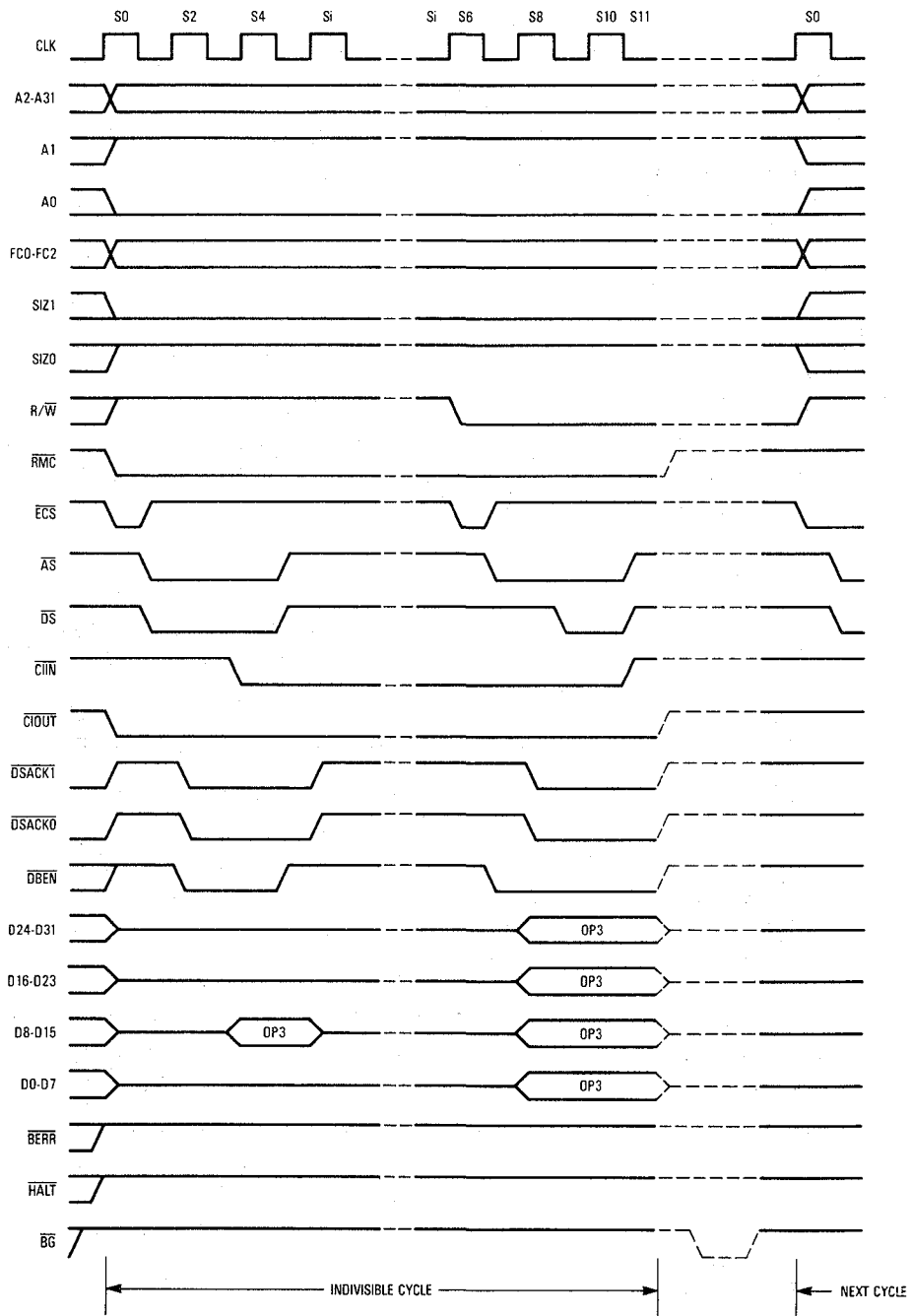


Figure 7-29. Asynchronous Read-Modify-Write Cycle Flowchart



**Figure 7-30. Asynchronous Byte Read-Modify-Write Cycle — 32-Bit Port
(TAS Instruction with $\overline{\text{CIOUT}}$ or $\overline{\text{CIIN}}$ Asserted)**

State 2

During state 2 (S2), the processor drives $\overline{DBEN}$ active to enable external data buffers. The selected device uses $R/\overline{W}$, $SIZ0-SIZ1$, $A0-A1$, and $\overline{DS}$ to place information on the data bus. Any or all of the bytes (D24–D31, D16–D23, D8–D15, and D0–D7) are selected by $SIZ0-SIZ1$ and $A0-A1$. Concurrently, the selected device may assert the $\overline{DSACKx}$ signals.

State 3

As long as at least one of the $\overline{DSACKx}$ signals is recognized by the end of S2 (meeting the asynchronous input setup time requirement), data is latched on the next falling edge of the clock, and the cycle terminates. If $\overline{DSACKx}$ is not recognized by the start of S3, the processor inserts wait states instead of proceeding to S4 and S5. To ensure that wait states are inserted, both $\overline{DSACK0}$ and $\overline{DSACK1}$ must remain negated throughout the asynchronous input setup and hold times around the end of S2. If wait states are added, the processor continues to sample the $\overline{DSACKx}$ signals on the falling edges of the clock until one is recognized.

State 4

The processor samples the level of $\overline{CIIN}$ at the beginning of S4. At the end of S4, the processor latches the incoming data.

State 5

The processor negates $\overline{AS}$, $\overline{DS}$, and $\overline{DBEN}$ during S5. If more than one read cycle is required to read in the operand(s), S0–S5 are repeated for each read cycle. When finished reading, the processor holds the address, $R/\overline{W}$, and $FC0-FC2$ valid in preparation for the write portion of the cycle.

The external device keeps its data and $\overline{DSACKx}$ signals asserted until it detects the negation of $\overline{AS}$ or $\overline{DS}$ (whichever it detects first). The device must remove the data and negate $\overline{DSACKx}$ within approximately one clock period after sensing the negation of $\overline{AS}$ or $\overline{DS}$. $\overline{DSACKx}$ signals that remain asserted beyond this limit may be prematurely detected for the next portion of the operation.

Idle States

The processor does not assert any new control signals during the idle states, but it may internally begin the modify portion of the cycle at this time. S6–S11 are omitted if no write cycle is required. If a write cycle is required, the $R/\overline{W}$ signal remains in the read mode until S6 to prevent bus conflicts with the preceding read portion of the cycle; the data bus is not driven until S8.

State 6

The processor asserts $\overline{\text{ECS}}$ and $\overline{\text{OCS}}$ in S6 to indicate that another external cycle is beginning. The processor drives $\text{R}/\overline{\text{W}}$ low for a write cycle. $\overline{\text{CIOUT}}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in a relevant TTx register. Depending on the write operation to be performed, the address lines may change during S6.

State 7

In S7, the processor asserts $\overline{\text{AS}}$, indicating that the address on the address bus is valid. The processor also asserts $\overline{\text{DBEN}}$, which can be used to enable data buffers during S7. In addition, the $\overline{\text{ECS}}$ (and $\overline{\text{OCS}}$, if asserted) signal is negated during S7.

State 8

During S8, the processor places the data to be written onto D0–D31.

State 9

The processor asserts $\overline{\text{DS}}$ during S9 indicating that the data is stable on the data bus. As long as at least one of the $\overline{\text{DSACKx}}$ signals is recognized by the end of S8 (meeting the asynchronous input setup time requirement), the cycle terminates one clock later. If $\overline{\text{DSACKx}}$ is not recognized by the start of S9, the processor inserts wait states instead of proceeding to S10 and S11. To ensure that wait states are inserted, both $\overline{\text{DSACK0}}$ and $\overline{\text{DSACK1}}$ must remain negated throughout the asynchronous input setup and hold times around the end of S8. If wait states are added, the processor continues to sample $\overline{\text{DSACKx}}$ signals on the falling edges of the clock until one is recognized.

The selected device uses $\text{R}/\overline{\text{W}}$, $\overline{\text{DS}}$, SIZ0 – SIZ1 , and A0 – A1 to latch data from the appropriate section(s) of the data bus (D24–D31, D16–D23, D8–D15, and D0–D7). SIZ0 – SIZ1 and A0 – A1 select the data bus sections. If it has not already done so, the device asserts $\overline{\text{DSACKx}}$ when it has successfully stored the data.

State 10

The processor issues no new control signals during S10.

State 11

The processor negates $\overline{AS}$ and $\overline{DS}$ during S11. It holds the address and data valid during S11 to provide address hold time for memory systems. $R/\overline{W}$ and FC0–FC2 also remain valid throughout S11.

If more than one write cycle is required, S6–S11 are repeated for each write cycle.

The external device keeps $\overline{DSACKx}$ asserted until it detects the negation of $\overline{AS}$ or $\overline{DS}$ (whichever it detects first). The device must remove its data and negate $\overline{DSACKx}$ within approximately one clock period after sensing the negation of $\overline{AS}$ or $\overline{DS}$.

7.3.4 Synchronous Read Cycle

A synchronous read cycle is terminated differently from an asynchronous read cycle; otherwise, the cycles assert and respond to the same signals, in the same sequence. $\overline{STERM}$ rather than $\overline{DSACKx}$ is asserted by the addressed external device to terminate a synchronous read cycle. Since $\overline{STERM}$ must meet the synchronous setup and hold times with respect to all rising edges of the clock while $\overline{AS}$ is asserted, it does not need to be synchronized by the processor. Only devices with 32-bit ports may assert $\overline{STERM}$. $\overline{STERM}$ is also used with the $\overline{CBREQ}$ and $\overline{CBACK}$ signals during burst mode operation. It provides a two-clock (minimum) bus cycle for 32-bit ports and single-clock (minimum) burst accesses, although wait states can be inserted for these cycles as well. Therefore, a synchronous cycle terminated with $\overline{STERM}$ with one wait cycle is a three-clock bus cycle. However, note that $\overline{STERM}$ is asserted one-half clock later than $\overline{DSACKx}$ would be for a similar asynchronous cycle with zero wait cycles (also three clocks). Thus, if dynamic bus sizing is not needed, $\overline{STERM}$ can be used to provide more decision time in an external cache design than is available with $\overline{DSACKx}$ for three-clock accesses.

Figure 7-31 is a flowchart of a synchronous long-word read cycle. Byte and word operations are similar. Figure 7-32 is a functional timing diagram of a synchronous long-word read cycle.

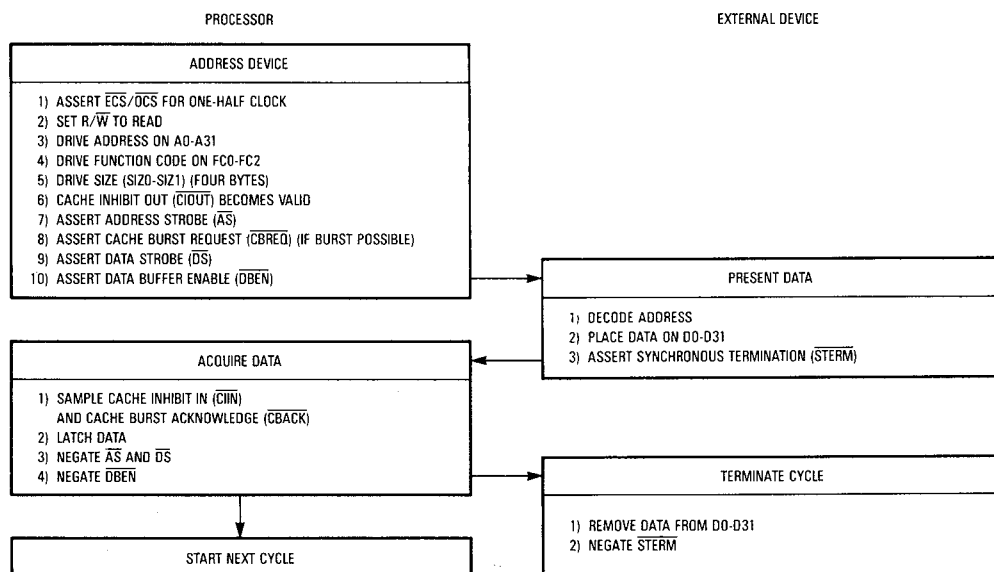


Figure 7-31. Synchronous Long-Word Read Cycle Flowchart — No Burst Allowed

State 0

The read cycle starts with S0. The processor drives $\overline{\text{ECS}}$ low, indicating the beginning of an external cycle. When the cycle is the first cycle of a read operand operation, $\overline{\text{OCS}}$ is driven low at the same time. During S0, the processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the cycle. The processor drives R/ $\overline{\text{W}}$ high for a read cycle and drives $\overline{\text{DBEN}}$ inactive to disable the data buffers. SIZ1–SIZ0 become valid, indicating the number of bytes to be transferred. $\overline{\text{CIOUT}}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TT_x register.

State 1

One-half clock later in S1, the processor asserts $\overline{\text{AS}}$, indicating that the address on the address bus is valid. The processor also asserts $\overline{\text{DS}}$ during S1. If the burst mode is enabled for the appropriate on-chip cache and all four long words of the cache entry are invalid, (i.e., four long words can be read in), $\overline{\text{CBREQ}}$ is asserted. In addition, the $\overline{\text{ECS}}$ (and $\overline{\text{OCS}}$, if asserted) signal is negated during S1.

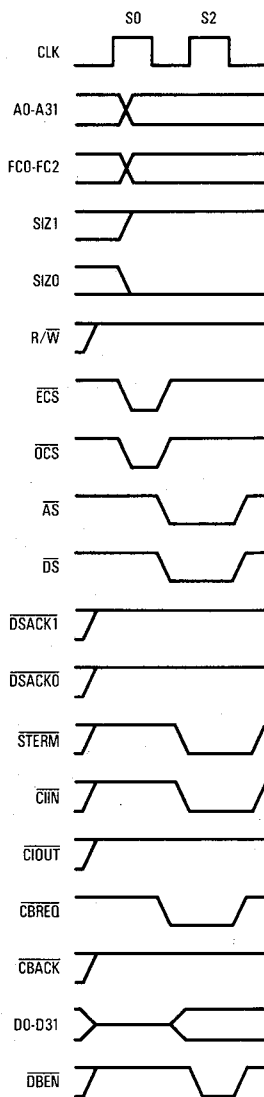


Figure 7-32. Synchronous Read with $\overline{\text{CSIN}}$ Asserted and $\overline{\text{CBACK}}$ Negated

State 2

The selected device uses $\overline{R/W}$, $SIZ0$ – $SIZ1$, $A0$ – $A1$, and $\overline{CIOUT}$ to place its information on the data bus. Any or all of the byte sections of the data bus ($D24$ – $D31$, $D16$ – $D23$, $D8$ – $D15$, and $D0$ – $D7$) are selected by $SIZ0$ – $SIZ1$ and $A0$ – $A1$. During $S2$, the processor drives $\overline{DBEN}$ active to enable external data buffers. In systems that use two-clock synchronous bus cycles, the timing of $\overline{DBEN}$ may prevent its use. At the beginning of $S2$, the processor samples the level of $\overline{STERM}$. If $\overline{STERM}$ is recognized, the processor latches the incoming data at the end of $S2$. If the selected data is not to be cached for the current cycle or if the device cannot supply 32 bits, $\overline{CIIN}$ must be asserted at the same time as $\overline{STERM}$. In addition, the state of $\overline{CBACK}$ is latched when $\overline{STERM}$ is recognized.

Since $\overline{CIIN}$, $\overline{CBACK}$, and $\overline{STERM}$ are synchronous signals, they must meet the synchronous input setup and hold times for all rising edges of the clock while $\overline{AS}$ is asserted. If $\overline{STERM}$ is negated at the beginning of $S2$, wait states are inserted after $S2$, and $\overline{STERM}$ is sampled on every rising edge thereafter until it is recognized. Once $\overline{STERM}$ is recognized, data is latched on the next falling edge of the clock (corresponding to the beginning of $S3$).

State 3

The processor negates $\overline{AS}$, $\overline{DS}$, and $\overline{DBEN}$ during $S3$. It holds the address valid during $S3$ to simplify memory interfaces. $\overline{R/W}$, $SIZ0$ – $SIZ1$, and $\overline{FC0}$ – $\overline{FC2}$ also remain valid throughout $S3$.

The external device must keep its data asserted throughout the synchronous hold time for data from the beginning of $S3$. The device must remove its data within one clock after asserting $\overline{STERM}$ and negate $\overline{STERM}$ within two clocks after asserting $\overline{STERM}$; otherwise, the processor may inadvertently use $\overline{STERM}$ for the next bus cycle.

7.3.5 Synchronous Write Cycle

A synchronous write cycle is terminated differently from an asynchronous write cycle and the data strobe may not be useful. Otherwise, the cycles assert and respond to the same signal, in the same sequence. $\overline{STERM}$ is asserted by the external device to terminate a synchronous write cycle. The discussion of $\overline{STERM}$ in the preceding section applies to write cycles as well as to read cycles.

$\overline{DS}$ is not asserted for two-clock synchronous write cycles; therefore, the clock ($\overline{CLK}$) may be used as the timing signal for latching the data. In addition, there is no time from the latest assertion of $\overline{AS}$ and the required assertion

Figure 7-33 is a flowchart of a synchronous write cycle. Figure 7-34 is a functional timing diagram of this operation with wait states.

Figure 7-33. Synchronous Write Cycle Flowchart

The write cycle starts with S0. The processor drives $\overline{\text{ECS}}$ low, indicating the beginning of an external cycle. When the cycle is the first cycle of a write operation, $\overline{\text{OCS}}$ is driven low at the same time. During S0, the processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the cycle. The processor drives $\text{R}/\overline{\text{W}}$ low for a write cycle. $\text{SIZ0}–\text{SIZ1}$ become valid, indicating the number of bytes to be transferred. $\overline{\text{CIOUT}}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TT_x register.

One-half clock later in S1, the processor asserts $\overline{AS}$, indicating that the address on the address bus is valid. The processor also asserts $\overline{DBEN}$ during S1, which may be used to enable the external data buffers. In addition, the $\overline{ECS}$ (and $\overline{OCS}$, if asserted) signal is negated during S1.

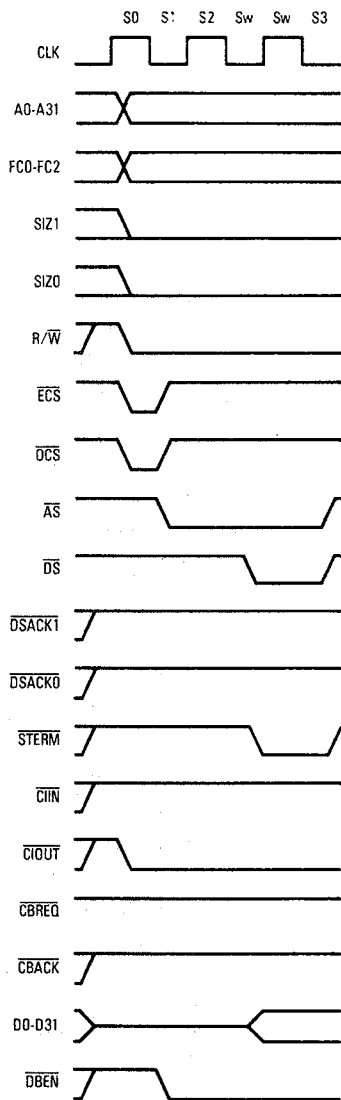


Figure 7-34. Synchronous Write Cycle with Wait States — $\overline{\text{CIOU}}$ Asserted

State 2

During S2, the processor places the data to be written onto D0–D31. The selected device uses $\overline{R/\overline{W}}$, CLK, SIZ0–SIZ1, and A0–A1 to latch data from the appropriate section(s) of the data bus (D24–D31, D16–D23, D8–D15, and D0–D7). SIZ0–SIZ1 and A0–A1 select the data bus sections. The device asserts $\overline{STERM}$ when it has successfully stored the data. If the device does not assert $\overline{STERM}$ by the rising edge of S2, the processor inserts wait states until it is recognized. The processor asserts $\overline{DS}$ at the end of S2 if wait states are inserted. For zero-wait-state synchronous write cycles, $\overline{DS}$ is not asserted.

State 3

The processor negates $\overline{AS}$ (and $\overline{DS}$, if necessary) during S3. It holds the address and data valid during S3 to simplify memory interfaces. $\overline{R/\overline{W}}$, SIZ0–SIZ1, FC0–FC2, and $\overline{DBEN}$ also remain valid throughout S3.

The addressed device must negate $\overline{STERM}$ within two clock periods after asserting it, or the processor may use $\overline{STERM}$ for the next bus cycle.

7.3.6 Synchronous Read-Modify-Write Cycle

A synchronous read-modify-write operation differs from an asynchronous read-modify-write operation only in the terminating signal of the read and write cycles and in the use of CLK instead of $\overline{DS}$ latching data in the write cycle. Like the asynchronous operation, the synchronous read-modify-write operation is indivisible. Although the operation is synchronous, the burst mode is never used during read-modify-write cycles.

Figure 7-35 is a flowchart of the synchronous read-modify-write operation. Timing for the cycle is shown in Figure 7-36.

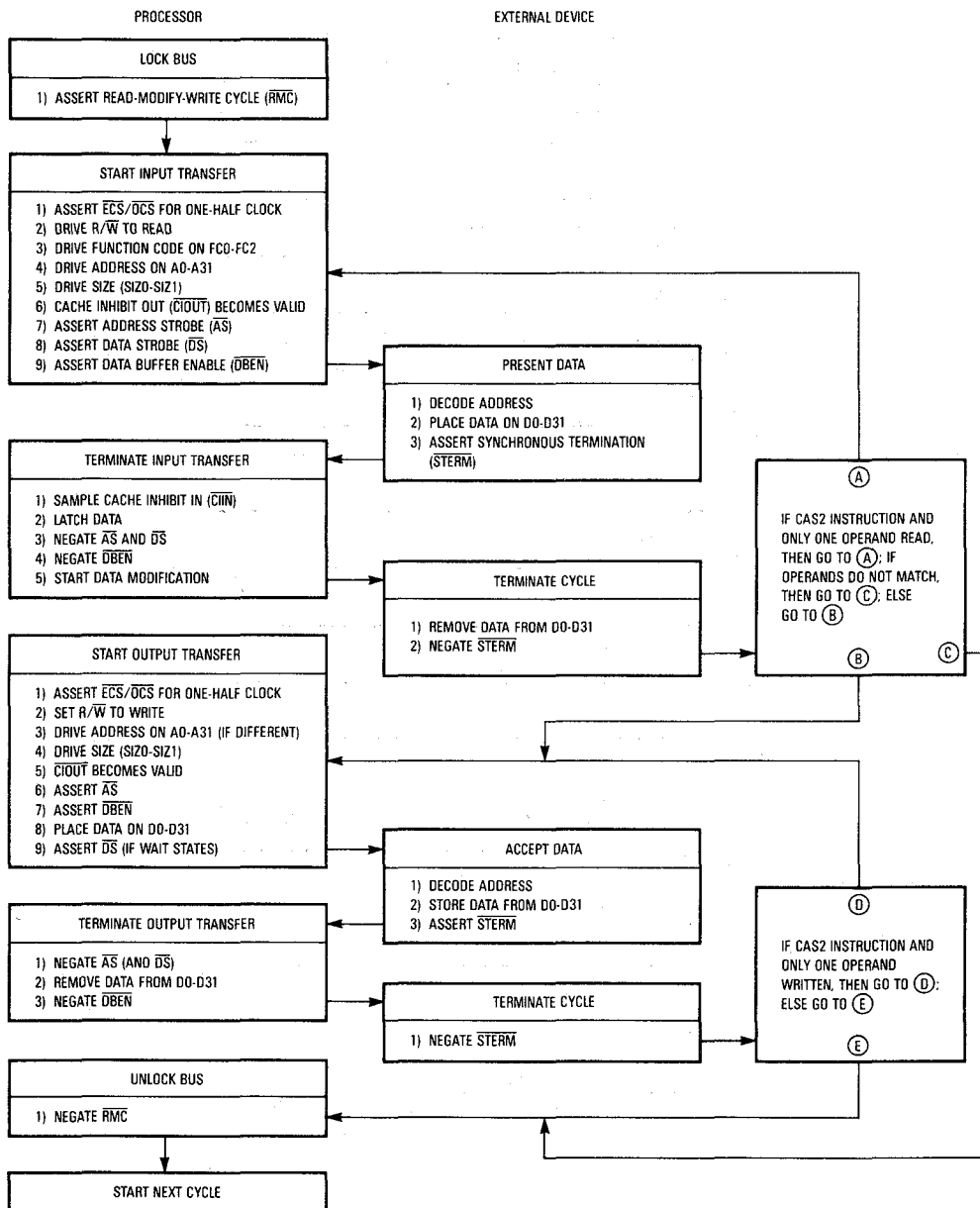


Figure 7-35. Synchronous Read-Modify-Write Cycle Flowchart

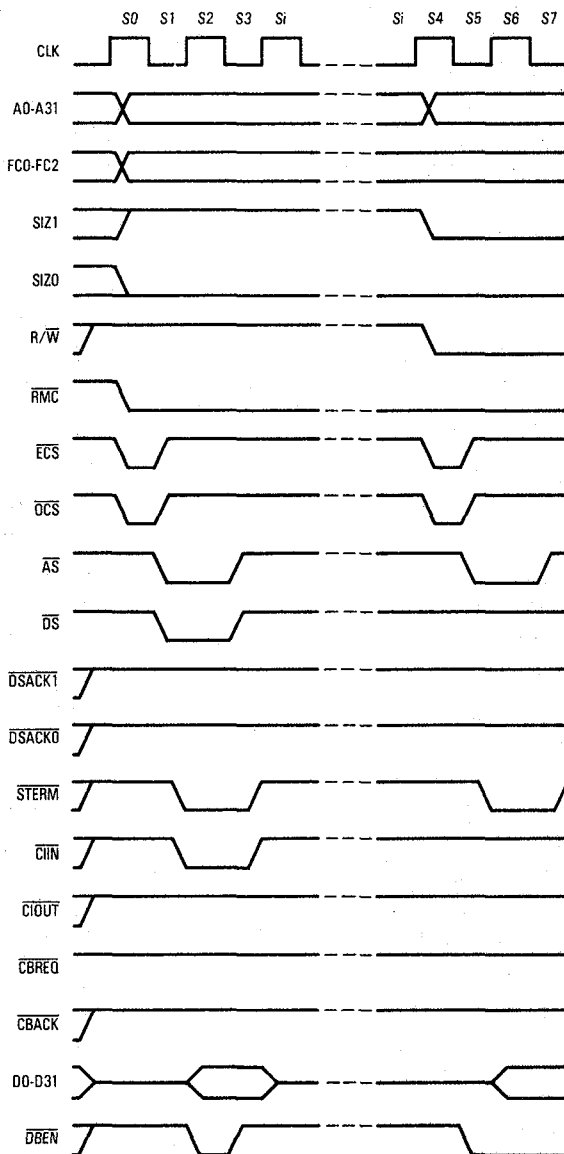


Figure 7-36. Synchronous Read-Modify-Write Cycle Timing — $\overline{CIIN}$ Asserted

State 0

The processor asserts $\overline{\text{ECS}}$ and $\overline{\text{OCS}}$ in S0 to indicate the beginning of an external operand cycle. The processor also asserts $\overline{\text{RMC}}$ in S0 to identify a read-modify-write cycle. The processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the operation. SIZ0–SIZ1 become valid in S0 to indicate the operand size. The processor drives $\overline{\text{R/W}}$ high for a read cycle and sets $\overline{\text{CIOUT}}$ to the value of the MMU CI bit in the address translation descriptor or in the appropriate TTx register. The processor drives $\overline{\text{DBEN}}$ inactive to disable the data buffers.

State 1

One-half clock later in S1, the processor asserts $\overline{\text{AS}}$, indicating that the address on the address bus is valid. The processor also asserts $\overline{\text{DS}}$ during S1. In addition, the $\overline{\text{ECS}}$ (and $\overline{\text{OCS}}$, if asserted) signal is negated during S1.

State 2

The selected device uses $\overline{\text{R/W}}$, SIZ0–SIZ1, A0–A1, and $\overline{\text{CIOUT}}$ to place its information on the data bus. Any or all of the byte sections (D24–D31, D16–D23, D8–D15, and D0–D7) are selected by SIZ0–SIZ1 and A0–A1. During S2, the processor drives $\overline{\text{DBEN}}$ active to enable external data buffers. In systems that use two-clock synchronous bus cycles, the timing of $\overline{\text{DBEN}}$ may prevent its use. At the beginning of S2, the processor samples the level of $\overline{\text{STERM}}$. If $\overline{\text{STERM}}$ is recognized, the processor latches the incoming data. If the selected data is not to be cached for the current cycle or if the device cannot supply 32 bits, $\overline{\text{CIIN}}$ must be asserted at the same time as $\overline{\text{STERM}}$.

Since $\overline{\text{CIIN}}$ and $\overline{\text{STERM}}$ are synchronous signals, they must meet the synchronous input setup and hold times for all rising edges of the clock while $\overline{\text{AS}}$ is asserted. If $\overline{\text{STERM}}$ is negated at the beginning of S2, wait states are inserted after S2, and $\overline{\text{STERM}}$ is sampled on every rising edge thereafter until it is recognized. Once $\overline{\text{STERM}}$ is recognized, data is latched on the next falling edge of the clock (corresponding to the beginning of S3).

State 3

The processor negates $\overline{AS}$, $\overline{DS}$, and $\overline{DBEN}$ during S3. If more than one read cycle is required to read in the operand(s), S0–S3 are repeated accordingly. When finished with the read cycle, the processor holds the address, R/W, and FC0–FC2 valid in preparation for the write portion of the cycle.

The external device must keep its data asserted throughout the synchronous hold time for data from the beginning of S3. The device must remove the data within one-clock cycle after asserting $\overline{STERM}$ to avoid bus contention. It must also negate $\overline{STERM}$ within two clocks after asserting $\overline{STERM}$; otherwise, the processor may inadvertently use $\overline{STERM}$ for the next bus cycle.

Idle States

The processor does not assert any new control signals during the idle states, but it may begin the modify portion of the cycle at this time. The R/W signal remains in the read mode until S4 to prevent bus conflicts with the preceding read portion of the cycle; the data bus is not driven until S6.

7

State 4

The processor asserts $\overline{ECS}$ and $\overline{OCS}$ in S4 to indicate that an external cycle is beginning. The processor drives R/W low for a write cycle. $\overline{CIOUT}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TTx register. Depending on the write operation to be performed, the address lines may change during S4.

State 5

In state 5 (S5), the processor asserts $\overline{AS}$ to indicate that the address on the address bus is valid. The processor also asserts $\overline{DBEN}$ during S5, which can be used to enable external data buffers.

State 6

During S6, the processor places the data to be written onto the D0–D31.

The selected device uses R/W, CLK, SIZ0–SIZ1, and A0–A1 to latch data from the appropriate byte(s) of the data bus (D24–D31, D16–D23, D8–D15, and D0–D7). SIZ0–SIZ1 and A0–A1 select the data bus sections. The device asserts $\overline{STERM}$ when it has successfully stored the data. If the device does not assert $\overline{STERM}$ by the rising edge of S6, the processor inserts wait states until it is recognized. The processor asserts $\overline{DS}$ at the end of S6 if wait states are inserted. Note that for zero-wait-state synchronous write cycles, $\overline{DS}$ is not asserted.

State 7

The processor negates $\overline{AS}$ (and $\overline{DS}$, if necessary) during S7. It holds the address and data valid during S7 to simplify memory interfaces. R/W and FC0–FC2 also remain valid throughout S7.

If more than one write cycle is required, S8–S11 are repeated for each write cycle.

The external device must negate $\overline{STERM}$ within two clock periods after asserting it, or the processor may inadvertently use $\overline{STERM}$ for the next bus cycle.

7.3.7 Burst Operation Cycles

The MC68030 supports a burst mode for filling the on-chip instruction and data caches.

The MC68030 provides a set of handshake control signals for the burst mode. When a miss occurs in one of the caches, the MC68030 initiates a bus cycle to obtain the required data or instruction stream fetch. If the data or instruction can be cached, the MC68030 attempts to fill a cache entry. Depending on the alignment for a data access, the MC68030 may attempt to fill two cache entries. The processor may also assert $\overline{CBREQ}$ to request a burst fill operation. That is, the processor can fill additional entries in the line. The MC68030 allows a burst of as many as four long words.

The mechanism that asserts the $\overline{CBREQ}$ signal for burstable cache entries is enabled by the data burst enable (DBE) and instruction burst enable (IBE) bits of the cache control register (CACR) for the data and instruction caches, respectively. Either of the following conditions cause the MC68030 to initiate a cache burst request (and assert $\overline{CBREQ}$) for a cachable read cycle:

- The logical address and function code signals of the current instruction or data fetch do not match the indexed tag field in the respective instruction or data cache.
- All four long words corresponding to the indexed tag in the appropriate cache are marked invalid.

However, the MC68030 does not assert $\overline{CBREQ}$ during the first portion of a misaligned access if the remainder of the access does not correspond to the same cache line. Refer to **6.1.3.1 SINGLE ENTRY MODE** for details.

If the appropriate cache is not enabled or if the cache freeze bit for the cache is set, the processor does not assert $\overline{\text{CBREQ}}$. $\overline{\text{CBREQ}}$ is not asserted during the read or write cycles of any read-modify-write operation.

The MC68030 allows burst filling only from 32-bit ports that terminate bus cycles with $\overline{\text{STERM}}$ and respond to $\overline{\text{CBREQ}}$ by asserting $\overline{\text{CBACK}}$. When the MC68030 recognizes $\overline{\text{STERM}}$ and $\overline{\text{CBACK}}$ and it has asserted $\overline{\text{CBREQ}}$, it maintains $\overline{\text{AS}}$, $\overline{\text{DS}}$, R/W , A0-A31 , FC0-FC2 , SIZ0-SIZ1 in their current state throughout the burst operation. The processor continues to accept data on every clock during which $\overline{\text{STERM}}$ is asserted until the burst is complete or an abnormal termination occurs.

$\overline{\text{CBACK}}$ indicates that the addressed device can respond to a cache burst request by supplying one more long word of data in the burst mode. It can be asserted independently of the $\overline{\text{CBREQ}}$ signal, and burst mode is only initiated if both of these signals are asserted for a synchronous cycle. If the MC68030 executes a full burst operation and fetches four long words, $\overline{\text{CBREQ}}$ is negated after $\overline{\text{STERM}}$ is asserted for the third cycle, indicating that the MC68030 only requests one more long word (the fourth cycle). $\overline{\text{CBACK}}$ can then be negated, and the MC68030 latches the data for the fourth cycle and completes the cache line fill.

7

The following conditions can abort a burst fill:

- $\overline{\text{CIIN}}$ asserted,
- $\overline{\text{BERR}}$ asserted, or
- $\overline{\text{CBACK}}$ negated prematurely.

The processing of a bus error during a burst fill operation is described in **7.5.1 Bus Errors**.

For the purposes of halting the processor or arbitrating the bus away from the processor with $\overline{\text{BR}}$, a burst operation is a single cycle since $\overline{\text{AS}}$ remains asserted during the entire operation. If the $\overline{\text{HALT}}$ signal is asserted during a burst operation, the processor halts at the end of the operation. Refer to **7.5.3 Halt Operation** for more information about the halt operation. An alternate bus master requesting the bus with $\overline{\text{BR}}$ may become bus master at the end of the operation provided $\overline{\text{BR}}$ is asserted early enough to be internally synchronized before another processor cycle begins. Refer to **7.7 BUS ARBITRATION** for more information about bus arbitration.

The simultaneous assertion of $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ during a bus cycle normally indicates that the cycle should be retried. However, during the second, third, or fourth cycle of a burst operation, this signal combination indicates a bus error condition, which aborts the burst operation. In addition, the processor remains in the halted state until $\overline{\text{HALT}}$ is negated. For information about bus error processing, refer to **7.5.1. Bus Errors**.

Figure 7-37 is a flowchart of the burst operation. The following timing diagrams show various burst operations. Figure 7-38 shows burst operations for long-word requests with two wait states inserted in the first access and one wait cycle inserted in the subsequent accesses. Figure 7-39 shows a burst operation that fails to complete normally due to $\overline{\text{CBACK}}$ negating prematurely. Figure 7-40 shows a burst operation that is deferred because the entire operand does not correspond to the same cache line. Figure 7-41 shows a burst operation aborted by $\overline{\text{CIIN}}$. Because $\overline{\text{CBACK}}$ corresponds to the *next* cycle, three long words are transferred even though $\overline{\text{CBACK}}$ is only asserted for two clock periods.

The burst operation sequence begins with states S0–S3, which are very similar to those states for a synchronous read cycle except that $\overline{\text{CBREQ}}$ is asserted. S4–S9 perform the final three reads for a complete burst operation.

State 0

The burst operation starts with S0. The processor drives $\overline{\text{ECS}}$ low, indicating the beginning of an external cycle. When the cycle is the first cycle of a read operation, $\overline{\text{OCS}}$ is driven low at the same time. During S0, the processor places a valid address on A0–A31 and valid function codes on FC0–FC2. The function codes select the address space for the cycle. The processor drives R/W high, indicating a read cycle, and drives $\overline{\text{DBEN}}$ inactive to disable the data buffers. SIZ0–SIZ1 become valid, indicating the number of operand bytes to be transferred. $\overline{\text{CIOUT}}$ also becomes valid, indicating the state of the MMU CI bit in the address translation descriptor or in the appropriate TTx register.

State 1

One-half clock later in S1, the processor asserts $\overline{\text{AS}}$ to indicate that the address on the address bus is valid. The processor also asserts $\overline{\text{DS}}$ during S1. $\overline{\text{CBREQ}}$ is also asserted, indicating that the MC68030 can perform a burst operation into one of its caches and can read in four long words. In addition, $\overline{\text{ECS}}$ (and $\overline{\text{OCS}}$, if asserted) is negated during S1.

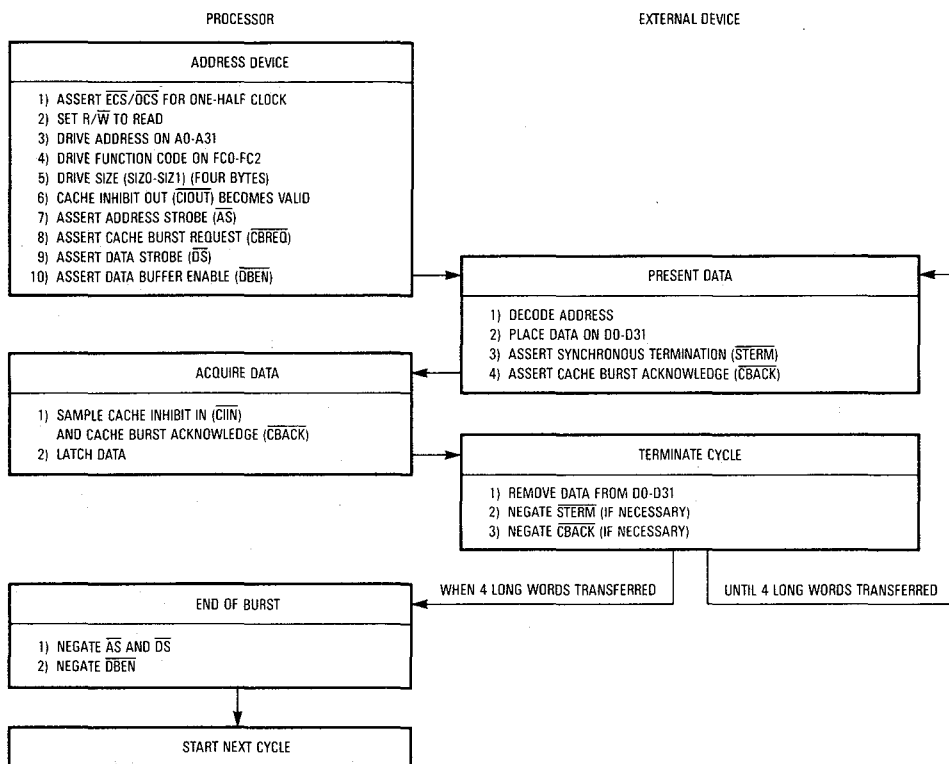


Figure 7-37. Burst Operation Flowchart — Four Long Words Transferred

State 2

The selected device uses $\overline{R/W}$, SIZ0–SIZ1, A0–A1, and $\overline{CIOUT}$ to place the data on the data bus. (The first cycle must supply the long word at the corresponding long-word boundary.) All of the byte sections (D24–D31, D16–D23, D8–D15, and D0–D7) of the data bus must be driven since the burst operation latches 32 bits on every cycle. During S2, the processor drives $\overline{DBEN}$ active to enable external data buffers. In systems that use two-clock synchronous bus cycles, the timing of $\overline{DBEN}$ may prevent its use. At the beginning of S2, the processor tests the level of $\overline{STERM}$. If $\overline{STERM}$ is recognized, the processor latches the incoming data at the end of S2. For the burst operation to proceed, $\overline{CBACK}$ must be asserted when $\overline{STERM}$ is recognized. If the data for the current cycle is not to be cached, $\overline{CIIN}$ must be asserted at the same time as $\overline{STERM}$. The assertion of $\overline{CIIN}$ also has the effect of aborting the burst operation.

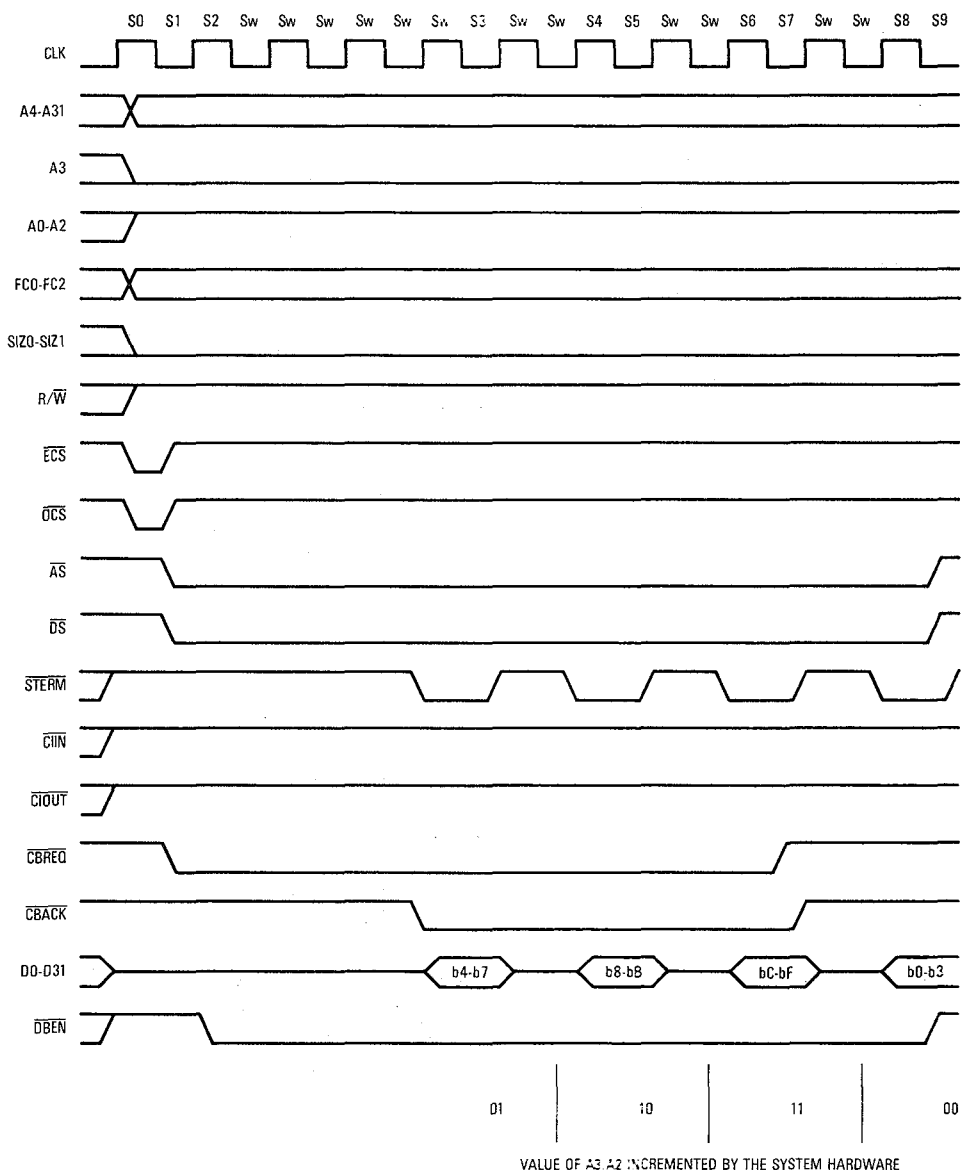


Figure 7-38. Long-Word Operand Request from \$07 with Burst Request and Wait Cycle

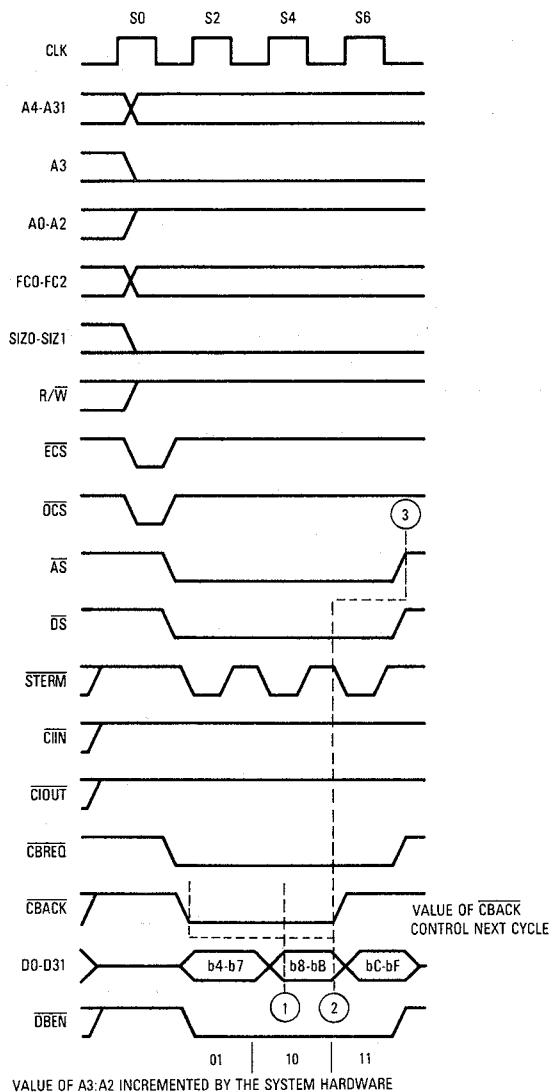


Figure 7-39. Long-Word Operand Request from \$07 with Burst Request — CBACK Negated Early

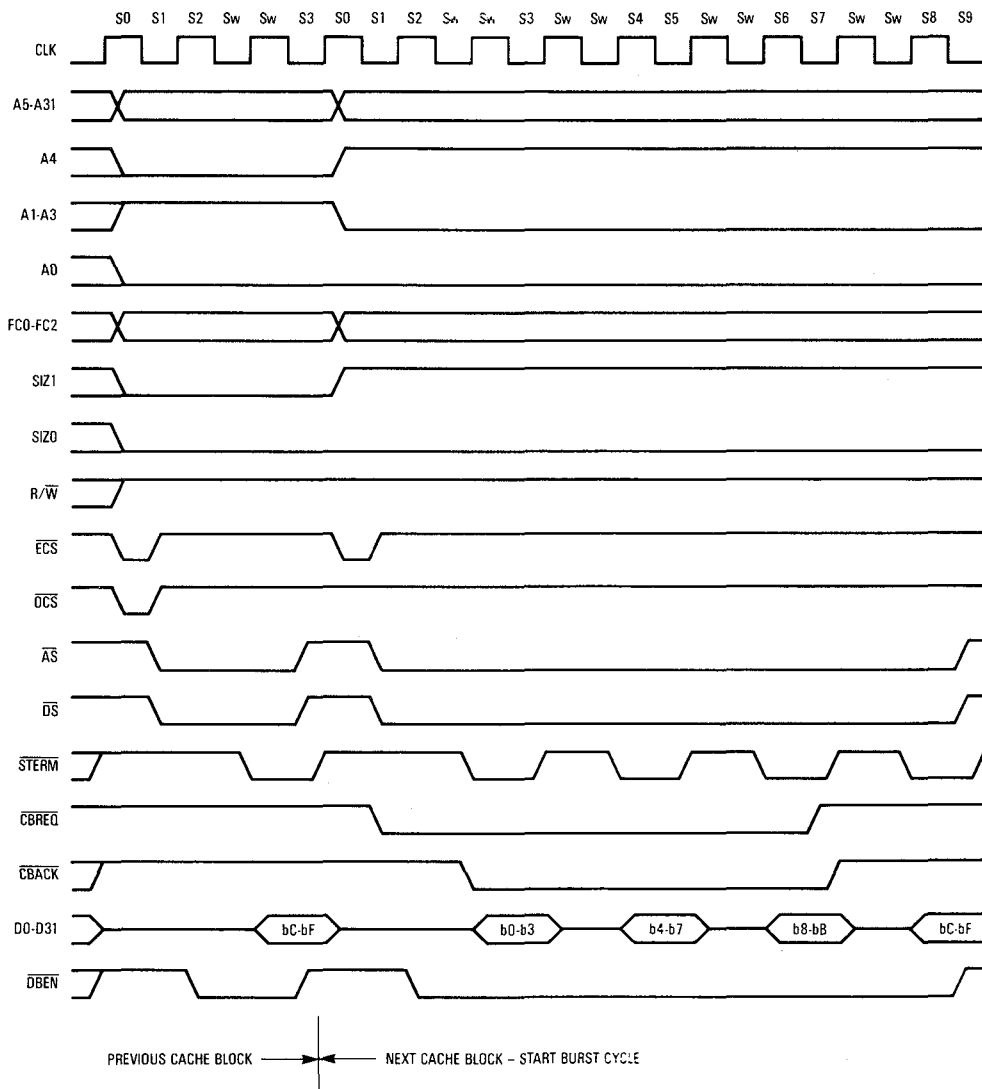


Figure 7-40. Long-Word Operand Request from S0E — Burst Fill Deferred

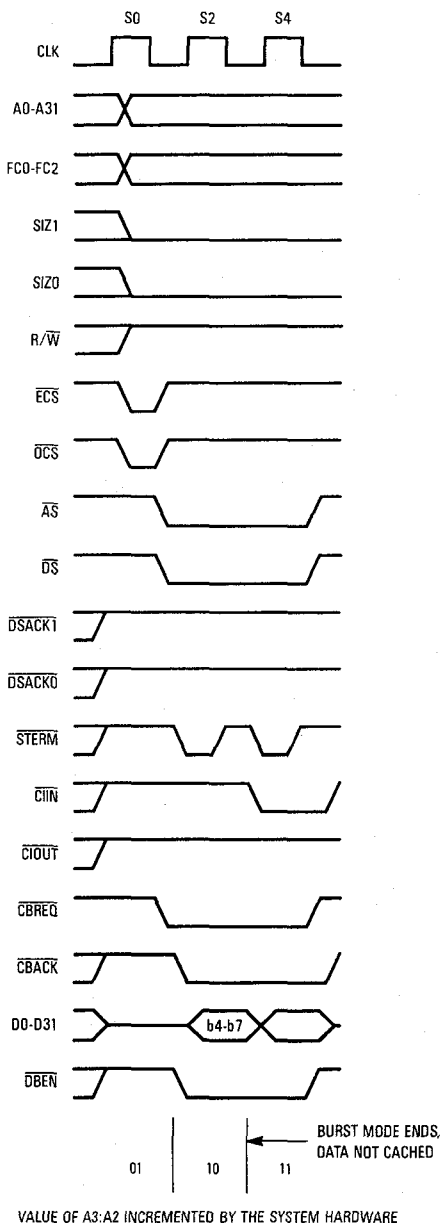


Figure 7-41. Long-Word Operand Request from \$07 with Burst Request — CBACK and CIIN Asserted

Since $\overline{\text{CIIN}}$, $\overline{\text{CBACK}}$, and $\overline{\text{STERM}}$ are synchronous signals, they must meet the synchronous input setup and hold times for all rising edges of the clock while $\overline{\text{AS}}$ is asserted. If $\overline{\text{STERM}}$ is negated at the beginning of S2, wait states are inserted after S2, and $\overline{\text{STERM}}$ is sampled on every rising edge of the clock thereafter until it is recognized. Once $\overline{\text{STERM}}$ is recognized, data is latched on the next falling edge of the clock (corresponding to the beginning of S3).

State 3

The processor maintains $\overline{\text{AS}}$, $\overline{\text{DS}}$, and $\overline{\text{DBEN}}$ asserted during S3. It also holds the address valid during S3 for continuation of the burst. $\text{R}/\overline{\text{W}}$, SIZ0-SIZ1 , and FC0-FC2 also remain valid throughout S3.

The external device must keep the data driven throughout the synchronous hold time for data from the beginning of S3. The device must negate $\overline{\text{STERM}}$ within one clock after asserting $\overline{\text{STERM}}$; otherwise, the processor may inadvertently use $\overline{\text{STERM}}$ prematurely for the next burst access. $\overline{\text{STERM}}$ need not be negated if subsequent accesses do not require wait cycles.

State 4

At the beginning of S4, the processor tests the level of $\overline{\text{STERM}}$. This state signifies the beginning of burst mode, and the remaining states correspond to burst fill cycles. If $\overline{\text{STERM}}$ is recognized, the processor latches the incoming data at the end of S4. This data corresponds to the second long word of the burst. If $\overline{\text{STERM}}$ is negated at the beginning of S4, wait states are inserted instead of S4 and S5, and $\overline{\text{STERM}}$ is sampled on every rising edge of the clock thereafter until it is recognized. As for synchronous cycles, the states of $\overline{\text{CBACK}}$ and $\overline{\text{CIIN}}$ are latched at the time $\overline{\text{STERM}}$ is recognized. The assertion of $\overline{\text{CBACK}}$ at this time indicates that the burst operation should continue, and the assertion of $\overline{\text{CIIN}}$ indicates that the data latched at the end of S4 should not be cached and that the burst should abort.

State 5

The processor maintains all the signals on the bus driven throughout S5 for continuation of the burst. The same hold times for $\overline{\text{STERM}}$ and data described for S3 apply here.

State 6

This state is identical to S4 except that once $\overline{\text{STERM}}$ is recognized, the third long word of data for the burst is latched at the end of S6.

State 7

During this state, the processor negates $\overline{\text{CBREQ}}$, and the memory device may negate $\overline{\text{CBACK}}$. Aside from this, all other bus signals driven by the processor remain driven. The same hold times for $\overline{\text{STERM}}$ and data described for S3 apply here.

State 8

This state is identical to S4 except that $\overline{\text{CBREQ}}$ is negated, indicating that the processor cannot continue to accept more data after this. The data latched at the end of S8 corresponds to the fourth long word of the burst.

State 9

The processor negates $\overline{\text{AS}}$, $\overline{\text{DS}}$, and $\overline{\text{DBEN}}$ during S9. It holds the address, $\text{R}/\overline{\text{W}}$, SIZ0-SIZ1 , and FC0-FC2 valid throughout S9. The same hold times for data described for S3 apply here.

Note that the address bus of the MC68030 remains driven to a constant value for the duration of a burst transfer operation (including the first transfer before burst mode is entered). If an external memory system requires incrementing of the long-word base address to supply successive long words of information, this function must be performed by external hardware. Additionally, in the case of burst transfers that cross a 16-byte boundary (i.e., the first long word transferred is not located at $\text{A3/A2}=00$), the external hardware must correctly control the continuation or termination of the burst transfer as desired. The burst may be terminated by negating $\overline{\text{CBACK}}$ during the transfer of the most significant long word of the 16-byte image ($\text{A3/A2}=11$) or may be continued (with $\overline{\text{CBACK}}$ asserted) by providing the long word located at $\text{A3/A2}=00$ (i.e., the count sequence wraps back to zero and continues as necessary). The MC68030 caches assume the higher order address lines (A4-A31) remain unchanged as the long-word accesses wrap back around to $\text{A3/A2}=00$.

7.4 CPU SPACE CYCLES

FC0-FC2 select user and supervisor program and data areas as listed in Table 4-1. The area selected by $\text{FC0-FC2}=\$7$ is classified as the CPU space. The interrupt acknowledge, breakpoint acknowledge, and coprocessor communication cycles described in the following sections utilize CPU space.

The CPU space type is encoded on A16–A19 during a CPU space operation and indicates the function that the processor is performing. On the MC68030, three of the encodings are implemented as shown in Figure 7-42. All unused values are reserved by Motorola for future additional CPU space types.

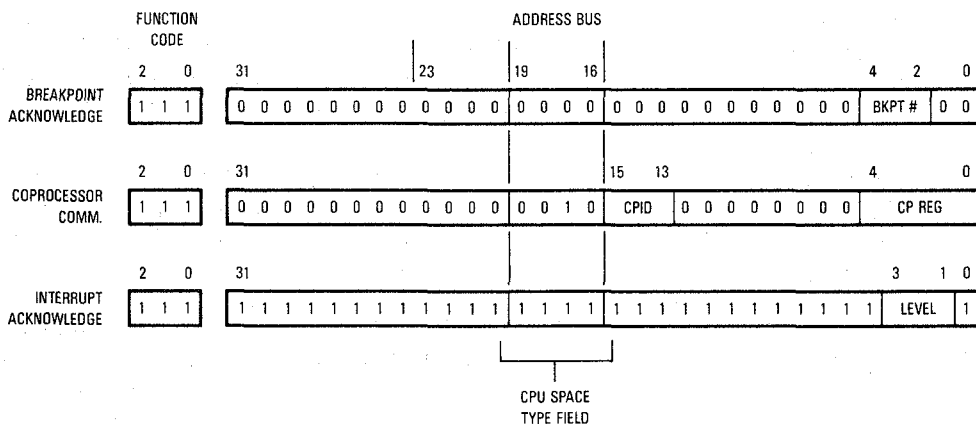


Figure 7-42. MC68030 CPU Space Address Encoding

7.4.1 Interrupt Acknowledge Bus Cycles

When a peripheral device signals the processor (with the $\overline{\text{IPL0}}\text{--}\overline{\text{IPL2}}$ signals) that the device requires service, and the internally synchronized value on these signals indicates a higher priority than the interrupt mask in the status register (or that a transition has occurred in the case of a level 7 interrupt), the processor makes the interrupt a pending interrupt. Refer to **8.1.9 Interrupt Exceptions** for details on the recognition of interrupts.

The MC68030 takes an interrupt exception for a pending interrupt within one instruction boundary (after processing any other pending exception with a higher priority). The following paragraphs describe the various kinds of interrupt acknowledge bus cycles that can be executed as part of interrupt exception processing.

7.4.1.1 INTERRUPT ACKNOWLEDGE CYCLE — TERMINATED NORMALLY. When the MC68030 processes an interrupt exception, it performs an interrupt acknowledge cycle to obtain the number of the vector that contains the starting location of the interrupt service routine.

Some interrupting devices have programmable vector registers that contain the interrupt vectors for the routines they use. The following paragraphs describe the interrupt acknowledge cycle for these devices. Other interrupting conditions or devices cannot supply a vector number and use the autovector cycle described in **7.4.1.2 AUTOVECTOR INTERRUPT ACKNOWLEDGE CYCLE**.

The interrupt acknowledge cycle is a read cycle. It differs from the asynchronous read cycle described in **7.3.1 Asynchronous Read Cycle** or the synchronous read cycle described in **7.3.4 Synchronous Read Cycle** in that it accesses the CPU address space. Specifically, the differences are:

1. FC0–FC2 are set to seven (FC0/FC1/FC2 = 111) for CPU address space.
2. A1, A2, and A3 are set to the interrupt request level (the inverted values of $\overline{\text{IPL0}}$, $\overline{\text{IPL1}}$, and $\overline{\text{IPL2}}$, respectively).
3. The CPU space type field (A16–A19) is set to \$F, the interrupt acknowledge code.
4. A20–A31, A4–A15, and A0 are set to one.

The responding device places the vector number on the data bus during the interrupt acknowledge cycle. Beyond this, the cycle is terminated normally with either $\overline{\text{STERM}}$ or $\overline{\text{DSACKx}}$. Figure 7-43 is the flowchart of the interrupt acknowledge cycle.

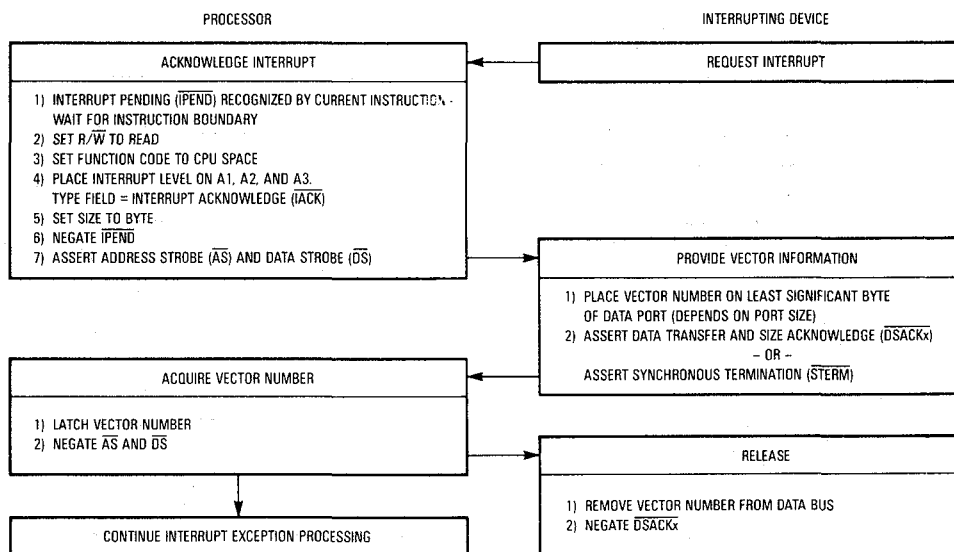


Figure 7-43. Interrupt Acknowledge Cycle Flowchart

Figure 7-44 shows the timing for an interrupt acknowledge cycle terminated with $\overline{DSACKx}$.

7.4.1.2 AUTOVECTOR INTERRUPT ACKNOWLEDGE CYCLE. When the interrupting device cannot supply a vector number, it requests an automatically generated vector or autovector. Instead of placing a vector number on the data bus and asserting $\overline{DSACKx}$ or $\overline{STERM}$, the device asserts the autovector signal ($\overline{AVEC}$) to terminate the cycle. Neither $\overline{STERM}$ nor $\overline{DSACKx}$ may be asserted during an interrupt acknowledge cycle terminated by $\overline{AVEC}$.

The vector number supplied in an autovector operation is derived from the interrupt level of the current interrupt. When $\overline{AVEC}$ is asserted instead of $\overline{DSACK}$ or $\overline{STERM}$ during an interrupt acknowledge cycle, the MC68030 ignores the state of the data bus and internally generates the vector number, the sum of the interrupt level plus 24 (\$18). There are seven distinct autovectors that can be used, corresponding to the seven levels of interrupt available with signals $\overline{IPL0}$ – $\overline{IPL2}$. Figure 7-45 shows the timing for an autovector operation.

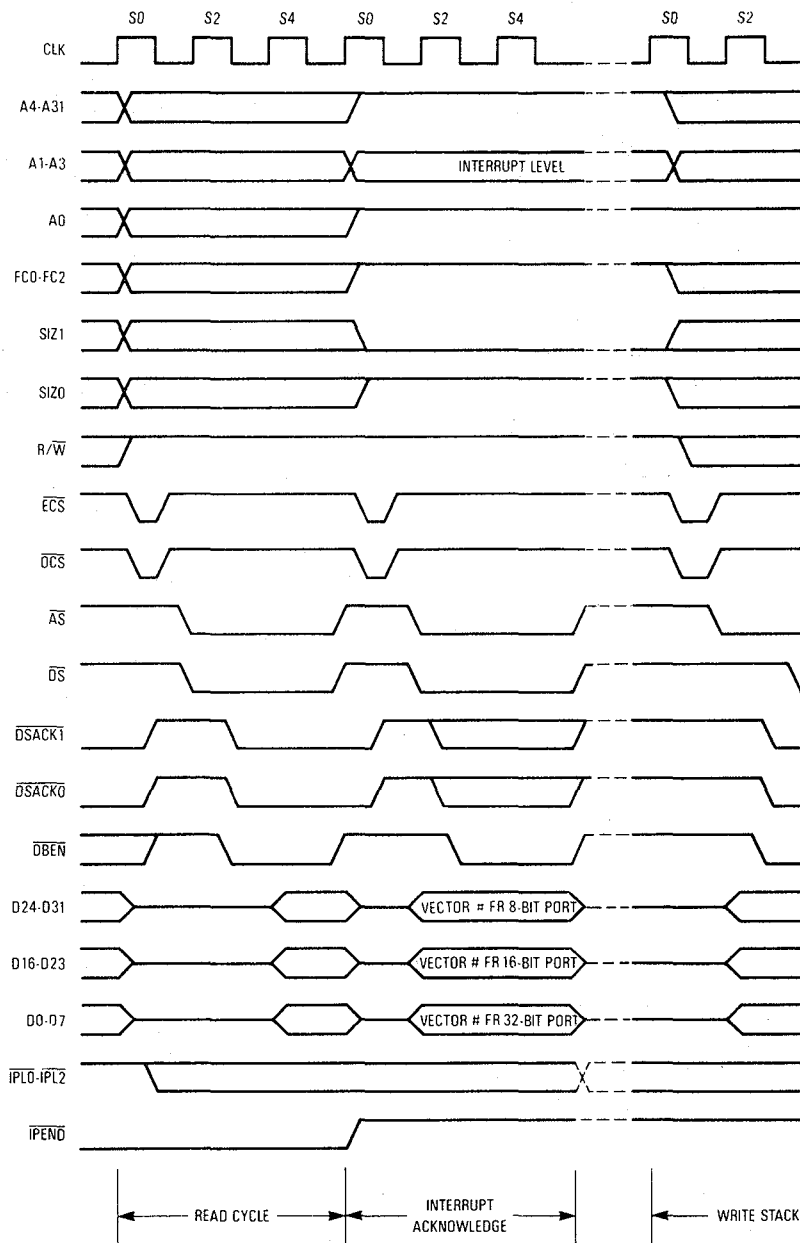


Figure 7-44. Interrupt Acknowledge Cycle Timing

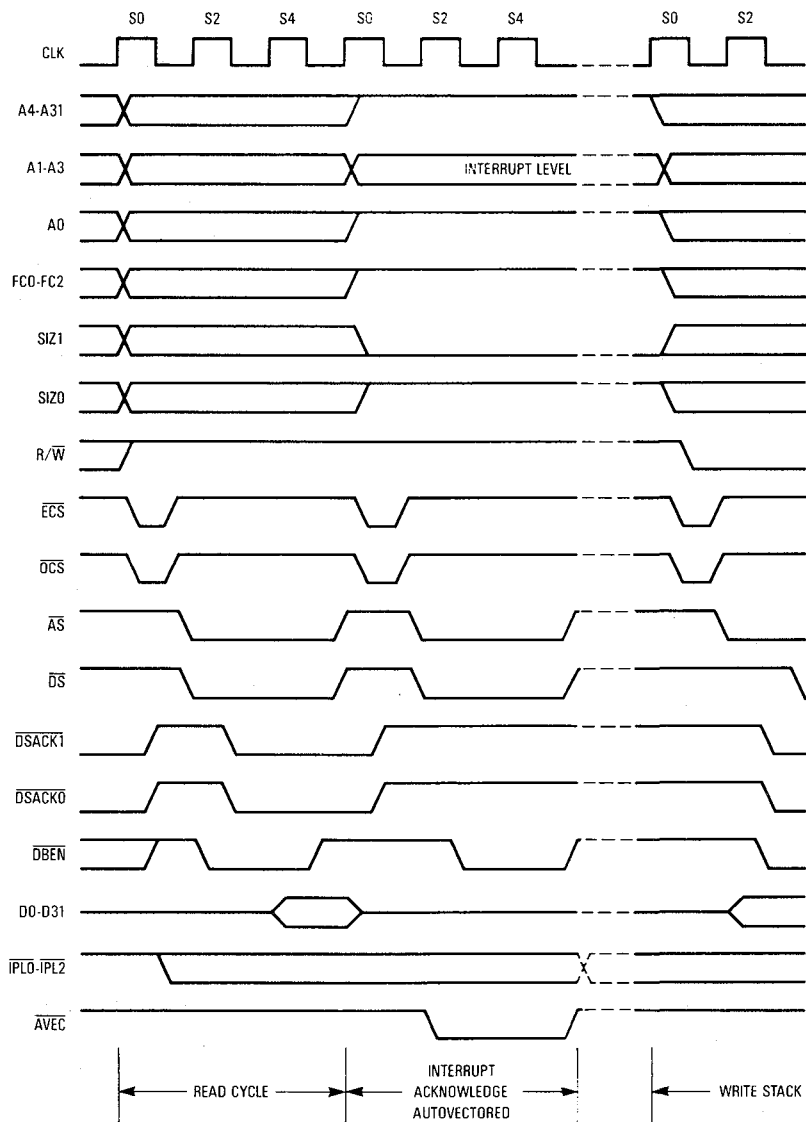


Figure 7-45. Autovector Operation Timing

7.4.1.3 SPURIOUS INTERRUPT CYCLE. When a device does not respond to an interrupt acknowledge cycle with $\overline{AVEC}$, $\overline{STERM}$, or $\overline{DSACKx}$, the external logic typically returns $\overline{BERR}$. The MC68030 automatically generates the spurious interrupt vector number, 24, instead of the interrupt vector number in this case. If $\overline{HALT}$ is also asserted, the processor retries the cycle.

7.4.2 Breakpoint Acknowledge Cycle

The breakpoint acknowledge cycle is generated by the execution of a breakpoint instruction (BKPT). The breakpoint acknowledge cycle allows the external hardware to provide an instruction word directly into the instruction pipeline as the program executes. This cycle accesses the CPU space with a type field of zero and provides the breakpoint number specified by the instruction on address lines A2–A4. If the external hardware terminates the cycle with $\overline{DSACKx}$ or $\overline{STERM}$, the data on the bus (an instruction word) is inserted into the instruction pipe, replacing the breakpoint opcode, and is executed after the breakpoint acknowledge cycle completes. The breakpoint instruction requires a word to be transferred so that if the first bus cycle accesses an 8-bit port, a second cycle is required. If the external logic terminates the breakpoint acknowledge cycle with $\overline{BERR}$ (i.e., no instruction word available), the processor takes an illegal instruction exception. Figure 7-46 is a flowchart of the breakpoint acknowledge cycle. Figure 7-47 shows the timing for a breakpoint acknowledge cycle that returns an instruction word. Figure 7-48 shows the timing for a breakpoint acknowledge cycle that signals an exception.

7.4.3 Coprocessor Communication Cycles

The MC68030 coprocessor interface provides instruction-oriented communication between the processor and as many as seven coprocessors. The bus communication required to support coprocessor operations uses the MC68030 CPU space with a type field of \$2.

Coprocessor accesses use the MC68030 bus protocol except that the address bus supplies access information rather than a 32-bit address. The CPU space type field (A16–A19) for a coprocessor operation is \$2. A13–A15 contain the coprocessor identification number (CpID), and A0–A4 specify the coprocessor interface register to be accessed. Coprocessor accesses to a CpID of zero correspond to MMU instructions and are not generated by the MC68030 as a result of the coprocessor interface. These cycles can only be generated by the MOVES instruction. Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for further information.

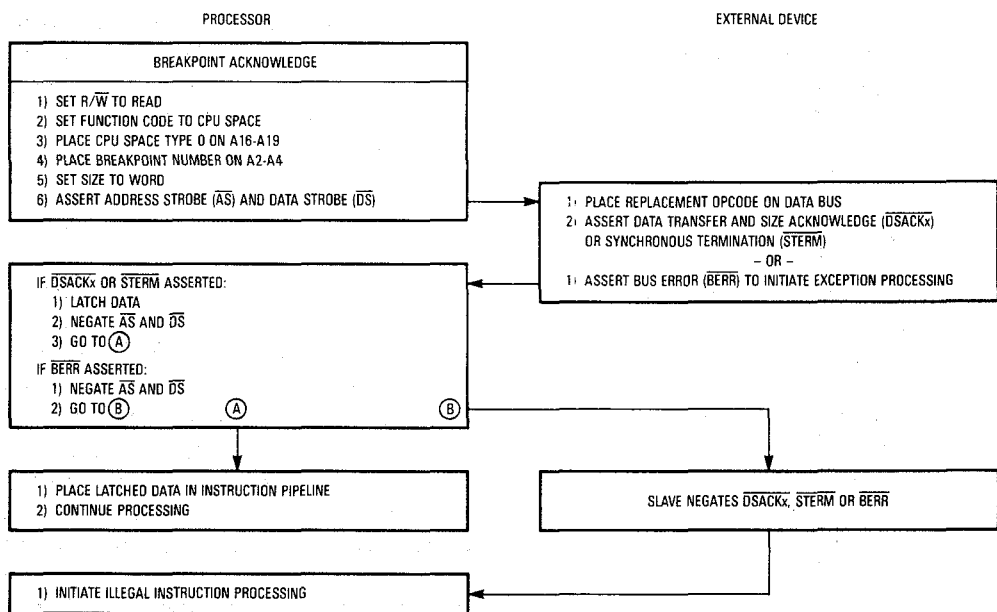


Figure 7-46. Breakpoint Operation Flow

7.5 BUS EXCEPTION CONTROL CYCLES

The MC68030 bus architecture requires assertion of either $\overline{DSACKx}$ or $\overline{STERM}$ from an external device to signal that a bus cycle is complete. $\overline{DSACKx}$, $\overline{STERM}$, or $\overline{AVEC}$ is not asserted if:

- The external device does not respond.
- No interrupt vector is provided.
- Various other application-dependent errors occur.

External circuitry can provide $\overline{BERR}$ when no device responds by asserting $\overline{DSACKx}$, $\overline{STERM}$, or $\overline{AVEC}$ within an appropriate period of time after the processor asserts $\overline{AS}$. This allows the cycle to terminate and the processor to enter exception processing for the error condition.

The MMU can also detect an internal bus error. This occurs when the processor attempts to access an address in a protected area of memory (a user program attempts to access supervisor data, for example) or after the MMU receives a bus error while searching the address table for an address translation description.

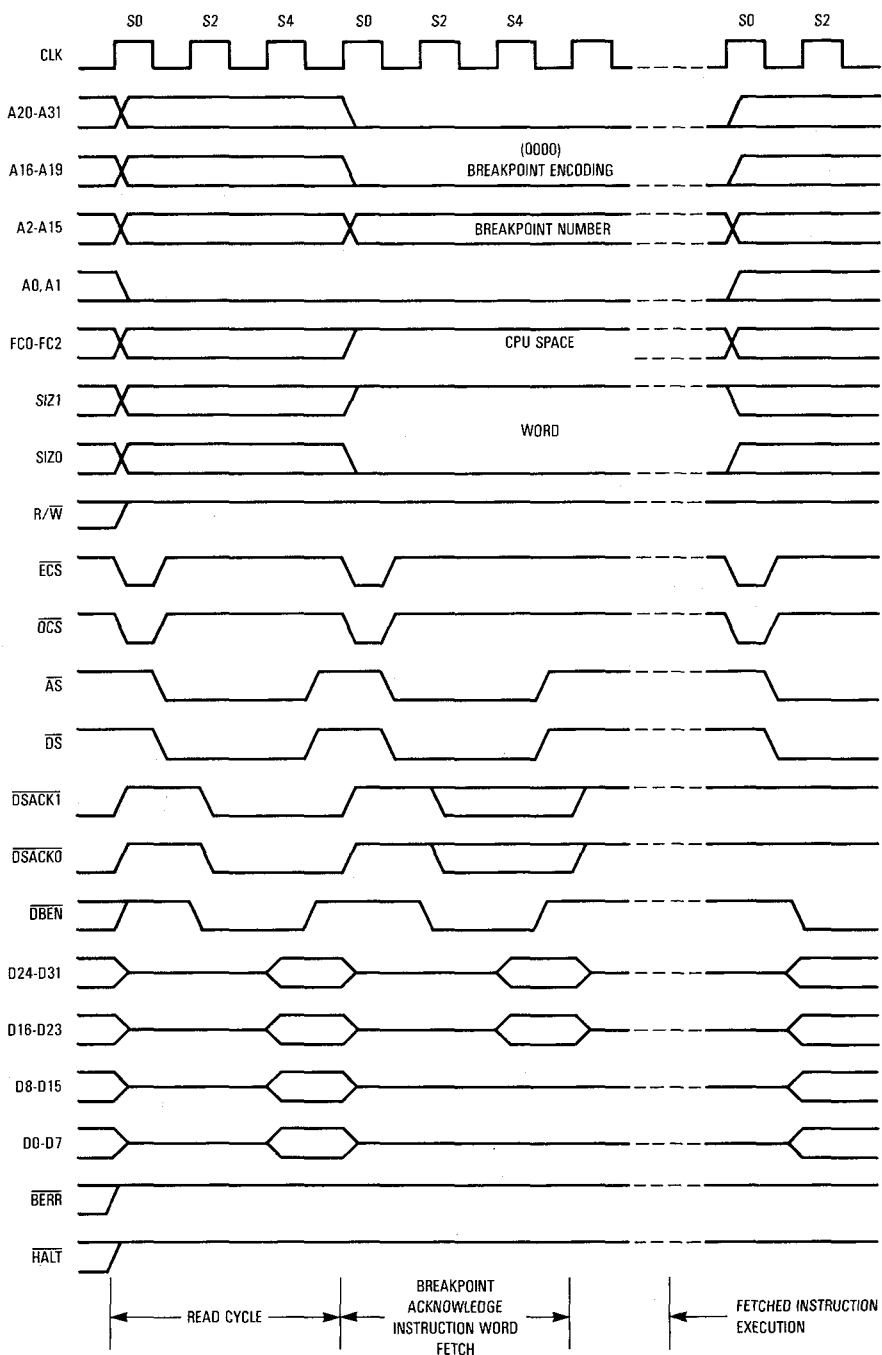


Figure 7-47. Breakpoint Acknowledge Cycle Timing

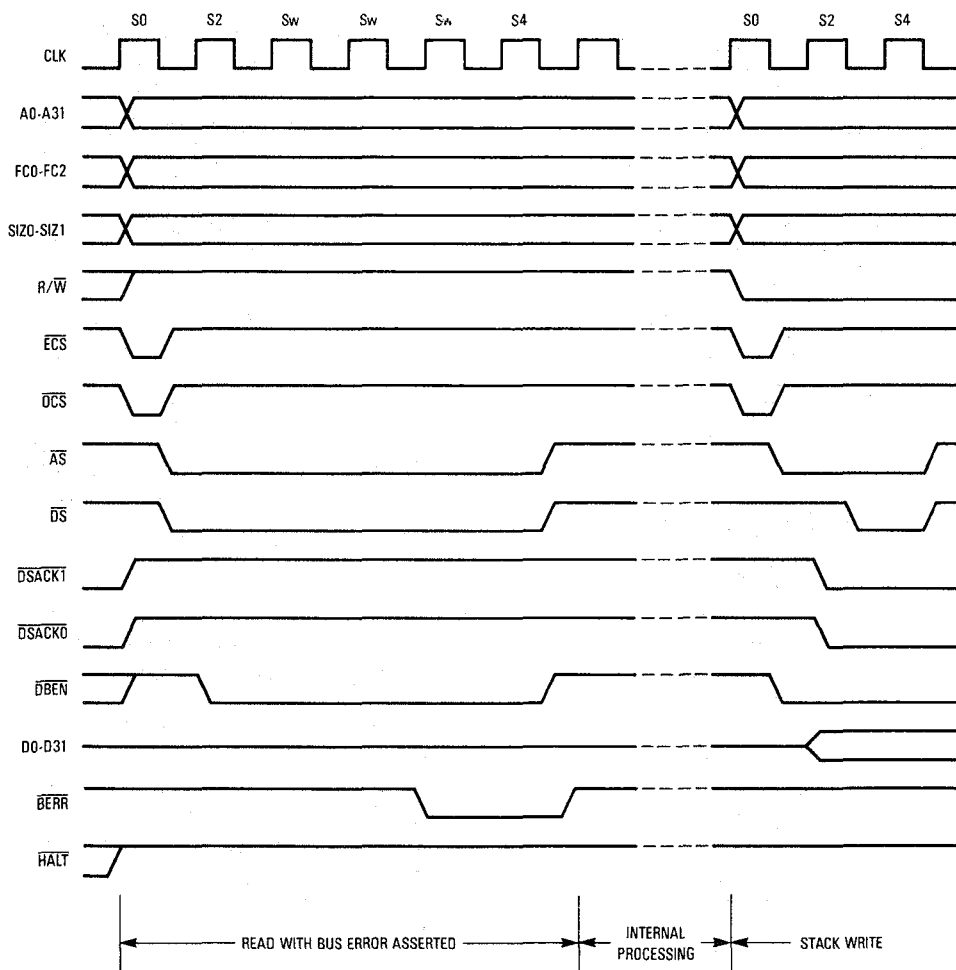


Figure 7-48. Breakpoint Acknowledge Cycle Timing (Exception Signaled)

Another signal that is used for bus exception control is $\overline{\text{HALT}}$. This signal can be asserted by an external device for debugging purposes to cause single bus cycle operation or (in combination with $\overline{\text{BERR}}$) a retry of a bus cycle in error.

To properly control termination of a bus cycle for a retry or a bus error condition, $\overline{\text{DSACKx}}$, $\overline{\text{BERR}}$, and $\overline{\text{HALT}}$ can be asserted and negated with the rising edge of the MC68030 clock. This assures that when two signals are asserted simultaneously, the required setup time (#47A) and hold time (#47B) for both of them is met for the same falling edge of the processor clock. (Refer to MC68030EC/D, *MC68030 Electrical Specifications* for timing requirements.) This or some equivalent precaution should be designed into the external circuitry that provides these signals.

The acceptable bus cycle terminations for asynchronous cycles are summarized in relation to $\overline{\text{DSACKx}}$ assertion as follows (case numbers refer to Table 7-8):

Normal Termination:

$\overline{\text{DSACKx}}$ is asserted; $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ remain negated (case 1).

Halt Termination:

$\overline{\text{HALT}}$ is asserted at same time or before $\overline{\text{DSACKx}}$, and $\overline{\text{BERR}}$ remains negated (case 2).

Bus Error Termination:

$\overline{\text{BERR}}$ is asserted in lieu of, at the same time, or before $\overline{\text{DSACKx}}$ (case 3) or after $\overline{\text{DSACKx}}$ (case 4), and $\overline{\text{HALT}}$ remains negated; $\overline{\text{BERR}}$ is negated at the same time or after $\overline{\text{DSACKx}}$.

Retry Termination:

$\overline{\text{HALT}}$ and $\overline{\text{BERR}}$ are asserted in lieu of, at the same time, or before $\overline{\text{DSACKx}}$ (case 5) or after $\overline{\text{DSACKx}}$ (case 6); $\overline{\text{BERR}}$ is negated at the same time or after $\overline{\text{DSACKx}}$; $\overline{\text{HALT}}$ may be negated at the same time or after $\overline{\text{BERR}}$.

Table 7-8. DSACK, BERR, and HALT Assertion Results

Case No.	Control Signal	Asserted on Rising Edge of State		Result
		N	N + 2	
1	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A NA NA	S NA X	Normal cycle terminate and continue.
2	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A NA A/S	S NA S	Normal cycle terminate and halt. Continue when $\overline{\text{HALT}}$ negated.
3	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	NA/A A NA	X S NA	Terminate and take bus error exception, possibly deferred.
4	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A NA NA	X A NA	Terminate and take bus error exception, possibly deferred.
5	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	NA/A A A/S	X S S	Terminate and retry when $\overline{\text{HALT}}$ negated.
6	$\overline{\text{DSACKx}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A NA NA	X A A	Terminate and retry when $\overline{\text{HALT}}$ negated.

LEGEND:

- N — The number of current even bus state (e.g., S2, S4, etc.)
- A — Signal is asserted in this bus state
- NA — Signal is not asserted in this state
- X — Don't care
- S — Signal was asserted in previous state and remains asserted in this state

Table 7-8 shows various combinations of control signal sequences and the resulting bus cycle terminations. To ensure predictable operation, $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ should be negated according to the specifications in MC68030EC/D, *MC68030 Electrical Specifications*. $\overline{\text{DSACKx}}$, $\overline{\text{BERR}}$, and $\overline{\text{HALT}}$ may be negated after AS. If $\overline{\text{DSACKx}}$ or $\overline{\text{BERR}}$ remain asserted into S2 of the next bus cycle, that cycle may be terminated prematurely.

The termination signal for a synchronous cycle is $\overline{\text{STERM}}$. An analogous set of bus cycle termination cases exists in relationship to $\overline{\text{STERM}}$ assertion. Note that $\overline{\text{STERM}}$ and $\overline{\text{DSACKx}}$ must never both be asserted in the same cycle. $\overline{\text{STERM}}$ has setup time (#60) and hold time (#61) requirements relative to each rising edge of the processor clock while AS is asserted. Bus error and retry terminations during burst cycles operate as described in **6.1.3.2 BURST MODE FILLING**, **7.5.1 Bus Error**, and **7.5.2 Retry Operation**.



For $\overline{\text{STERM}}$, the bus cycle terminations are summarized as follows (case numbers refer to Table 7-9):

Normal Termination:

$\overline{\text{STERM}}$ is asserted; $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ remain negated (case 1).

Halt Termination:

$\overline{\text{HALT}}$ is asserted before $\overline{\text{STERM}}$, and $\overline{\text{BERR}}$ remains negated (case 2).

Bus Error Termination:

$\overline{\text{BERR}}$ is asserted in lieu of, at the same time, or before $\overline{\text{STERM}}$ (case 3) or after $\overline{\text{STERM}}$ (case 4), and $\overline{\text{HALT}}$ remains negated; $\overline{\text{BERR}}$ is negated at the same time or after $\overline{\text{STERM}}$.

Retry Termination:

$\overline{\text{HALT}}$ and $\overline{\text{BERR}}$ are asserted in lieu of, at the same time, or before $\overline{\text{STERM}}$ (case 5) or after $\overline{\text{STERM}}$ (case 6); $\overline{\text{BERR}}$ is negated at the same time or after $\overline{\text{STERM}}$; $\overline{\text{HALT}}$ may be negated at the same time or after $\overline{\text{BERR}}$.

Table 7-9. $\overline{\text{STERM}}$, $\overline{\text{BERR}}$, and $\overline{\text{HALT}}$ Assertion Results

Case No.	Control Signal	Asserted on Rising Edge of State		Result
		N	N + 2	
1	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A NA NA	— — —	Normal cycle terminate and continue.
2	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	NA NA A/S	A NA S	Normal cycle terminate and halt. Continue when $\overline{\text{HALT}}$ negated.
3	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	NA A/S NA	A S NA	Terminate and take bus error exception, possibly deferred.
4	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A A NA	— — —	Terminate and take bus error exception, possibly deferred.
5	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	NA A A/S	A S S	Terminate and retry when $\overline{\text{HALT}}$ negated.
6	$\overline{\text{STERM}}$ $\overline{\text{BERR}}$ $\overline{\text{HALT}}$	A A A	— — —	Terminate and retry when $\overline{\text{HALT}}$ negated.

LEGEND:

- N — The number of current even bus state (e.g., S2, S4, etc.)
- A — Signal is asserted in this bus state
- NA — Signal is not asserted in this state
- X — Don't care
- S — Signal was asserted in previous state and remains asserted in this state
- — State N + 2 not part of bus cycle

EXAMPLE A:

A system uses a watchdog timer to terminate accesses to an unpopulated address space. The timer asserts $\overline{\text{BERR}}$ after timeout (case 3).

EXAMPLE B:

A system uses error detection and correction on RAM contents. The designer may:

1. Delay $\overline{\text{DSACKx}}$ until data is verified; assert $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ simultaneously to indicate to the processor to automatically retry the error cycle (case 5) or, if data is valid, assert $\overline{\text{DSACKx}}$ (case 1).
2. Delay $\overline{\text{DSACKx}}$ until data is verified and assert $\overline{\text{BERR}}$ with or without $\overline{\text{DSACKx}}$ if data is in error (case 3). This initiates exception processing for software handling of the condition.
3. Return $\overline{\text{DSACKx}}$ prior to data verification. If data is invalid, $\overline{\text{BERR}}$ is asserted on the next clock cycle (case 4). This initiates exception processing for software handling of the condition.
4. Return $\overline{\text{DSACKx}}$ prior to data verification; if data is invalid, assert $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ on the next clock cycle (case 6). The memory controller can then correct the RAM prior to or during the automatic retry.

7.5.1 Bus Errors

The bus error signal can be used to abort the bus cycle and the instruction being executed. $\overline{\text{BERR}}$ takes precedence over $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$ provided it meets the timing constraints described in MC68030EC/D, *MC68030 Electrical Specifications*. If $\overline{\text{BERR}}$ does not meet these constraints, it may cause unpredictable operation of the MC68030. If $\overline{\text{BERR}}$ remains asserted into the next bus cycle, it may cause incorrect operation of that cycle.

When the bus error signal is issued to terminate a bus cycle, the MC68030 may enter exception processing immediately following the bus cycle, or it may defer processing the exception. The instruction prefetch mechanism requests instruction words from the bus controller and the instruction cache before it is ready to execute them. If a bus error occurs on an instruction fetch, the processor does not take the exception until it attempts to use that instruction word. Should an intervening instruction cause a branch or should a task switch occur, the bus error exception does not occur.

The bus error signal is recognized during a bus cycle in any of the following cases:

- $\overline{\text{DSACKx}}$ (or $\overline{\text{STERM}}$) and $\overline{\text{HALT}}$ are negated and $\overline{\text{BERR}}$ is asserted.
- $\overline{\text{HALT}}$ and $\overline{\text{BERR}}$ are negated and $\overline{\text{DSACKx}}$ is asserted. $\overline{\text{BERR}}$ is then asserted within one clock cycle ($\overline{\text{HALT}}$ remains negated).
- $\overline{\text{BERR}}$ is asserted and recognized on the next falling clock edge following the rising clock edge on which $\overline{\text{STERM}}$ is asserted and recognized ($\overline{\text{HALT}}$ remains negated).

When the processor recognizes a bus error condition, it terminates the current bus cycle in the normal way. Figure 7-49 shows the timing of a bus error for the case in which neither $\overline{\text{DSACKx}}$ nor $\overline{\text{STERM}}$ is asserted. Figure 7-50 shows the timing for a bus error that is asserted after $\overline{\text{DSACKx}}$. Exceptions are taken in both cases. (Refer to **8.1.2 Bus Error Exception** for details of bus error exception processing.) When $\overline{\text{BERR}}$ is asserted during a read cycle that supplies data to either on-chip cache, the data in the cache is marked invalid. However, when a write cycle that writes data into the data cache results in an externally generated bus error, the data in the cache is not marked invalid.

In the second case, where $\overline{\text{BERR}}$ is asserted after $\overline{\text{DSACKx}}$ is asserted, $\overline{\text{BERR}}$ must be asserted within specification #48 (refer to MC68030EC/D, *MC68030 Electrical Specifications*) for purely asynchronous operation, or it must be asserted and remain stable during the sample window, defined by specifications #27A and #47B, around the next falling edge of the clock after $\overline{\text{DSACKx}}$ is recognized. If $\overline{\text{BERR}}$ is not stable at this time, the processor may exhibit erratic behavior. $\overline{\text{BERR}}$ has priority over $\overline{\text{DSACKx}}$. In this case, data may be present on the bus, but may not be valid. This sequence may be used by systems that have memory error detection and correction logic and by external cache memories.

The assertion of $\overline{\text{BERR}}$ described in the third case (recognized after $\overline{\text{STERM}}$) has requirements similar to those described in the preceding paragraph. $\overline{\text{BERR}}$ must be stable throughout the sample window for the next falling edge of the clock, as defined by specifications #27A and #28A. Figure 7-51 shows the timing for this case.

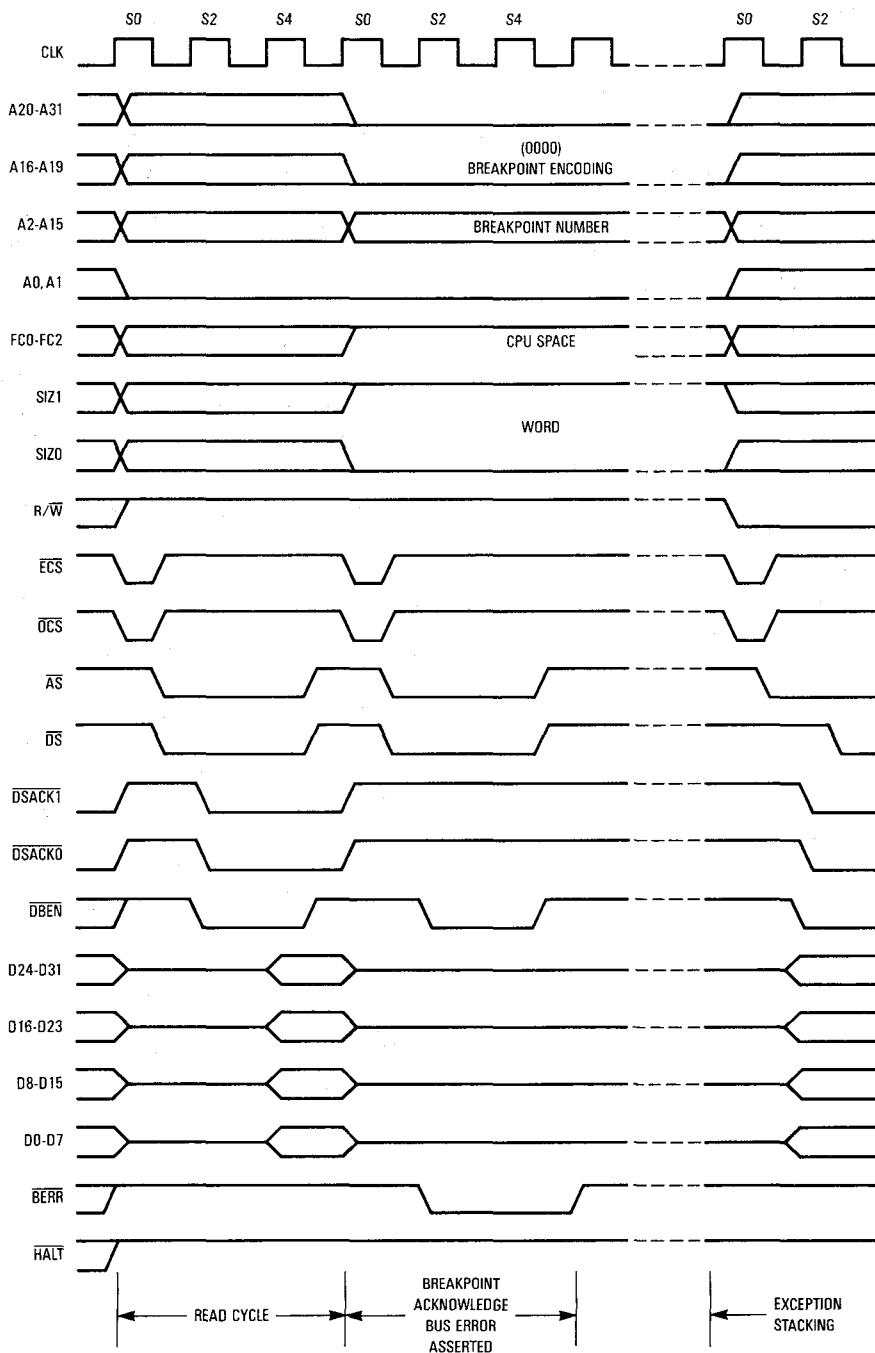


Figure 7-49. Bus Error without DSACKx

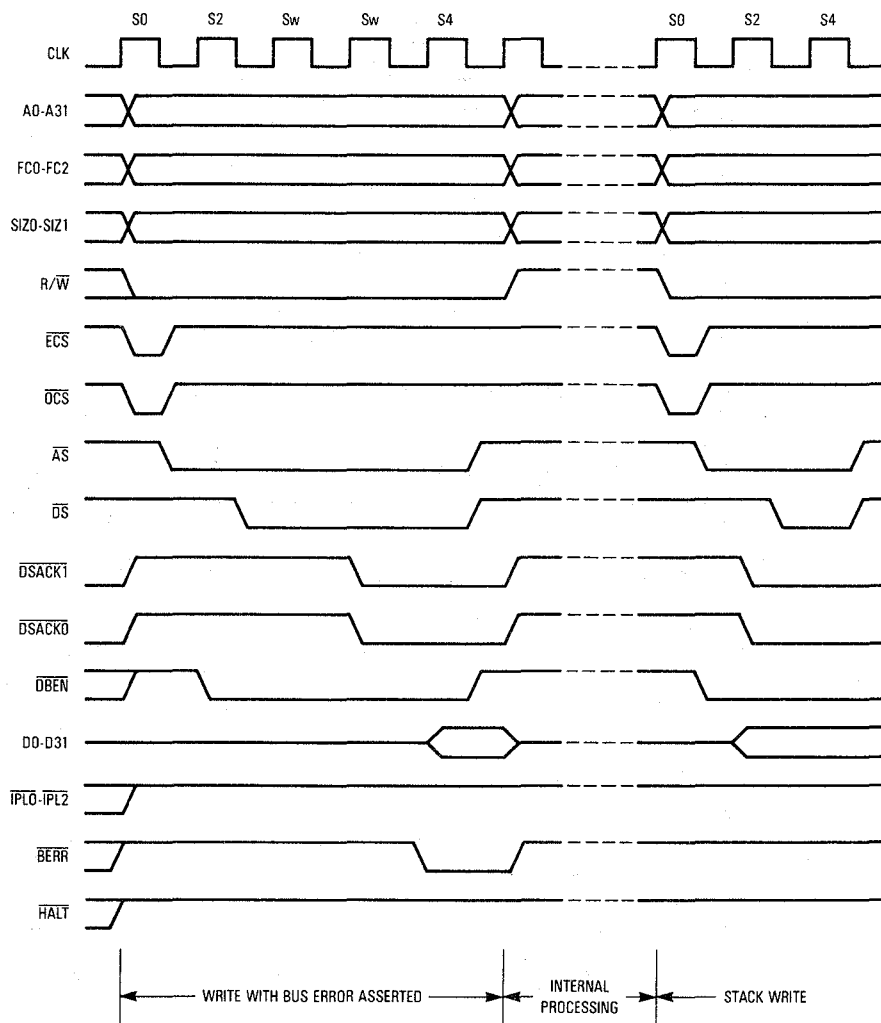


Figure 7-50. Late Bus Error with DSACKx

A bus error occurring during a burst fill operation is a special case. If a bus error occurs during the first cycle of a burst, the data is ignored, the entire cache line is marked invalid, and the burst operation is aborted. If the cycle is for an instruction fetch, a bus error exception is made pending. This bus error is processed only if the execution unit attempts to use either of the two

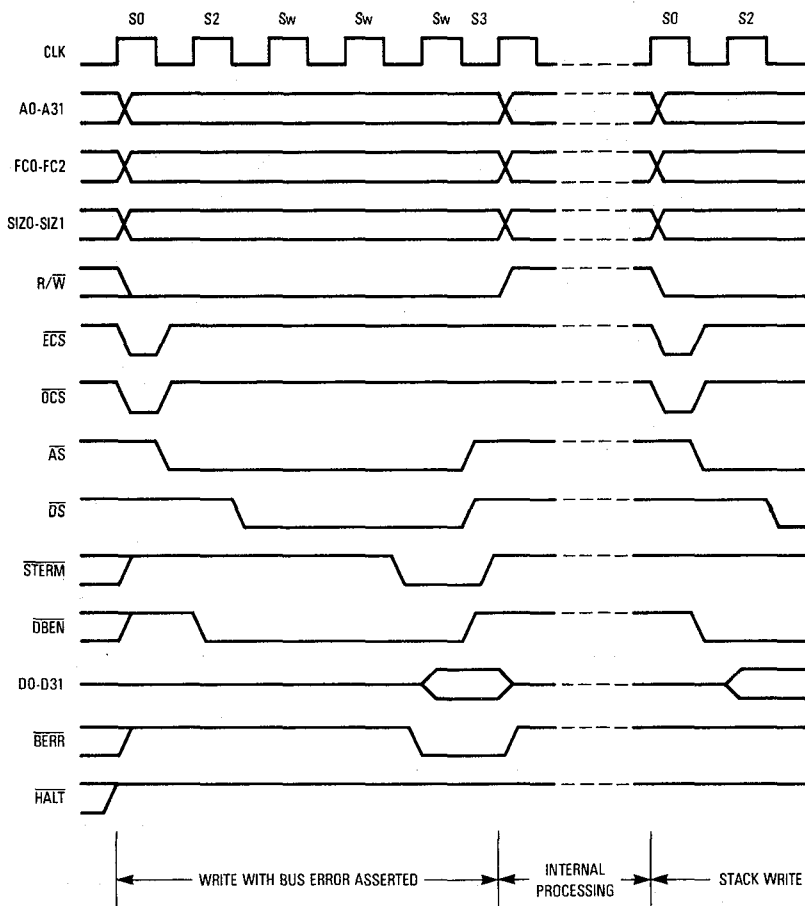


Figure 7-51. Late Bus Error with STERM — Exception Taken

words latched during the bus cycle. If the cycle is for a data fetch, the bus error exception is taken immediately. Refer to **SECTION 11 INSTRUCTION EXECUTION TIMING** for more information about pipeline operation.

When a bus error occurs after the burst mode has been entered (that is, on the second access or later), the processor terminates the burst operation, and the cache entry corresponding to that cycle is marked invalid, but the processor does not take an exception (see Figure 7-52). If the second cycle is for a portion of a misaligned operand fetch, the processor runs another

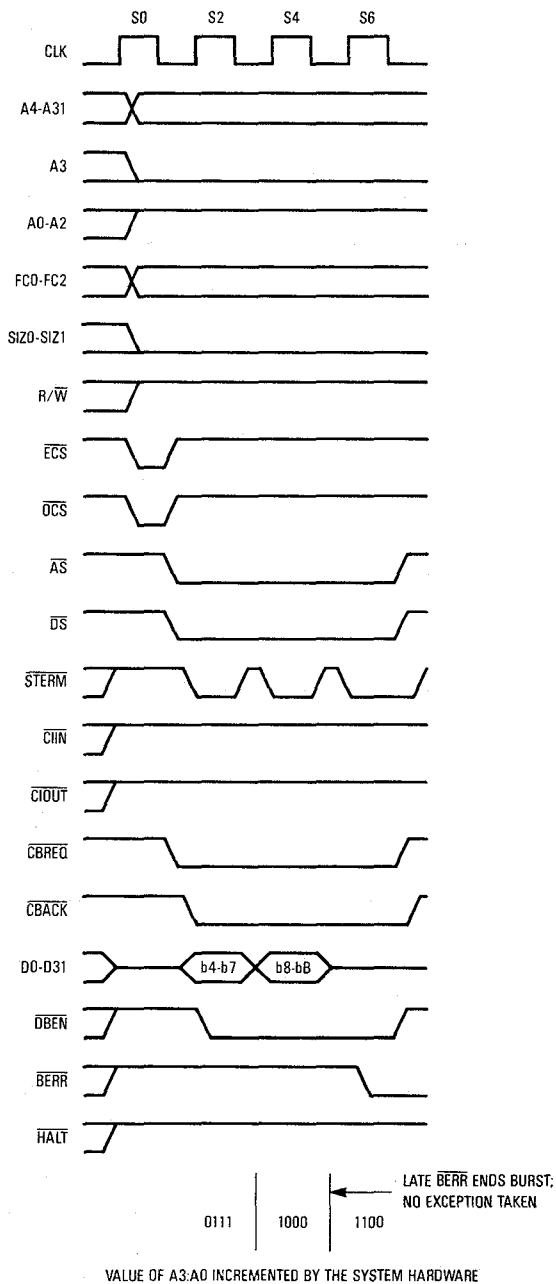


Figure 7-52. Long-Word Operand Request — Late $\overline{BERR}$ on Third Access

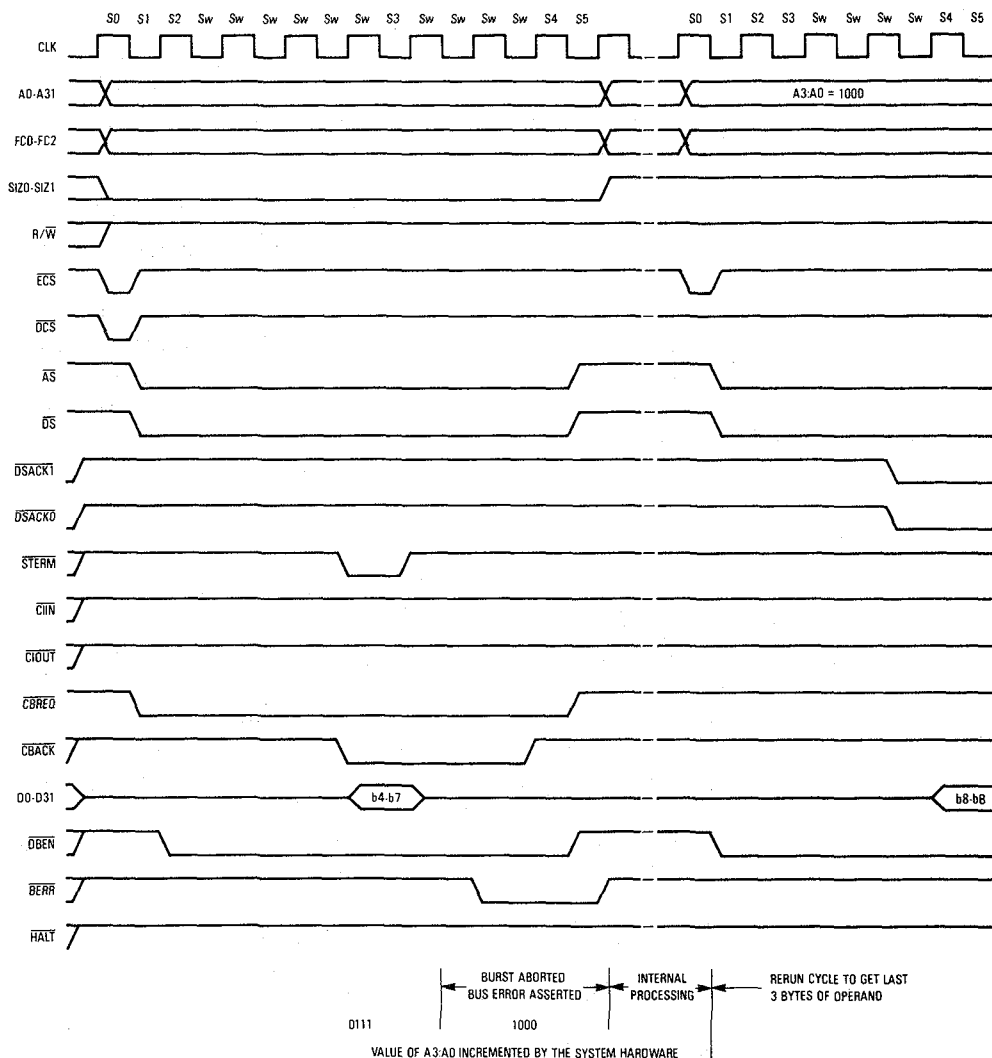


Figure 7-53. Long-Word Operand Request — $\overline{\text{BERR}}$ on Second Access

read cycle for the second portion with $\overline{\text{CBREQ}}$ negated, as shown in Figure 7-53. If $\overline{\text{BERR}}$ is asserted again, the MC68030 then takes an exception. The MC68030 supports late bus errors during a burst fill operation; the timing is the same relative to STERM and the clock as for a late bus error in a normal synchronous cycle.

7.5.2 Retry Operation

When the $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ signals are both asserted by an external device during a bus cycle, the processor enters the retry sequence. A delayed retry, similar to the delayed bus error signal described previously, can also occur, both for synchronous and asynchronous cycles.

The processor terminates the bus cycle, places the control signals in their inactive state, and does not begin another bus cycle until the $\overline{\text{HALT}}$ signal is negated by external logic. After a synchronization delay, the processor retries the previous cycle using the same access information (address, function code, size, etc.) The $\overline{\text{BERR}}$ signal should be negated before S2 of the read cycle to ensure correct operation of the retried cycle. Figure 7-54 shows a retry operation of an asynchronous cycle, and Figure 7-55 shows a retry operation of a synchronous cycle.

The processor retries any read or write cycle of a read-modify-write operation separately; $\overline{\text{RMC}}$ remains asserted during the entire retry sequence.

On the initial access of a burst operation, a retry (indicated by the assertion of $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$) causes the processor to retry the bus cycle and assert $\overline{\text{CBREQ}}$ again. Figure 7-56 shows a late retry operation that causes an initial burst operation to be repeated. However, signaling a retry with simultaneous $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ during the second, third, or fourth cycle of a burst operation does not cause a retry operation, even if the requested operand is misaligned. Assertion of $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ during a subsequent cycle of a burst operation causes independent $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ operations. The external bus activity remains halted until $\overline{\text{HALT}}$ is negated and the processor acts as previously described for the bus error during a burst operation.

Asserting $\overline{\text{BR}}$ along with $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ provides a relinquish and retry operation. The MC68030 does not relinquish the bus during a read-modify-write operation, except during the first read cycle. Any device that requires the processor to give up the bus and retry a bus cycle during a read-modify-write cycle must either assert $\overline{\text{BERR}}$ and $\overline{\text{BR}}$ only ($\overline{\text{HALT}}$ must not be included) or use the single wire arbitration method discussed in **7.7.4 Bus Arbitration Control**. The bus error handler software should examine the read-modify-write bit in the special status word (refer to **8.2.1 Special Status Word**) and take the appropriate action to resolve this type of fault when it occurs.

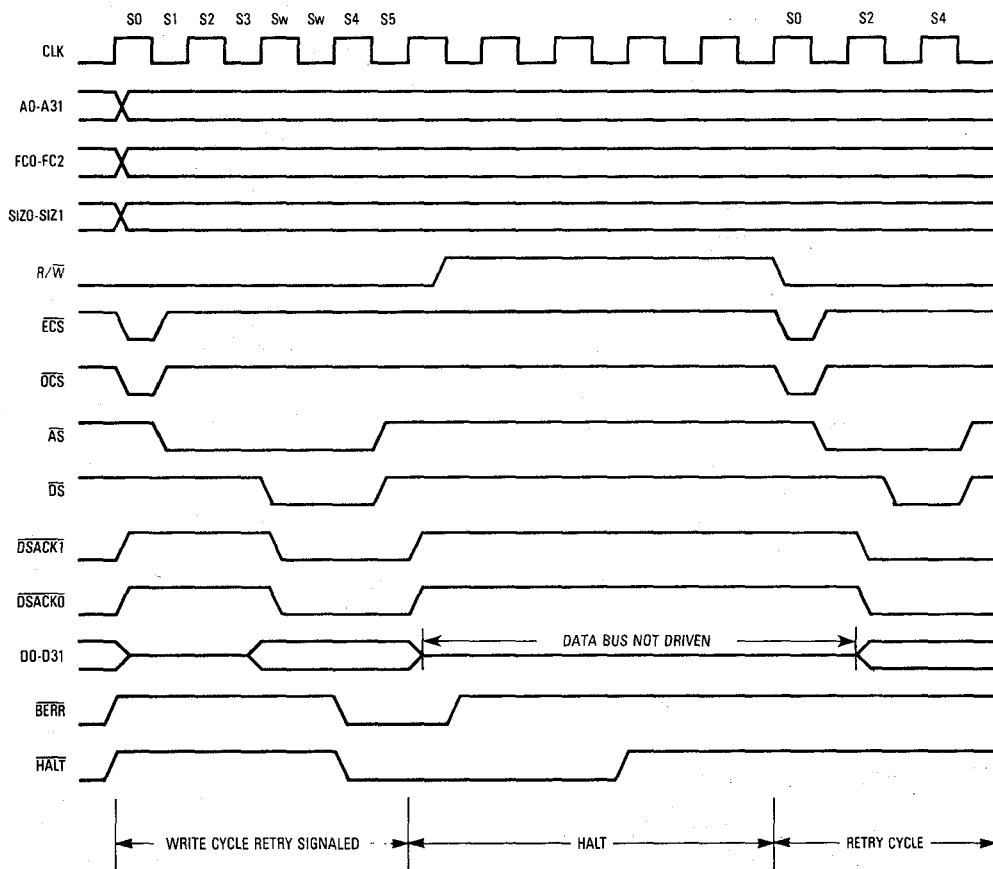


Figure 7-54. Asynchronous Late Retry

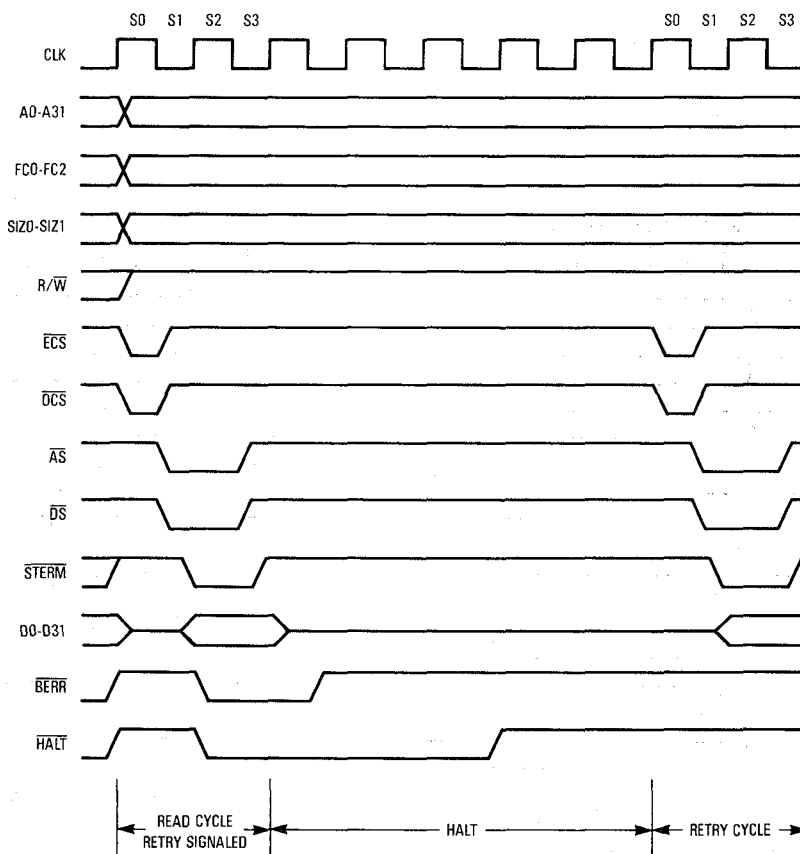


Figure 7-55. Synchronous Late Retry

7.5.3 Halt Operation

When $\overline{\text{HALT}}$ is asserted and $\overline{\text{BERR}}$ is not asserted, the MC68030 halts external bus activity at the next bus cycle boundary. $\overline{\text{HALT}}$ by itself does not terminate a bus cycle. Negating and reasserting $\overline{\text{HALT}}$ in accordance with the correct timing requirements provides a single-step (bus cycle to bus cycle) operation. The $\overline{\text{HALT}}$ signal affects external bus cycles only; thus, a program that resides in the instruction cache and performs no data writes (or reads that miss in the data cache) may continue executing, unaffected by the $\overline{\text{HALT}}$ signal.

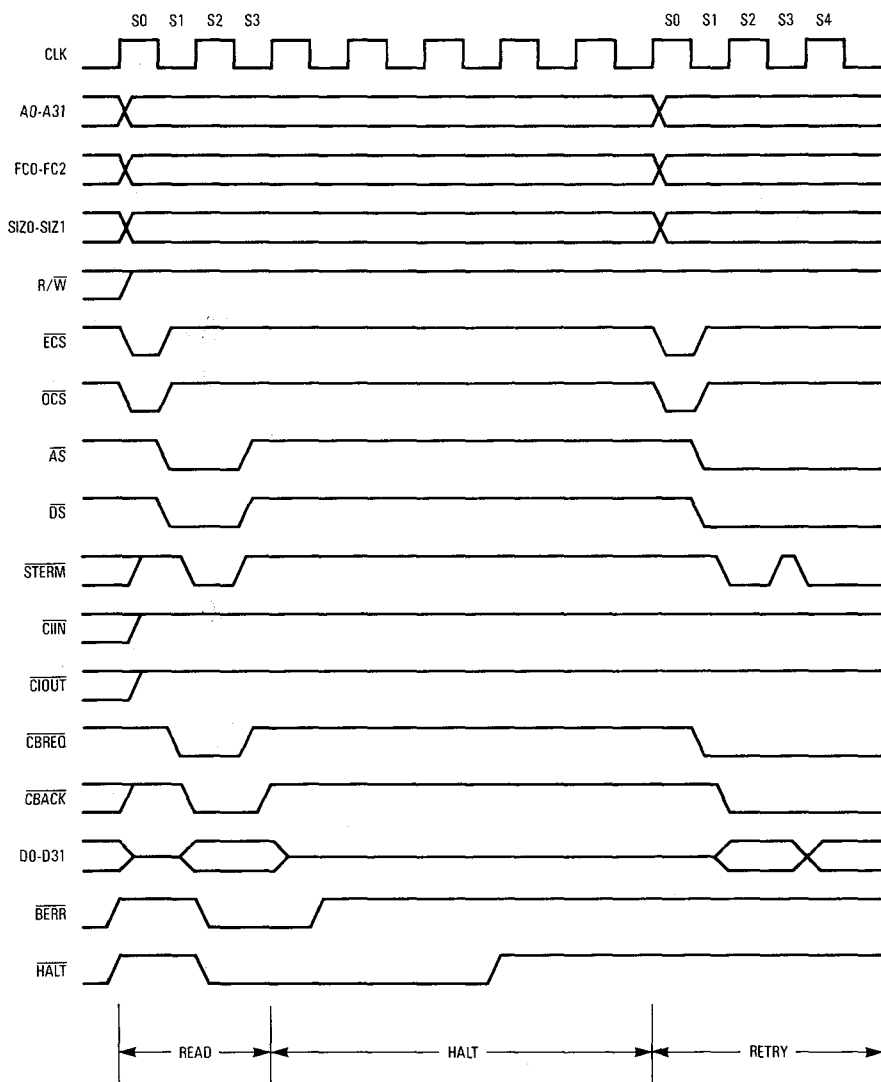


Figure 7-56. Late Retry Operation for a Burst

The single-cycle mode allows the user to proceed through (and debug) external processor operations, one bus cycle at a time. Figure 7-57 shows the timing requirements for a single-cycle operation. Since the occurrence of a bus error while $\overline{\text{HALT}}$ is asserted causes a retry operation, the user must anticipate retry cycles while debugging in the single-cycle mode. The single-

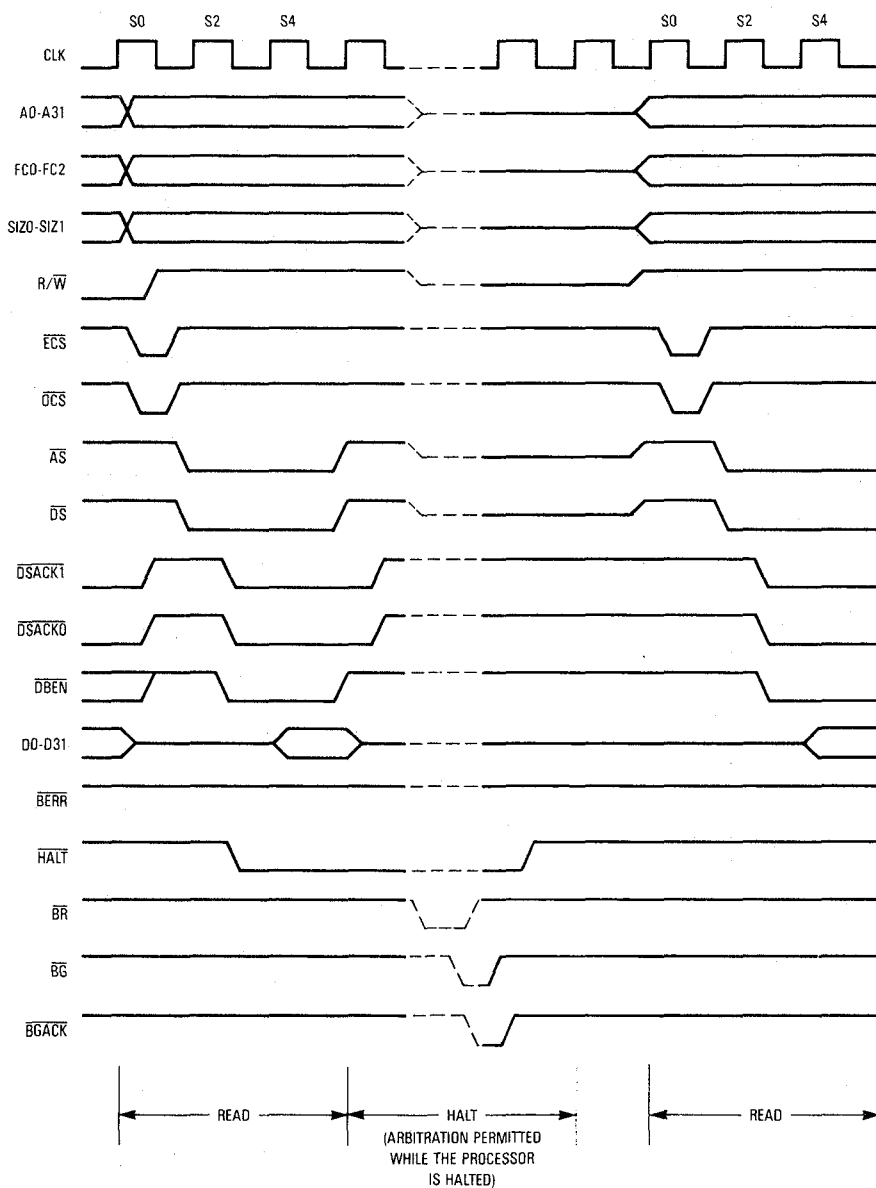


Figure 7-57. Halt Operation Timing

step operation and the software trace capability allow the system debugger to trace single bus cycles, single instructions, or changes in program flow. These processor capabilities, along with a software debugging package, give complete debugging flexibility.

When the processor completes a bus cycle with the $\overline{\text{HALT}}$ signal asserted, the data bus is placed in the high-impedance state, and bus control signals are driven inactive (not high-impedance state); the address, function code, size, and read/write signals remain in the same state. The halt operation has no effect on bus arbitration (refer to **7.7 BUS ARBITRATION**). When bus arbitration occurs while the MC68030 is halted, the address and control signals are also placed in the high-impedance state. Once bus mastership is returned to the MC68030, if $\overline{\text{HALT}}$ is still asserted, the address, function code, size, and read/write signals are again driven to their previous states. The processor does not service interrupt requests while it is halted, but it may assert the $\overline{\text{IPEND}}$ signal as appropriate.

7.5.4 Double Bus Fault

When a bus error or an address error occurs during the exception processing sequence for a previous bus error, a previous address error, or a reset exception, the bus or address error causes a double bus fault. For example, the processor attempts to stack several words containing information about the state of the machine while processing a bus error exception. If a bus error exception occurs during the stacking operation, the second error is considered a double bus fault. Only an external reset operation can restart a halted processor. However, bus arbitration can still occur (refer to **7.7 BUS ARBITRATION**).

The MC68030 indicates that a double bus fault condition has occurred by continuously asserting the $\overline{\text{STATUS}}$ signal until the processor is reset. The processor asserts $\overline{\text{STATUS}}$ for one, two, or three clock periods to signal other microsequencer status indications. Refer to **SECTION 12 APPLICATIONS INFORMATION** for a description of the interpretation of the $\overline{\text{STATUS}}$ signal.

A second bus error or address error that occurs after exception processing has completed (during the execution of the exception handler routine or later) does not cause a double bus fault. A bus cycle that is retried does not constitute a bus error or contribute to a double bus fault. The processor continues to retry the same bus cycle as long as the external hardware requests it.

7.0 BUS SYNCHRONIZATION

The MC68030 overlaps instruction execution; that is, during bus activity for one instruction, instructions that do not use the external bus can be executed. Due to the independent operation of the on-chip caches relative to the operation of the bus controller, many subsequent instructions can be executed, resulting in seemingly nonsequential instruction execution. When this is not desired and the system depends on sequential execution following bus activity, the NOP instruction can be used. The NOP instruction forces instruction and bus synchronization in that it freezes instruction execution until all pending bus cycles have completed.

An example of the use of the NOP instruction for this purpose is the case of a write operation of control information to an external register, where the external hardware attempts to control program execution based on the data that is written with the conditional assertion of BERR. If the data cache is enabled and the write cycle results in a hit in the data cache, the cache is updated. That data, in turn, may be used in a subsequent instruction before the external write cycle completes. Since the MC68030 cannot process the bus error until the end of the bus cycle, the external hardware has not successfully interrupted program execution. To prevent a subsequent instruction from executing until the external cycle completes, a NOP instruction can be inserted after the instruction causing the write. In this case, bus error exception processing proceeds immediately after the write before subsequent instructions are executed. This is an irregular situation, and the use of the NOP instruction for this purpose is not required by most systems.

Note that even in a system with error detection/correction circuitry, the NOP is not required for this synchronization. Since the MMU always checks the validity of write cycles before they proceed to the data cache and are executed externally, the MC68030 is guaranteed to write correct data to the cache. Thus, there is no danger in subsequent instructions using erroneous data from the cache before an external bus error signals an error.

A bus synchronization example is given in Figure 7-58.

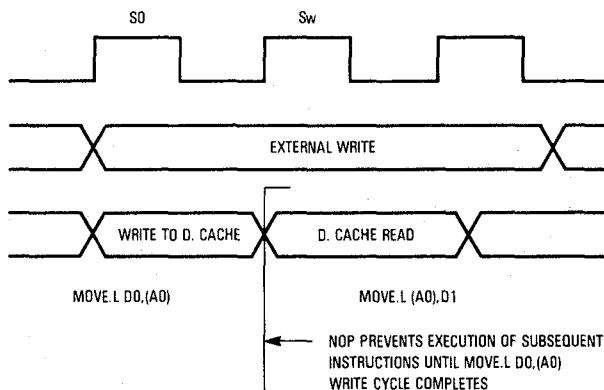


Figure 7-58. Bus Synchronization Example

7.7 BUS ARBITRATION

The bus design of the MC68030 provides for a single bus master at any one time: either the processor or an external device. One or more of the external devices on the bus can have the capability of becoming bus master. Bus arbitration is the protocol by which an external device becomes bus master; the bus controller in the MC68030 manages the bus arbitration signals so that the processor has the lowest priority. External devices that need to obtain the bus must assert the bus arbitration signals in the sequences described in the following paragraphs. Systems having several devices that can become bus master require external circuitry to assign priorities to the device so that, when two or more external devices attempt to become bus master at the same time, the one having the highest priority becomes bus master first. The sequence of the protocol is:

1. An external device asserts the bus request signal.
2. The processor asserts the bus grant signal to indicate that the bus will become available at the end of the current bus cycle.
3. The external device asserts the bus grant acknowledge signal to indicate that it has assumed bus mastership.

$\overline{BR}$ may be issued any time during a bus cycle or between cycles. $\overline{BG}$ is asserted in response to $\overline{BR}$; it is usually asserted as soon as $\overline{BR}$ has been synchronized and recognized, except when the MC68030 has made an internal decision to execute a bus cycle. Then, the assertion of $\overline{BG}$ is deferred until the bus cycle has begun. Additionally, $\overline{BG}$ is not asserted until the end of a read-modify-write operation (when $\overline{RMC}$ is negated) in response to a $\overline{BR}$

signal. When the requesting device receives $\overline{BG}$ and more than one external device can be bus master, the requesting device should begin whatever arbitration is required. The external device asserts $\overline{BGACK}$ when it assumes bus mastership and maintains $\overline{BGACK}$ during the entire bus cycle (or cycles) for which it is bus master. The following conditions must be met for an external device to assume mastership of the bus through the normal bus arbitration procedure:

- It must have received $\overline{BG}$ through the arbitration process.
- $\overline{AS}$ must be negated, indicating that no bus cycle is in progress, and the external device must ensure that all appropriate processor signals have been placed in the high-impedance state (by observing specification #7 in MC68030EC/D, *MC68030 Electrical Specifications*).
- The termination signal ($\overline{DSACKx}$ or $\overline{STERM}$) for the most recent cycle must have become inactive, indicating that external devices are off the bus (optional, refer to **7.7.3 Bus Grant Acknowledge**).
- $\overline{BGACK}$ must be inactive, indicating that no other bus master has claimed ownership of the bus.

Figure 7-59 is a flowchart showing the detail involved in bus arbitration for a single device. Figure 7-60 is a timing diagram for the same operation. This technique allows processing of bus requests during data transfer cycles.

The timing diagram shows that $\overline{BR}$ is negated at the time that $\overline{BGACK}$ is asserted. This type of operation applies to a system consisting of the processor and one device capable of bus mastership. In a system having a number of devices capable of bus mastership, the bus request line from each device can be wire-ORed to the processor. In such a system, more than one bus request can be asserted simultaneously.

The timing diagram in Figure 7-60 shows that $\overline{BG}$ is negated a few clock cycles after the transition of the $\overline{BGACK}$ signal. However, if bus requests are still pending after the negation of $\overline{BG}$, the processor asserts another $\overline{BG}$ within a few clock cycles after it was negated. This additional assertion of $\overline{BG}$ allows external arbitration circuitry to select the next bus master before the current bus master has finished with the bus. The following paragraphs provide additional information about the three steps in the arbitration process.

Bus arbitration requests are recognized during normal processing, $\overline{RESET}$ assertion, $\overline{HALT}$ assertion, and even when the processor has halted due to a double bus fault.

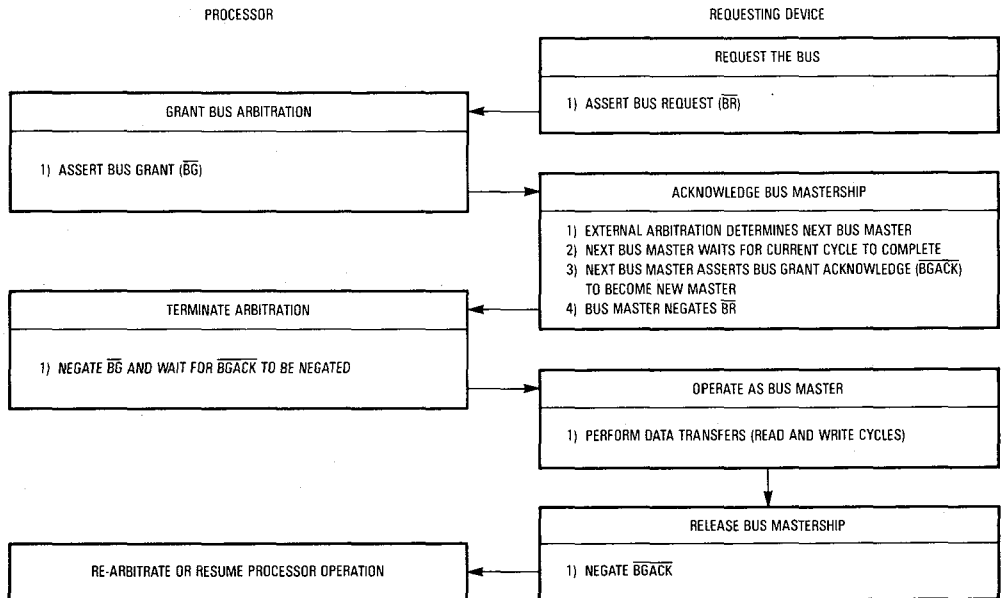


Figure 7-59. Bus Arbitration Flowchart for Single Request

7.7.1 Bus Request

External devices capable of becoming bus masters request the bus by asserting $\overline{BR}$. This can be a wire-ORed signal (although it need not be constructed from open-collector devices) that indicates to the processor that some external device requires control of the bus. The processor is effectively at a lower bus priority level than the external device and relinquishes the bus after it has completed the current bus cycle (if one has started).

If no acknowledge is received while the $\overline{BR}$ is active, the processor remains bus master once $\overline{BR}$ is negated. This prevents unnecessary interference with ordinary processing if the arbitration circuitry inadvertently responds to noise or if an external device determines that it no longer requires use of the bus before it has been granted mastership.

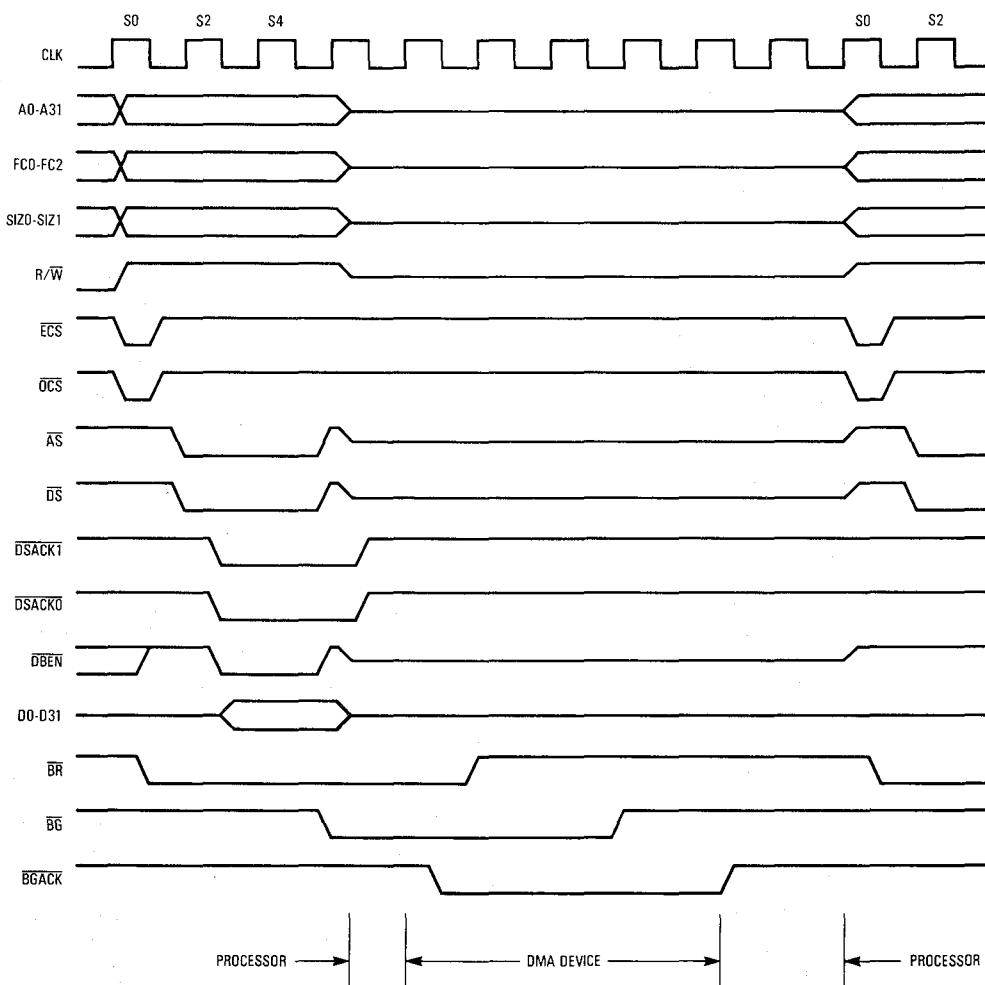


Figure 7-60. Bus Arbitration Operation Timing

7.7.2 Bus Grant

The processor asserts $\overline{BG}$ as soon as possible after receipt of $\overline{BR}$. This is immediately following internal synchronization except during a read-modify-write cycle or following an internal decision to execute a bus cycle. During a read-modify-write cycle, the processor does not assert $\overline{BG}$ until the entire operation has completed. $\overline{RMC}$ is asserted to indicate that the bus is locked. In the case an internal decision to execute another bus cycle, $\overline{BG}$ is deferred until the bus cycle has begun.

$\overline{BG}$ may be routed through a daisy-chained network or through a specific priority-encoded network. The processor allows any type of external arbitration that follows the protocol.

7.7.3 Bus Grant Acknowledge

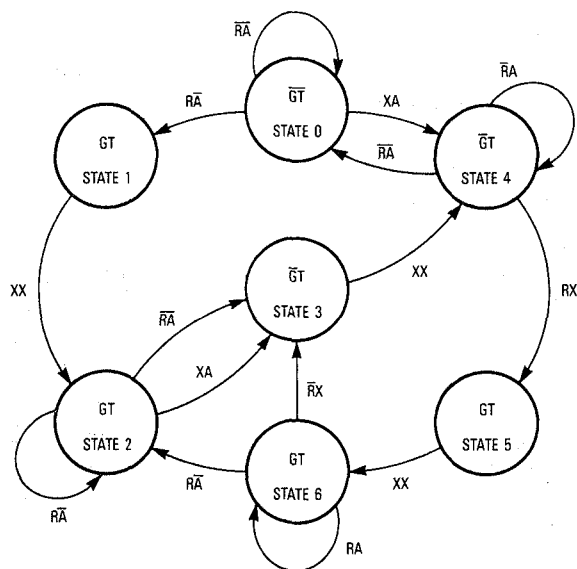
Upon receiving $\overline{BG}$, the requesting device waits until $\overline{AS}$, $\overline{DSACKx}$ (or synchronous termination, $\overline{STERM}$), and $\overline{BGACK}$ are negated before asserting its own $\overline{BGACK}$. The negation of the $\overline{AS}$ indicates that the previous master releases the bus after specification #7 (refer to MC68030EC/D, *MC68030 Electrical Specifications*). The negation of $\overline{DSACKx}$ or $\overline{STERM}$ indicates that the previous slave has completed its cycle with the previous master. Note that in some applications, $\overline{DSACKx}$ might not be used in this way.

General-purpose devices are then connected to be dependent only on $\overline{AS}$. When $\overline{BGACK}$ is asserted, the device is the bus master until it negates $\overline{BGACK}$. $\overline{BGACK}$ should not be negated until all bus cycles required by the alternate bus master are completed. Bus mastership terminates at the negation of $\overline{BGACK}$. The $\overline{BR}$ from the granted device should be negated after $\overline{BGACK}$ is asserted. If a $\overline{BR}$ is still pending after the assertion of $\overline{BGACK}$, another $\overline{BG}$ is asserted within a few clocks of the negation of $\overline{BG}$, as described in the **7.7.4 Bus Arbitration Control**. Note that the processor does not perform any external bus cycles before it reasserts $\overline{BG}$ in this case.

7.7.4 Bus Arbitration Control

The bus arbitration control unit in the MC68030 is implemented with a finite state machine. As discussed previously, all asynchronous inputs to the MC68030 are internally synchronized in a maximum of two cycles of the processor clock.

As shown in Figure 7-61, input signals labeled R and A are internally synchronized versions of the $\overline{BR}$ and $\overline{BGACK}$ signals, respectively. The $\overline{BG}$ output is labeled G, and the internal high-impedance control signal is labeled T. If T is true, the address, data, and control buses are placed in the high-impedance state after the next rising edge following the negation of $\overline{AS}$ and $\overline{RMC}$. All signals are shown in positive logic (active high), regardless of their true active voltage level.



R - BUS REQUEST
 A - BUS GRANT ACKNOWLEDGE
 G - BUS GRANT
 T - THREE-STATE CONTROL TO BUS CONTROL LOGIC
 X - DON'T CARE

NOTE: The $\overline{BG}$ output will not be asserted while $\overline{RMC}$ is asserted.

Figure 7-61. Bus Arbitration State Diagram

State changes occur on the next rising edge of the clock after the internal signal is valid. The $\overline{BG}$ signal transitions on the falling edge of the clock after a state is reached during which G changes. The bus control signals (controlled by T) are driven by the processor, immediately following a state change, when bus mastership is returned to the MC68030.

State 0, at the top center of the diagram, in which G and T are both negated, is the state of the bus arbiter while the processor is bus master. Request R and acknowledge A keep the arbiter in state 0 as long as they are both negated. When a request R is received, both grant G and signal T are asserted (in state 1 at the top left). The next clock causes a change to state 2, at the lower left, in which G and T are held. The bus arbiter remains in that state until acknowledge A is asserted or request R is negated. Once either occurs, the arbiter changes to the center state, state 3, and negates grant G. The next clock takes the arbiter to state 4, at the upper right, in which grant G remains

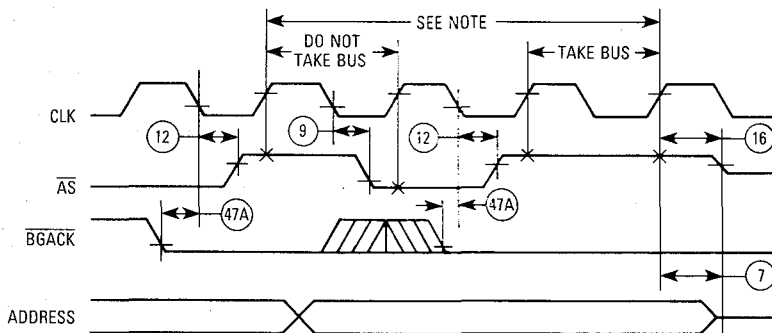
negated and signal T remains asserted. With acknowledge A asserted, the arbiter remains in state 4 until A is negated or request R is again asserted. When A is negated, the arbiter returns to the original state, state 0, and negates signal T. This sequence of states follows the normal sequence of signals for relinquishing the bus to an external bus master. Other states apply to other possible sequences of combinations of R and A. As shown by the path from state 0 to state 4, $\overline{\text{BGACK}}$ alone can be used to place the processor's external bus buffers in the high-impedance state, providing single-wire arbitration capability.

The read-modify-write sequence is normally indivisible to support semaphore operations and multiprocessor synchronization. During this indivisible sequence, the MC68030 asserts the $\overline{\text{RMC}}$ signal and causes the bus arbitration state machine to ignore bus requests (assertions of $\overline{\text{BR}}$) that occur after the first read cycle of the read-modify-write sequence by not issuing bus grants (asserting BG).

7

In some cases, however, it may be necessary to force the MC68030 to release the bus during an read-modify-write sequence. One way for an alternate bus master to force the MC68030 to release the bus applies only to the first read cycle of an read-modify-write sequence. The MC68030 allows normal bus arbitration during this read cycle; a normal relinquish and retry operation (asserting $\overline{\text{BERR}}$, $\overline{\text{HALT}}$, and $\overline{\text{BR}}$ at the same time) is used. Note that this method applies only to the first read cycle of the read-modify-write sequence, but this method preserves the integrity of the read-modify-write sequence without imposing any constraint on the alternate bus master.

A second method is single-wire arbitration, the timing of which is shown in Figure 7-62. An alternate master forces the MC68030 to release the bus by asserting $\overline{\text{BGACK}}$ and waits for $\overline{\text{AS}}$ to negate before taking the bus. It applies to all bus cycles of a read-modify-write sequence, but can cause system integrity problems if used improperly. The alternate bus master must guarantee the integrity of the read-modify-write sequence by not altering the contents of memory locations accessed by the read-modify-write sequence. Note that for the method to operate properly, $\overline{\text{AS}}$ must be observed to be negated (high) on two consecutive clock edges before the alternate bus master takes the bus. Waiting for this condition ensures that any current or pending bus activity has completed or has been pre-empted.



NOTE: The alternate bus master must sample $\overline{AS}$ high on two consecutive rising edges of the clock (after $\overline{BGACK}$ is recognized low) before taking the bus.

Figure 7-62. Single-Wire Bus Arbitration Timing Diagram

A timing diagram of the bus arbitration sequence during a processor bus cycle is shown in Figure 7-60. The bus arbitration sequence while the bus is inactive (i.e., executing internal operations such as a multiply instruction) is shown in Figure 7-63.

7

7.8 RESET OPERATION

$\overline{RESET}$ is a bidirectional signal with which an external device resets the system or the processor resets external devices. When power is applied to the system, external circuitry should assert $\overline{RESET}$ for a minimum of 520 clocks after V_{CC} is within tolerance. Figure 7-64 is a timing diagram of the powerup reset operation, showing the relationships between $\overline{RESET}$, V_{CC} , and bus signals. The clock signal is required to be stable by the time V_{CC} reaches the minimum operating specification. During the reset period, the entire bus three-states (except for non-three-statable signals, which are driven to their inactive state). Once $\overline{RESET}$ negates, all control signals are driven to their inactive state, the data bus is in read mode, and the address bus is driven. After this, the first bus cycle for reset exception processing begins.

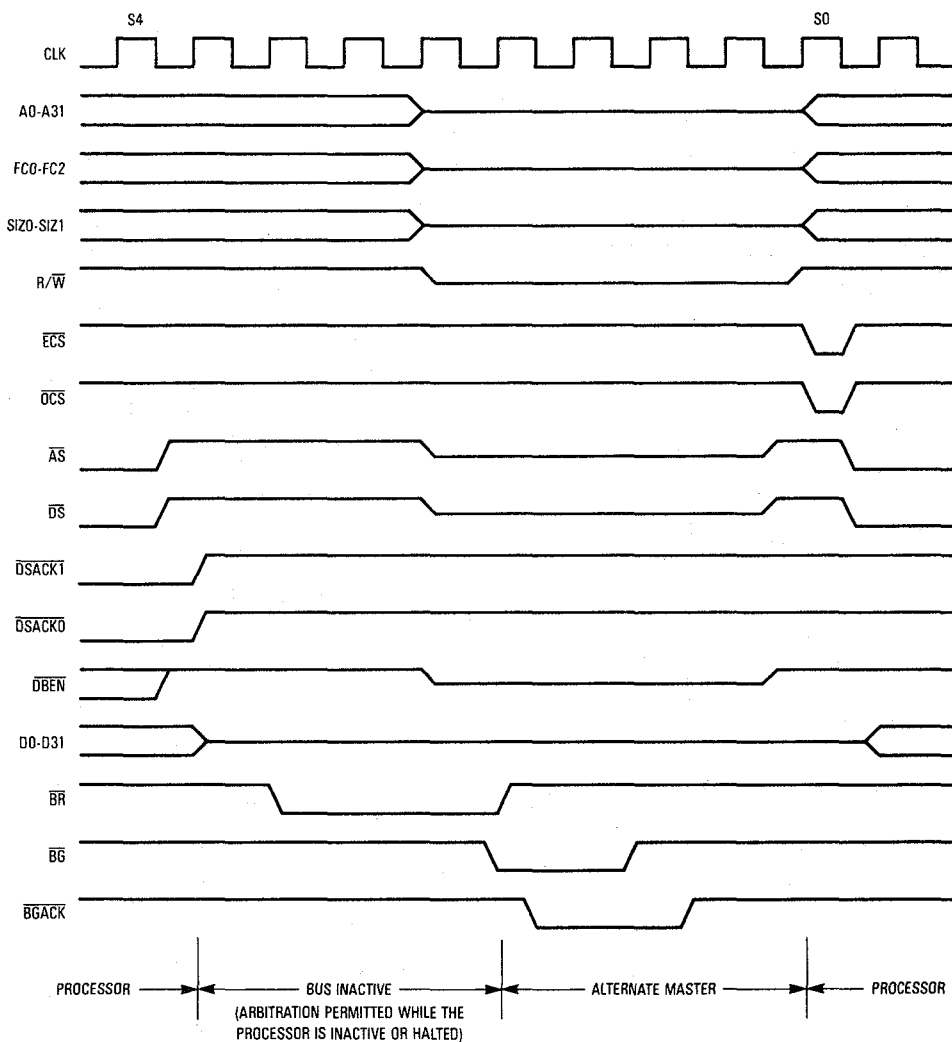


Figure 7-63. Bus Arbitration Operation (Bus Inactive)

The external $\overline{\text{RESET}}$ signal resets the processor and the entire system. Except for the initial reset, $\overline{\text{RESET}}$ should be asserted for at least 520 clock periods to ensure that the processor resets. Asserting $\overline{\text{RESET}}$ for 10 clock periods is sufficient for resetting the processor logic; the additional clock periods prevent a reset instruction from overlapping the external $\overline{\text{RESET}}$ signal.

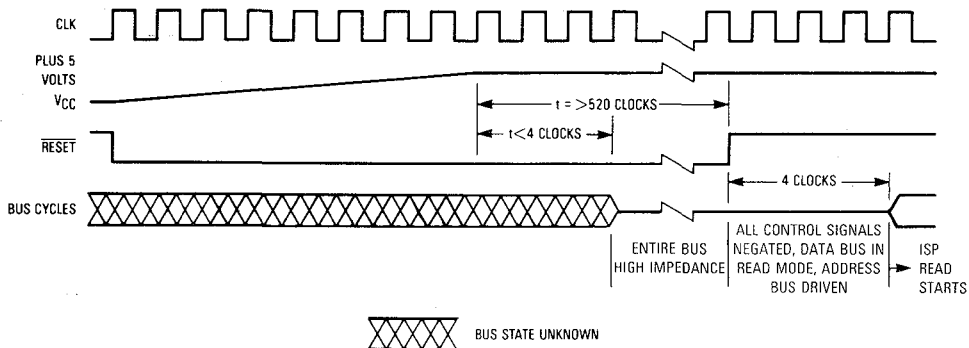


Figure 7-64. Initial Reset Operation Timing

Resetting the processor causes any bus cycle in progress to terminate as if $\overline{\text{DSACKx}}$, $\overline{\text{BERR}}$, or $\overline{\text{STERM}}$ had been asserted. In addition, the processor initializes registers appropriately for a reset exception. Exception processing for a reset operation is described in **8.1.1 Reset Exception**.

When a reset instruction is executed, the processor drives the $\overline{\text{RESET}}$ signal for 512 clock cycles. In this case, the processor resets the external devices of the system, and the internal registers of the processor are unaffected. The external devices connected to the $\overline{\text{RESET}}$ signal are reset at the completion of the reset instruction. An external $\overline{\text{RESET}}$ signal that is asserted to the processor during execution of a reset instruction must extend beyond the reset period of the instruction by at least eight clock cycles to reset the processor. Figure 7-65 shows the timing information for the reset instruction.

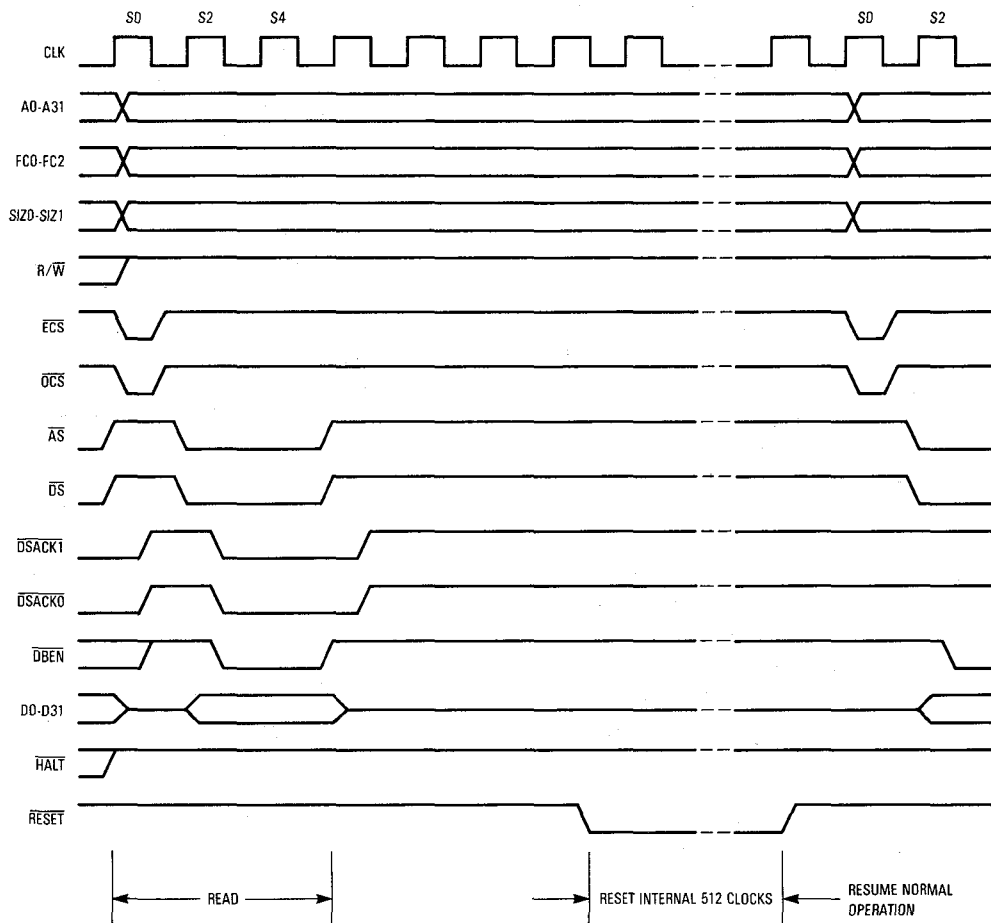


Figure 7-65. Processor-Generated Reset Operation